

```

    1);
  });

test("in a notebook the desk is trimmed away; on a whiteboard the area is kept as drawn", () => {
  const nb = notebook([page("a"), page("b")]);
  const drawn = { x: -200, y: 100, width: A4.width + 400, height: 300 };
  assert.deepEqual(clampCaptureRegion(nb, 0, drawn), { x: 0, y: 100, width: A4.width, height: 300 });
  assert.deepEqual(clampCaptureRegion(canvas([]), 0, drawn), drawn);
});

test("an area entirely on the desk, or a sliver, is not a capture", () => {
  const nb = notebook([page("a")]);
  assert.equal(clampCaptureRegion(nb, 0, { x: -300, y: 0, width: 200, height: 200 }), null);
  assert.equal(clampCaptureRegion(nb, 0, { x: 10, y: 10, width: 2, height: 200 }), null);
});

test("an area across the gap between two sheets takes both, at their stride", () => {
  const nb = notebook([page("a"), page("b"), page("c")], 1);
  // Anchored on page 2 (index 1): page 1 sits one stride above y = 0.
  const region = { x: 0, y: -100, width: 300, height: 200 };
  const hit = sheetsInRegion(nb, 1, region).map((s) => [s.page.id, s.offsetY]);
  assert.deepEqual(hit, [["a", -STRIDE], ["b", 0]]);
});

test("the notebook bounds follow the anchored page", () => {
  const nb = notebook([page("a"), page("b"), page("c")], 2);
  const r = clampCaptureRegion(nb, 2, { x: 0, y: -10_000, width: 50, height: 20_000 });
  assert.equal(r.y, -2 * STRIDE);
  assert.equal(r.height, 3 * A4.height + 2 * PAGE_GAP);
});

test("a selection is captured with a little air, constant on screen", () => {
  const els = [rect("a", 100, 100, 50, 50), rect("b", 300, 100, 10, 10)];
  const at1 = selectionCaptureRegion(els, ["a"], 1);
  const at2 = selectionCaptureRegion(els, ["a"], 2);
  assert.deepEqual(at1, { x: 86, y: 86, width: 78, height: 78 });
  assert.equal(at2.x, 93);
  assert.equal(selectionCaptureRegion(els, ["nope"], 1), null);
});

test("sharpness starts at what the screen showed, between 2x and 3x", () => {
  assert.equal(defaultCaptureScale(1, 1), 2);
  assert.equal(defaultCaptureScale(1, 2), 2);
  assert.equal(defaultCaptureScale(1.4, 2), 3);
  assert.equal(defaultCaptureScale(0.4, 2), 2);
  assert.equal(defaultCaptureScale(0, 0), 2);
});

test("a huge area is scaled down to what a canvas can hold", () => {
  const small = capturePixelSize({ x: 0, y: 0, width: 400, height: 300 }, 3);
  assert.deepEqual([small.width, small.height], [1200, 900]);
  const huge = capturePixelSize({ x: 0, y: 0, width: 8000, height: 8000 }, 3);
  assert.ok(huge.width * huge.height <= 40_000_000 + 16_000, `area ${huge.width * huge.height}`);
  assert.ok(huge.scale < 3);
});

test("selection only keeps a connector just when both its ends were picked", () => {
  const keep = captureKeep(["a", "b"]);
  const conn = (s: string, t: string) =>
    ({ id: `c${s}${t}`, type: "connection", source: { elementId: s }, target: { elementId: t } }) as unknown as
    s WhiteboardElement;
  assert.equal(keep(rect("a", 0, 0)), true);
  assert.equal(keep(rect("z", 0, 0)), false);
  assert.equal(keep(conn("a", "b")), true);
  assert.equal(keep(conn("a", "z")), false);
  assert.equal(captureKeep(undefined), undefined);
});

console.log(NL + "■■ Capture: the picture ■■");

function srcFor(board: Board, extra: Partial<CaptureSource> = {}): CaptureSource {
  return { board, region: { x: 0, y: 0, width: 200, height: 100 }, anchorIndex: 0, ...extra };
}

test("the SVG's viewBox IS the area", () => {
  const { svg, width, height } = buildCaptureSvg(
    srcFor(canvas([rect("a", 10, 10)]), { region: { x: -50, y: 25.5, width: 120, height: 80 } }),
    { background: "page", dark: false }
  );

```

```

    });
    assert.ok(svg.includes('viewBox="-50 25.5 120 80"'), svg.slice(0, 300));
    assert.deepEqual([width, height], [120, 80]);
  });

  test("'page' paints the sheet, its ruling and the PDF page; ink on top", () => {
    const nb = notebook([page("p1", [rect("ink", 20, 20)], { pdf: { docId: "d", pageNumber: 1 } })]);
    const { svg } = buildCaptureSvg(srcFor(nb), { background: "page", dark: false }, [], undefined, new Map([["p1", ""]]]));
    assert.ok(svg.includes("<pattern\"", "ruling"));
    assert.ok(svg.includes("", "PDF page"));
    assert.ok(svg.includes('preserveAspectRatio="xMidYMid meet"', "PDF letterboxed like the canvas"));
    assert.ok(svg.indexOf("") < svg.indexOf("#112233"), "ink over the page");
  });

  test("'white' drops ruling and paper colour but keeps the PDF page", () => {
    const nb = notebook([page("p1", [rect("ink", 20, 20)], { pdf: { docId: "d", pageNumber: 1 } })]);
    const { svg } = buildCaptureSvg(srcFor(nb), { background: "white", dark: true }, [], undefined, new Map([["p1", "data:x"]]]));
    assert.ok(!svg.includes("<pattern\""));
    assert.ok(svg.includes('fill="#ffffff"'));
    assert.ok(svg.includes("data:x"));
  });

  test("'none' leaves only the ink: no paper, no ruling, no PDF page", () => {
    const nb = notebook([page("p1", [rect("ink", 20, 20)], { pdf: { docId: "d", pageNumber: 1 } })]);
    const { svg, report } = buildCaptureSvg(srcFor(nb), { background: "transparent", dark: false }, [], undefined, new Map([["p1", "data:x"]]]));
    assert.ok(!svg.includes("<pattern\""));
    assert.ok(!svg.includes("data:x"));
    assert.ok(!/<rect x="0" y="0" width="\d+" height="\d+" fill=/.test(svg), "no sheet fill");
    assert.ok(svg.includes("#112233"));
    assert.equal(report.missingPdfPages, 0, "a PDF left out on purpose is not 'missing'");
  });

  test("a PDF page that could not be loaded is reported", () => {
    const nb = notebook([page("p1", [], { pdf: { docId: "d", pageNumber: 1 } })]);
    assert.equal(buildCaptureSvg(srcFor(nb), { background: "page", dark: false }).report.missingPdfPages, 1);
  });

  test("a freeform loop masks the picture", () => {
    const polygon = [{ x: 0, y: 0 }, { x: 100, y: 0 }, { x: 50, y: 80 }];
    const { svg } = buildCaptureSvg(srcFor(canvas([rect("a", 10, 10)], { polygon }, { background: "white", dark: false })));
    assert.ok(svg.includes('<clipPath id="pn-capture-mask"><polygon points="0,0 100,0 50,80' />'));
    assert.ok(svg.includes('clip-path="url(#pn-capture-mask)"'));
  });

  test("selection only draws just the chosen elements", () => {
    const board = canvas([rect("a", 10, 10), { ...rect("b", 40, 10), strokeColor: "#abcdef" } as WhiteboardElement]);
    const { svg } = buildCaptureSvg(srcFor(board, { onlyIds: ["a"] }), { background: "transparent", dark: false });
    assert.ok(svg.includes("#112233"));
    assert.ok(!svg.includes("#abcdef"));
  });

  test("the canvas' own text markup and embedded fonts are used when given", () => {
    const board = canvas([rect("t", 0, 0), type: "text", text: "hello" } as unknown as WhiteboardElement]);
    const live = { markup: new Map([["t", "<g id=\"LIVE\"/>"]]), fontCss: "@font-face{font-family:X}" };
    const { svg } = buildCaptureSvg(srcFor(board), { background: "page", dark: false }, [], undefined, undefined, live);
    assert.ok(svg.includes('<g id="LIVE"/>'));
    assert.ok(svg.includes("<![CDATA[");
    assert.ok(svg.includes("@font-face{font-family:X}"));
    assert.ok(!svg.includes(">hello<"), "the serializer's own text is not drawn as well");
  });

  test("a whiteboard with no colour of its own captures the theme's background", () => {
    const board = canvas([]);
    assert.ok(buildCaptureSvg(srcFor(board), { background: "page", dark: false }).svg.includes('fill="#ffffff"'));
    assert.ok(buildCaptureSvg(srcFor(board), { background: "page", dark: true }).svg.includes('fill="#0a0a0a"'));
  });

  test("file names are safe and distinct to the second", () => {
    const when = new Date(2026, 8, 29, 14, 3, 7);
  });

```

```

    assert.equal(captureFilename("Physics / notes: ch.1", when), "Physics__notes__ch_1_capture_2026-09-29_140307.png");
    assert.equal(captureFilename("", when), "capture_capture_2026-09-29_140307.png");
  });

  console.log(NL + "■■■ Capture: where it lands ■■■");

  test("on a fresh page: its own size, centred at the top, only ever shrunk", () => {
    assert.deepEqual(placeCaptureOnPage({ width: 800, height: 1000 }, { width: 200, height: 100 }), {
      x: 300, y: 48, width: 200, height: 100,
    });
    const big = placeCaptureOnPage({ width: 800, height: 1000 }, { width: 1408, height: 704 });
    assert.equal(Math.round(big.width), 704);
    assert.equal(Math.round(big.height), 352);
  });

  test("on a whiteboard: beside what is already there, or at the origin", () => {
    assert.deepEqual(placeCaptureBeside([rect("a", 0, 10, 100, 50)], { width: 30, height: 20 }), {
      x: 180, y: 10, width: 30, height: 20,
    });
    assert.deepEqual(placeCaptureBeside([], { width: 30, height: 20 }), { x: -15, y: -10, width: 30, height: 20 });
  });

  const S = () => useWhiteboard.getState();
  const img = { src: "asset:img_x", naturalWidth: 400, naturalHeight: 200, size: { width: 200, height: 100 } };

  test("'new page' inserts after the current page and one undo removes it", () => {
    const id = S().createNotebook("Cap NB", null);
    S().setActiveBoard(id);
    S().addPage();
    const b0 = S().boards.find((b) => b.id === id!);
    const first = getPages(b0)[0].id;
    S().setActivePage(first);
    const res = S().addCaptureToBoard(id, { ...img, afterPageId: first });
    const b1 = S().boards.find((b) => b.id === id!);
    assert.equal(getPages(b1).length, 3);
    assert.equal(getPages(b1)[1].id, res.pageId);
    assert.equal(res.pageNumber, 2);
    const el = getPages(b1)[1].elements[0] as WhiteboardElement & { src: string; locked?: boolean };
    assert.equal(el.src, "asset:img_x");
    assert.ok(!el.locked, "a clipping can be moved");
    S().undo();
    const b2 = S().boards.find((b) => b.id === id!);
    assert.equal(getPages(b2).length, 2);
    assert.equal(getActivePage(b2)!.id, first);
  });

  test("adding to ANOTHER document is undone THERE, never from where you captured", () => {
    const here = S().createNotebook("Here", null);
    const there = S().createBoard("There", null);
    S().setActiveBoard(there);
    S().addElement(rect("existing", 0, 0, 100, 100));
    S().setActiveBoard(here);
    const res = S().addCaptureToBoard(there, img);
    const t1 = S().boards.find((b) => b.id === there!);
    const placed = getBoardElements(t1).find((e) => e.id === res.elementId!);
    assert.ok(placed.x >= 100, "placed beside the existing content");
    assert.equal(S().activeBoardId, here, "the current document stays current");
    // Undo here must not reach into the other document: its whole-document
    // snapshot would wipe anything written there since.
    S().undo();
    const t2 = S().boards.find((b) => b.id === there!);
    assert.ok(getBoardElements(t2).some((e) => e.id === res.elementId), "undo here leaves the other document alone");
    // Work done in the other document afterwards survives, and Undo there takes
    // the steps back in order: the later edit first, then the capture.
    S().setActiveBoard(there);
    S().addElement(rect("later", 400, 0, 50, 50));
    S().undo();
    const t3 = S().boards.find((b) => b.id === there!);
    assert.deepEqual(getBoardElements(t3).map((e) => e.id).sort(), ["existing", res.elementId].sort());
    S().undo();
    const t4 = S().boards.find((b) => b.id === there!);
    assert.deepEqual(getBoardElements(t4).map((e) => e.id), ["existing"]);
  });

  test("an unknown document adds nothing", () => {

```

```

    assert.equal(S().addCaptureToBoard("no-such-board", img), null);
  });

  console.log(NL + "■■■ Capture: mode and clipboard ■■■");

  test("screenshot mode only takes over the select tool's gestures", () => {
    assert.equal(screenshotShapeFor(true, "select"), "rectangle");
    assert.equal(screenshotShapeFor(true, "lasso"), "lasso");
    assert.equal(screenshotShapeFor(true, "gel"), null);
    assert.equal(screenshotShapeFor(true, "laser"), null);
    assert.equal(screenshotShapeFor(false, "select"), null);
  });

  test("the app clipboard stands in only when the system refused the copy", () => {
    clearAppClip();
    assert.equal(appClipForPaste(), null);
    putAppClipImage({ dataUrl: "data:a", width: 2, height: 2, systemCopied: true });
    assert.equal(appClipForPaste(), null, "the system clipboard has it: it is the truth");
    putAppClipImage({ dataUrl: "data:b", width: 2, height: 2, systemCopied: false });
    assert.equal(appClipForPaste()?.dataUrl, "data:b");
    clearAppClip();
  });

  console.log(` ${passed} passed, ${failed} failed`);
  if (failed > 0) process.exit(1);

  =====
  // === END FILE: tests/capture.test.ts ===
  =====

  =====
  // === START FILE: tests/challenger_2_empirical.test.ts ===
  // File 138 of 194 | Lines: 461 | Size: 23372 bytes
  =====

  /**
   * CHALLENGER 2 EMPIRICAL TEST SUITE
   * =====
   *
   * Exhaustive Stress Harnesses covering:
   * 1. Smart Connector 3 routing modes (Curved Bezier, Orthogonal step, Straight) under extreme coordinates and
   *    intersecting nodes.
   * 2. Board duplication ID remapping integrity across 100 duplicated boards with dense interconnected node gra
   *    phs.
   * 3. 50-step circular undo/redo history stack stress test (overflows, boundary edge cases, branching, deep re
   *    stores).
   */

  import {
    getCurvedBezierPath,
    getOrthogonalPath,
    getStraightPath,
    getConnectionPath,
    getArrowheadPath,
    getNodeAnchorPorts,
    getResolvedAnchorPoint,
    getOptimalConnectionAnchors,
    getElementBounds,
    type AnchorPoint,
  } from "../src/lib/whiteboard/geometry";
  import { useWhiteboard } from "../src/store/whiteboard";
  import type {
    Board,
    ConnectionElement,
    RectangleElement,
    StickyElement,
    TextElement,
    StarElement,
    CloudElement,
    EllipseElement,
    DiamondElement,
    WhiteboardElement,
  } from "../src/lib/whiteboard/types";

  let passedAssertions = 0;
  let totalAssertions = 0;

```

```

function assert(condition: boolean, message: string) {
  totalAssertions++;
  if (!condition) {
    console.error(■ FAILED ASSERTION [`${totalAssertions}]: ${message}`);
    throw new Error(`Assertion failed: ${message}`);
  }
  passedAssertions++;
}

console.log("=====");
console.log("=== RUNNING CHALLENGER 2 EMPIRICAL ADVERSARIAL STRESS SUITE ===");
console.log("=====\\n");

// =====
// SECTION 1: SMART CONNECTOR 3 ROUTING MODES UNDER EXTREME COORDINATES & OVERLAPS
// =====
console.log("[Section 1] Smart Connector 3 Routing Modes & Intersecting Nodes Stress...");

// 1.1 Extreme Coordinate Values & Numerical Stability
const extremeCoordinates = [
  -1e7, -1e6, -99999.99, -1000, -1, -0.0001, 0, 0.0001, 1, 1000, 99999.99, 1e6, 1e7,
];

for (const x1 of [-1e6, -500, 0, 500, 1e6]) {
  for (const y1 of [-1e6, -500, 0, 500, 1e6]) {
    for (const x2 of [-1e6, -200, 0, 200, 1e6]) {
      for (const y2 of [-1e6, -200, 0, 200, 1e6]) {
        const p1: AnchorPoint = { x: x1, y: y1, nx: 1, ny: 0, anchor: "right" };
        const p2: AnchorPoint = { x: x2, y: y2, nx: -1, ny: 0, anchor: "left" };

        const curved = getCurvedBezierPath(p1, p2);
        const ortho = getOrthogonalPath(p1, p2);
        const straight = getStraightPath(p1, p2);

        // Verify all 3 paths return non-empty strings with NO NaN, Infinity, or undefined
        assert(typeof curved === "string" && curved.startsWith("M"), `Curved path valid string for (${x1},${y1})->(${x2},${y2})`);
        assert(!curved.includes("NaN") && !curved.includes("undefined") && !curved.includes("Infinity"), `Curved path has no NaN/Infinity`);

        assert(typeof ortho === "string" && ortho.startsWith("M"), `Ortho path valid string for (${x1},${y1})->(${x2},${y2})`);
        assert(!ortho.includes("NaN") && !ortho.includes("undefined") && !ortho.includes("Infinity"), `Ortho path has no NaN/Infinity`);

        assert(typeof straight === "string" && straight.startsWith("M"), `Straight path valid string for (${x1},${y1})->(${x2},${y2})`);
        assert(!straight.includes("NaN") && !straight.includes("undefined") && !straight.includes("Infinity"), `Straight path has no NaN/Infinity`);
      }
    }
  }
}

console.log(" ✓ Extreme coordinates numerical stability verified (no NaN / Infinity / undefined)");

// 1.2 Boundary Distances (Zero, Sub-pixel, Exact threshold values)
const boundaryDistances = [0, 0.00001, 0.5, 1.99, 2.0, 2.01, 3.99, 4.0, 4.01, 10, 50];
for (const dist of boundaryDistances) {
  const p1: AnchorPoint = { x: 100, y: 100, nx: 1, ny: 0, anchor: "right" };
  const p2: AnchorPoint = { x: 100 + dist, y: 100, nx: -1, ny: 0, anchor: "left" };

  const curved = getCurvedBezierPath(p1, p2);
  const ortho = getOrthogonalPath(p1, p2);
  const straight = getStraightPath(p1, p2);

  if (dist < 2) {
    // Curved Bezier falls back to straight line when dist < 2
    assert(curved === `M 100,100 L ${100 + dist},100`, `Curved Bezier falls back to linear L on dist < 2`);
  } else {
    assert(curved.includes("C"), `Curved Bezier uses cubic C for dist >= 2`);
  }

  if (dist < 4) {
    // Orthogonal falls back to straight line when dist < 4
    assert(ortho === `M 100,100 L ${100 + dist},100`, `Orthogonal falls back to linear L on dist < 4`);
  } else {
    assert(ortho.includes("L"), `Orthogonal path contains line segments`);
  }
}

```

```
    assert(straight === `M 100,100 L ${100 + dist},100`, `Straight path exact linear segment`);
  }
  console.log(" ✓ Boundary distance thresholds (<2px, <4px, sub-pixel, 0px) verified");

// 1.3 Intersecting, Overlapping & Concentric Nodes
const concentricSource: RectangleElement = {
  id: "node-source",
  type: "rectangle",
  x: 0,
  y: 0,
  width: 200,
  height: 200,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  fillColor: "#fff",
  fillStyle: "solid",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: Date.now(),
  updatedAt: Date.now(),
};

const concentricTarget: RectangleElement = {
  id: "node-target",
  type: "rectangle",
  x: 50,
  y: 50,
  width: 100,
  height: 100,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  fillColor: "#fff",
  fillStyle: "solid",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: Date.now(),
  updatedAt: Date.now(),
};

// Target inside source (concentric)
const concentricAnchors = getOptimalConnectionAnchors(concentricSource, concentricTarget, "auto", "auto");
assert(concentricAnchors.source !== undefined && concentricAnchors.target !== undefined, "Concentric nodes and
  hor resolution");
const concentricCurved = getConnectionPath(concentricAnchors.source, concentricAnchors.target, "curved");
const concentricOrtho = getConnectionPath(concentricAnchors.source, concentricAnchors.target, "orthogonal");
const concentricStraight = getConnectionPath(concentricAnchors.source, concentricAnchors.target, "straight");

assert(!concentricCurved.includes("NaN"), "Concentric curved path valid");
assert(!concentricOrtho.includes("NaN"), "Concentric ortho path valid");
assert(!concentricStraight.includes("NaN"), "Concentric straight path valid");

// 1.4 Rotated & Overlapping Nodes
for (const rotDeg of [0, 30, 45, 90, 135, 180, 270, 315]) {
  const rotRad = (rotDeg * Math.PI) / 180;
  const rotatedSource: StickyElement = {
    id: `rot-s-${rotDeg}`,
    type: "sticky",
    x: 100,
    y: 100,
    width: 150,
    height: 150,
    rotation: rotRad,
    opacity: 1,
    text: "Source",
    color: "yellow",
    fontSize: 16,
    align: "left",
    createdAt: Date.now(),
    updatedAt: Date.now(),
  };

  const rotatedTarget: StickyElement = {
    id: `rot-t-${rotDeg}`,
    type: "sticky",
    x: 180, // Overlapping partially with source
  };
}
```

```

    y: 120,
    width: 150,
    height: 150,
    rotation: -rotRad,
    opacity: 1,
    text: "Target",
    color: "blue",
    fontSize: 16,
    align: "left",
    createdAt: Date.now(),
    updatedAt: Date.now(),
  });

const anchors = getOptimalConnectionAnchors(rotatedSource, rotatedTarget, "auto", "auto");
const portsSource = getNodeAnchorPorts(rotatedSource);
const portsTarget = getNodeAnchorPorts(rotatedTarget);

assert(isFinite(portsSource.top.x) && isFinite(portsSource.top.y), `Rotated source top port finite for ${rotDeg}°`);
assert(isFinite(portsSource.right.nx) && isFinite(portsSource.right.ny), `Rotated source normals finite for ${rotDeg}°`);

for (const mode of ["curved", "orthogonal", "straight"] as const) {
  const path = getConnectionPath(anchors.source, anchors.target, mode);
  assert(!path.includes("NaN") && !path.includes("undefined") && path.length > 5, `Rotated ${rotDeg}° ${mode} path valid`);
}
}
console.log(" ✓ Intersecting, overlapping, concentric, and rotated nodes routing verified");

// 1.5 Arrowheads Stress
const arrowheadTypes = ["arrow", "triangle", "stealth", "diamond", "circle", "none"] as const;
for (const type of arrowheadTypes) {
  // Normal delta
  const normalHead = getArrowheadPath(100, 100, 80, 100, type, 12);
  if (type === "none") {
    assert(normalHead === null, "Arrowhead type 'none' returns null");
  } else {
    assert(normalHead !== null && normalHead.path.startsWith("M"), `Arrowhead ${type} produces valid SVG path`);
    assert(!normalHead!.path.includes("NaN"), `Arrowhead ${type} contains no NaN`);
  }
}

// Zero delta (fromX === tipX, fromY === tipY)
const zeroHead = getArrowheadPath(100, 100, 100, 100, type, 12);
assert(zeroHead === null, `Arrowhead with zero delta safely returns null (no division by zero)`);

// Sub-millimeter delta (len < 0.001)
const subMilliHead = getArrowheadPath(100, 100, 100.0001, 100.0001, type, 12);
assert(subMilliHead === null, `Arrowhead with sub-millimeter delta safely returns null`);
}
console.log(" ✓ Arrowhead generator stress and zero-division defenses verified\n");

// =====
// SECTION 2: BOARD DUPLICATION ID REMAPPING INTEGRITY ACROSS 100 BOARDS
// =====
console.log("[Section 2] Board Duplication ID Remapping Integrity (100 Duplicated Boards)...");

// Build an initial complex board with 8 nodes and 8 connections
const store = useWhiteboard.getState();
const initialBoardId = store.createBoard("Master Template Board", null, "#6366f1");

const sampleNodes: WhiteboardElement[] = [
  { id: "n1", type: "rectangle", x: 0, y: 0, width: 120, height: 80, rotation: 0, opacity: 1, strokeColor: "#000", fillColor: "#fff", fillStyle: "solid", dashStyle: "solid", strokeWidth: 2, createdAt: Date.now(), updatedAt: Date.now() },
  { id: "n2", type: "ellipse", x: 200, y: 0, width: 120, height: 80, rotation: 0, opacity: 1, strokeColor: "#000", fillColor: "#fff", fillStyle: "solid", dashStyle: "solid", strokeWidth: 2, createdAt: Date.now(), updatedAt: Date.now() },
  { id: "n3", type: "diamond", x: 400, y: 0, width: 100, height: 100, rotation: 0, opacity: 1, strokeColor: "#000", fillColor: "#fff", fillStyle: "solid", dashStyle: "solid", strokeWidth: 2, createdAt: Date.now(), updatedAt: Date.now() },
  { id: "n4", type: "sticky", x: 0, y: 200, width: 150, height: 120, rotation: 0.05, opacity: 1, text: "Idea 1", color: "yellow", fontSize: 16, align: "left", createdAt: Date.now(), updatedAt: Date.now() },
  { id: "n5", type: "sticky", x: 200, y: 200, width: 150, height: 120, rotation: -0.02, opacity: 1, text: "Idea 2", color: "blue", fontSize: 16, align: "left", createdAt: Date.now(), updatedAt: Date.now() },
  { id: "n6", type: "text", x: 400, y: 200, width: 180, height: 60, rotation: 0, opacity: 1, text: "Conclusion"

```

```

    text", color: "#000", fontSize: 18, fontFamily: "var(--font-inter)", fontWeight: 400, align: "left", createdAt: Date.now(), updatedAt: Date.now() },
    { id: "n7", type: "star", x: 600, y: 100, width: 100, height: 100, rotation: 0, opacity: 1, strokeColor: "#000", fillColor: "#fff", fillStyle: "solid", dashStyle: "solid", strokeWidth: 2, createdAt: Date.now(), updatedAt: Date.now() },
    { id: "n8", type: "cloud", x: 600, y: 300, width: 140, height: 90, rotation: 0, opacity: 1, strokeColor: "#000", fillColor: "#fff", fillStyle: "solid", dashStyle: "solid", strokeWidth: 2, createdAt: Date.now(), updatedAt: Date.now() },
  ];

const sampleConnections: ConnectionElement[] = [
  { id: "c1", type: "connection", x: 0, y: 0, rotation: 0, opacity: 1, source: { boardId: initialBoardId, elementId: "n1", anchor: "right" }, target: { boardId: initialBoardId, elementId: "n2", anchor: "left" }, linkType: "relates", routing: "curved", dashStyle: "solid", startArrowhead: "none", endArrowhead: "arrow", strokeWidth: 2, createdAt: Date.now(), updatedAt: Date.now() },
  { id: "c2", type: "connection", x: 0, y: 0, rotation: 0, opacity: 1, source: { boardId: initialBoardId, elementId: "n2", anchor: "right" }, target: { boardId: initialBoardId, elementId: "n3", anchor: "left" }, linkType: "leads_to", routing: "orthogonal", dashStyle: "solid", startArrowhead: "none", endArrowhead: "arrow", strokeWidth: 2, createdAt: Date.now(), updatedAt: Date.now() },
  { id: "c3", type: "connection", x: 0, y: 0, rotation: 0, opacity: 1, source: { boardId: initialBoardId, elementId: "n1", anchor: "bottom" }, target: { boardId: initialBoardId, elementId: "n4", anchor: "top" }, linkType: "contains", routing: "straight", dashStyle: "solid", startArrowhead: "none", endArrowhead: "arrow", strokeWidth: 2, createdAt: Date.now(), updatedAt: Date.now() },
  { id: "c4", type: "connection", x: 0, y: 0, rotation: 0, opacity: 1, source: { boardId: initialBoardId, elementId: "n4", anchor: "right" }, target: { boardId: initialBoardId, elementId: "n5", anchor: "left" }, linkType: "causes", routing: "curved", dashStyle: "solid", startArrowhead: "none", endArrowhead: "arrow", strokeWidth: 2, createdAt: Date.now(), updatedAt: Date.now() },
  { id: "c5", type: "connection", x: 0, y: 0, rotation: 0, opacity: 1, source: { boardId: initialBoardId, elementId: "n5", anchor: "right" }, target: { boardId: initialBoardId, elementId: "n6", anchor: "left" }, linkType: "relates", routing: "orthogonal", dashStyle: "solid", startArrowhead: "none", endArrowhead: "arrow", strokeWidth: 2, createdAt: Date.now(), updatedAt: Date.now() },
  { id: "c6", type: "connection", x: 0, y: 0, rotation: 0, opacity: 1, source: { boardId: initialBoardId, elementId: "n3", anchor: "right" }, target: { boardId: initialBoardId, elementId: "n7", anchor: "left" }, linkType: "depends_on", routing: "curved", dashStyle: "solid", startArrowhead: "none", endArrowhead: "arrow", strokeWidth: 2, createdAt: Date.now(), updatedAt: Date.now() },
  { id: "c7", type: "connection", x: 0, y: 0, rotation: 0, opacity: 1, source: { boardId: initialBoardId, elementId: "n6", anchor: "right" }, target: { boardId: initialBoardId, elementId: "n8", anchor: "left" }, linkType: "supports", routing: "straight", dashStyle: "solid", startArrowhead: "none", endArrowhead: "arrow", strokeWidth: 2, createdAt: Date.now(), updatedAt: Date.now() },
  // Cyclic connection n8 -> n1
  { id: "c8", type: "connection", x: 0, y: 0, rotation: 0, opacity: 1, source: { boardId: initialBoardId, elementId: "n8", anchor: "bottom" }, target: { boardId: initialBoardId, elementId: "n1", anchor: "left" }, linkType: "relates", routing: "curved", dashStyle: "solid", startArrowhead: "none", endArrowhead: "arrow", strokeWidth: 2, createdAt: Date.now(), updatedAt: Date.now() },
];

// Add elements to initial board
store.addElements([...sampleNodes, ...sampleConnections], false);

const duplicatedBoardIds: string[] = [];
let currentSourceBoardId = initialBoardId;

const allSeenElementIds = new Set<string>();
for (const el of sampleNodes) allSeenElementIds.add(el.id);
for (const el of sampleConnections) allSeenElementIds.add(el.id);

// Duplicate 50 in sequential chain (A -> B -> C -> ...)
for (let i = 1; i <= 50; i++) {
  const dupId = store.duplicateBoard(currentSourceBoardId);
  assert(dupId !== null, `Duplicate sequential board ${i} returned valid ID`);
  duplicatedBoardIds.push(dupId!);
  currentSourceBoardId = dupId!;
}

// Duplicate 50 from master root (A -> B1, A -> B2, ...)
for (let i = 51; i <= 100; i++) {
  const dupId = store.duplicateBoard(initialBoardId);
  assert(dupId !== null, `Duplicate radial board ${i} returned valid ID`);
  duplicatedBoardIds.push(dupId!);
}

assert(duplicatedBoardIds.length === 100, "Created 100 duplicated boards");

// Verify integrity of all 100 duplicated boards
const allBoards = useWhiteboard.getState().boards;
for (let idx = 0; idx < duplicatedBoardIds.length; idx++) {
  const boardId = duplicatedBoardIds[idx];
  const board = allBoards.find((b) => b.id === boardId);

```

```

assert(board !== undefined, `Duplicated board ${idx + 1} exists in store`);
assert(board!.elements.length === sampleNodes.length + sampleConnections.length, `Board ${idx + 1} has exact
  element count (16)`);

const boardElementIds = new Set<string>();
const nonConnElements = board!.elements.filter((e) => e.type !== "connection");
const connElements = board!.elements.filter((e) => e.type === "connection") as ConnectionElement[];

// 1. Verify every element ID is unique and not previously seen in other boards
for (const el of board!.elements) {
  assert(!allSeenElementIds.has(el.id), `Element ID ${el.id} in board ${idx + 1} is globally unique and not
    colliding`);
  allSeenElementIds.add(el.id);
  assert(!boardElementIds.has(el.id), `Element ID ${el.id} is unique within board ${idx + 1}`);
  boardElementIds.add(el.id);
}

// 2. Verify connection source and target element IDs point to elements inside THIS board
for (const conn of connElements) {
  assert(conn.source.boardId === boardId, `Connection ${conn.id} source.boardId is remapped to new board ${b
    oardId}`);
  assert(conn.target.boardId === boardId, `Connection ${conn.id} target.boardId is remapped to new board ${b
    oardId}`);
  assert(boardElementIds.has(conn.source.elementId), `Connection ${conn.id} source.elementId ${conn.source.e
    lementId} exists in board`);
  assert(boardElementIds.has(conn.target.elementId), `Connection ${conn.id} target.elementId ${conn.target.e
    lementId} exists in board`);

  // Verify source and target do not point to old IDs ("n1", "n2", etc.)
  assert(!["n1", "n2", "n3", "n4", "n5", "n6", "n7", "n8"].includes(conn.source.elementId), `Source ID was remapped
    away from old prototype ID`);
  assert(!["n1", "n2", "n3", "n4", "n5", "n6", "n7", "n8"].includes(conn.target.elementId), `Target ID was remapped
    away from old prototype ID`);
}
}
console.log(" ✓ 100 duplicated boards verified: 0 dangling IDs, 0 stale IDs, 100% topological remapping integ
  rity\n");

// =====
// SECTION 3: 50-STEP CIRCULAR UNDO/REDO HISTORY STACK STRESS TEST
// =====
console.log("[Section 3] 50-Step Circular Undo/Redo History Stack Stress Test...");

const historyBoardId = store.createBoard("History Test Board", null, "#10b981");
store.setActiveBoard(historyBoardId);

// 3.1 Verify initial history state
assert(!store.canUndo(), "Initial board canUndo is false");
assert(!store.canRedo(), "Initial board canRedo is false");

// 3.2 Add 100 sequential elements with commitHistory to test 100-step circular bound
for (let i = 1; i <= 100; i++) {
  const el: StickyElement = {
    id: `history-sticky-${i}`,
    type: "sticky",
    x: i * 10,
    y: i * 10,
    width: 100,
    height: 100,
    rotation: 0,
    opacity: 1,
    text: `Note ${i}`,
    color: "yellow",
    fontSize: 16,
    align: "left",
    createdAt: Date.now(),
    updatedAt: Date.now(),
  };
  store.addElement(el, true);

  const historyPastLen = useWhiteboard.getState().history.past.length;
  // History past is capped at 99 (which with current active board makes 100 total discrete steps)
  assert(historyPastLen <= 100, `History past length (${historyPastLen}) is strictly bounded <= 100 on step ${
    i}`);
}

const finalElementsCount = useWhiteboard.getState().boards.find((b) => b.id === historyBoardId)!.elements.leng

```

```
    th;
assert(finalElementsCount === 100, "100 elements added successfully");
assert(useWhiteboard.getState().canUndo(), "canUndo is true after 100 additions");

// 3.3 Perform 100 consecutive Undos
let undoCount = 0;
while (useWhiteboard.getState().canUndo() && undoCount < 110) {
  store.undo();
  undoCount++;
}

assert(undoCount === 99 || undoCount === 100, `Exactly ${undoCount} undos performed (bounded by 100-step capacity)`);
assert(!useWhiteboard.getState().canUndo(), "canUndo is false after exhausting past stack");
assert(useWhiteboard.getState().canRedo(), "canRedo is true after undos");

const elementsAfterUndo = useWhiteboard.getState().boards.find((b) => b.id === historyBoardId)!.elements.length;
assert(elementsAfterUndo === 100 - undoCount, `Elements count restored to ${100 - undoCount}`);

// 3.4 Boundary: Extra Undos on empty past stack (must be safe no-op)
for (let i = 0; i < 10; i++) {
  store.undo();
  assert(!useWhiteboard.getState().canUndo(), "canUndo remains false on over-undo");
}
assert(useWhiteboard.getState().boards.find((b) => b.id === historyBoardId)!.elements.length === elementsAfterUndo, "Elements unchanged on over-undo");

// 3.5 Perform Redos back to latest state
let redoCount = 0;
while (useWhiteboard.getState().canRedo() && redoCount < 110) {
  store.redo();
  redoCount++;
}

assert(redoCount === undoCount, `Redid exact number of steps (${redoCount})`);
assert(!useWhiteboard.getState().canRedo(), "canRedo is false after exhausting future stack");
assert(useWhiteboard.getState().boards.find((b) => b.id === historyBoardId)!.elements.length === 100, "Restore d all 100 elements on full redo");

// 3.6 Boundary: Extra Redos on empty future stack (must be safe no-op)
for (let i = 0; i < 10; i++) {
  store.redo();
  assert(!useWhiteboard.getState().canRedo(), "canRedo remains false on over-redo");
}

// 3.7 Branching History Invariant
// Undo 20 steps
for (let i = 0; i < 20; i++) {
  store.undo();
}
assert(useWhiteboard.getState().history.future.length === 20, "Future stack holds 20 undone actions");

// Commit a new divergent action (add a Text element)
const branchingText: TextElement = {
  id: "branch-text-1",
  type: "text",
  x: 999,
  y: 999,
  width: 200,
  height: 50,
  rotation: 0,
  opacity: 1,
  text: "Branching Action",
  color: "#000",
  fontSize: 20,
  fontFamily: "var(--font-inter)",
  fontWeight: 400,
  align: "left",
  createdAt: Date.now(),
  updatedAt: Date.now(),
};
store.addElement(branchingText, true);

// Future history must be cleared on divergent action
assert(useWhiteboard.getState().history.future.length === 0, "Branching action completely clears future redo history");
assert(!useWhiteboard.getState().canRedo(), "canRedo is false after new branch committed");
```

```
// Verify undoing the branch restores the pre-branch state
store.undo();
const currentElements = useWhiteboard.getState().boards.find((b) => b.id === historyBoardId)!.elements;
assert(!currentElements.some((e) => e.id === "branch-text-1"), "Undoing branch removes the divergent element")
;
assert(useWhiteboard.getState().canRedo(), "canRedo is true after undoing branch");

store.redo();
const redoneElements = useWhiteboard.getState().boards.find((b) => b.id === historyBoardId)!.elements;
assert(redoneElements.some((e) => e.id === "branch-text-1"), "Redoing branch restores the divergent element");

console.log("  ✓ 50-step circular history stack, boundary overflows, and branching invariants verified\n");

// =====
// FINAL SUMMARY
// =====
console.log("=====");
console.log(`=== CHALLENGER 2 SUITE COMPLETE: ${passedAssertions}/${totalAssertions} ASSERTIONS PASSED (100%)
===`);
console.log("=====");

// === END FILE: tests/challenger_2_empirical.test.ts ===
=====

// === START FILE: tests/challenger_geometry_stress.test.ts ===
// File 139 of 194 | Lines: 450 | Size: 18190 bytes
=====

import assert from "node:assert/strict";
import {
  hitTestElement,
  getElementBounds,
  getElementCenter,
  getUnionBounds,
  getNodeAnchorPorts,
  getResolvedAnchorPoint,
  getOptimalConnectionAnchors,
  getCurvedBezierPath,
  getOrthogonalPath,
  getStraightPath,
  getConnectionPath,
  getArrowheadPath,
  worldToScreen,
  screenToWorld,
  worldToScreenDelta,
  screenToWorldDelta,
  rotatePoint,
  clamp,
  lerp,
  alignElements,
  distributeElements,
  isVisibleInViewport,
  getResizeHandles,
} from "../src/lib/whiteboard/geometry";
import type {
  StrokeElement,
  RectangleElement,
  EllipseElement,
  DiamondElement,
  TriangleElement,
  StarElement,
  CloudElement,
  LineElement,
  ArrowElement,
  ConnectionElement,
  TextElement,
  StickyElement,
} from "../src/lib/whiteboard/types";

console.log("=====");
console.log("=== RUNNING CHALLENGER STRESS SUITE: GEOMETRY & HIT-TESTING ===");
console.log("=====");

let totalChecks = 0;
```

```

function pass(msg: string) {
  totalChecks++;
  // console.log(` ✓ ${msg}`);
}

// -----
// Section 1: Single-Tap Dot & Stroke Hit-Testing Stress
// -----
console.log("\n[Section 1] Single-Tap Dot & Stroke Hit-Testing Adversarial Tests...");

// 1.1 Single Dot at world (0, 0), strokeWidth = 2 -> radius = max(2/2, 2) = 2. Tolerance = 8 -> hit radius = 10
const dotBasic: StrokeElement = {
  id: "dot-basic",
  type: "stroke",
  x: 0,
  y: 0,
  rotation: 0,
  opacity: 1,
  points: [[0, 0, 0.5]],
  color: "#000",
  strokeWidth: 2,
  strokeStyle: "pen",
  createdAt: Date.now(),
  updatedAt: Date.now(),
};

assert.equal(hitTestElement(dotBasic, 0, 0), true, "Center hit must be true");
assert.equal(hitTestElement(dotBasic, 10, 0), true, "Hit at exactly boundary distance (10px) must be true");
assert.equal(hitTestElement(dotBasic, 0, 10), true, "Hit at y-boundary (10px) must be true");
assert.equal(hitTestElement(dotBasic, 7.07, 7.07), true, "Hit at diagonal ~10px must be true");
assert.equal(hitTestElement(dotBasic, 10.01, 0), false, "Hit just outside boundary (10.01px) must be false");
assert.equal(hitTestElement(dotBasic, 0, -10.01), false, "Hit just outside negative boundary must be false");
pass("Basic single dot radial boundary hit testing verified");

// 1.2 Single Dot with custom tolerance (0px, 20px)
assert.equal(hitTestElement(dotBasic, 2.0, 0, 0), true, "Hit with tol=0 at r=2 must be true");
assert.equal(hitTestElement(dotBasic, 2.01, 0, 0), false, "Hit with tol=0 at r=2.01 must be false");
assert.equal(hitTestElement(dotBasic, 22.0, 0, 20), true, "Hit with tol=20 at r=22 must be true");
assert.equal(hitTestElement(dotBasic, 22.01, 0, 20), false, "Hit with tol=20 at r=22.01 must be false");
pass("Single dot hit testing with custom tolerance parameters (0px and 20px) verified");

// 1.3 Dot with large strokeWidth = 50 -> radius = 25. Tolerance = 8 -> hit radius = 33
const dotLarge: StrokeElement = {
  ...dotBasic,
  strokeWidth: 50,
};
assert.equal(hitTestElement(dotLarge, 33, 0), true, "Hit large dot at boundary (33px) must be true");
assert.equal(hitTestElement(dotLarge, 33.1, 0), false, "Hit large dot outside boundary must be false");
pass("Large single dot (strokeWidth=50) hit testing verified");

// 1.4 Single Dot at negative coordinates with element offset
const dotNeg: StrokeElement = {
  ...dotBasic,
  x: -500,
  y: -300,
  points: [[10, 20, 0.5]], // world pos = (-490, -280)
};
assert.equal(hitTestElement(dotNeg, -490, -280), true, "Hit at negative world pos must be true");
assert.equal(hitTestElement(dotNeg, -490 + 9, -280), true, "Hit near negative world pos must be true");
assert.equal(hitTestElement(dotNeg, -490 + 11, -280), false, "Hit outside negative world pos must be false");
pass("Single dot at negative world coordinates verified");

// 1.5 Locked stroke should NEVER hit
const dotLocked: StrokeElement = {
  ...dotBasic,
  locked: true,
};
assert.equal(hitTestElement(dotLocked, 0, 0), false, "Locked stroke must never hit");
pass("Locked stroke element returns false");

// 1.6 Multi-point stroke hit testing
const strokeMulti: StrokeElement = {
  id: "stroke-multi",
  type: "stroke",
  x: 100,
  y: 100,
  rotation: 0,

```

```
opacity: 1,
points: [
  [0, 0, 0.5],
  [50, 0, 0.5],
  [50, 50, 0.5],
],
color: "#000",
strokeWidth: 4, // radius = 2, hit radius with tol(8) = 10
strokeStyle: "pen",
createdAt: Date.now(),
updatedAt: Date.now(),
};
// World coords: (100,100) -> (150,100) -> (150,150)
assert.equal(hitTestElement(strokeMulti, 125, 100), true, "Segment 1 midpoint hits");
assert.equal(hitTestElement(strokeMulti, 125, 109), true, "Near segment 1 within tolerance hits");
assert.equal(hitTestElement(strokeMulti, 125, 111), false, "Outside segment 1 tolerance misses");
assert.equal(hitTestElement(strokeMulti, 150, 125), true, "Segment 2 midpoint hits");
assert.equal(hitTestElement(strokeMulti, 141, 125), true, "Near segment 2 hits");
assert.equal(hitTestElement(strokeMulti, 139, 125), false, "Far from segment 2 misses");
pass("Multi-segment stroke hit-testing along segments verified");

// -----
// Section 2: All 8 Vector Shape Tools Hit-Testing Stress
// -----
console.log("\n[Section 2] All 8 Vector Shapes (Rect, Ellipse, Diamond, Triangle, Star, Cloud, Line, Arrow)...
");

// 2.1 Rectangle (x=100, y=100, w=200, h=100, tol=8)
const rect: RectangleElement = {
  id: "rect-test",
  type: "rectangle",
  x: 100,
  y: 100,
  width: 200,
  height: 100,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  fillColor: "transparent",
  fillStyle: "transparent",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: Date.now(),
  updatedAt: Date.now(),
};
assert.equal(hitTestElement(rect, 200, 150), true, "Center of rectangle hits");
assert.equal(hitTestElement(rect, 100 - 8, 100 - 8), true, "Top-left corner within tolerance hits");
assert.equal(hitTestElement(rect, 300 + 8, 200 + 8), true, "Bottom-right corner within tolerance hits");
assert.equal(hitTestElement(rect, 100 - 8.1, 150), false, "Outside left boundary misses");
assert.equal(hitTestElement(rect, 300 + 8.1, 150), false, "Outside right boundary misses");

// Rotated Rectangle (45 deg)
const rectRot: RectangleElement = {
  ...rect,
  rotation: Math.PI / 4,
};
// Center remains (200, 150)
assert.equal(hitTestElement(rectRot, 200, 150), true, "Center of rotated rectangle hits");
// Original top-left unrotated (100, 100) rotated around (200, 150) by 45 deg
const origTLRot = rotatePoint(100, 100, 200, 150, Math.PI / 4);
assert.equal(hitTestElement(rectRot, origTLRot.x, origTLRot.y), true, "Rotated corner hits");
// Point far from rotated rectangle
assert.equal(hitTestElement(rectRot, 0, 0), false, "Point outside rotated rectangle misses");
pass("Rectangle unrotated and rotated 45° hit testing verified");

// 2.2 Ellipse (x=100, y=100, w=200, h=100) -> center (200, 150), rx=100, ry=50
const ellipse: EllipseElement = {
  id: "ellipse-test",
  type: "ellipse",
  x: 100,
  y: 100,
  width: 200,
  height: 100,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  fillColor: "transparent",
  fillStyle: "transparent",
```

```

    dashStyle: "solid",
    strokeWidth: 2,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  });
assert.equal(hitTestElement(ellipse, 200, 150), true, "Ellipse center hits");
assert.equal(hitTestElement(ellipse, 300, 150), true, "Ellipse right vertex (200+100, 150) hits");
assert.equal(hitTestElement(ellipse, 200, 200), true, "Ellipse bottom vertex (200, 150+50) hits");
// Bounding box corner (100, 100): dx=100, dy=50 -> (100/(100+8))^2 + (50/(50+8))^2 = (100/108)^2 + (50/58)^2
= 0.857 + 0.743 = 1.6 > 1 -> MISS!
assert.equal(hitTestElement(ellipse, 100, 100), false, "Bounding box corner outside ellipse misses");
pass("Ellipse curvature boundary testing verified (correctly excludes bounding box corners)");

// 2.3 Diamond (x=100, y=100, w=200, h=100) -> center (200, 150)
const diamond: DiamondElement = {
  id: "dia-test",
  type: "diamond",
  x: 100,
  y: 100,
  width: 200,
  height: 100,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  fillColor: "transparent",
  fillStyle: "transparent",
  dashStyle: "solid",
  strokeWidth: 2,
  createdAt: Date.now(),
  updatedAt: Date.now(),
};
assert.equal(hitTestElement(diamond, 200, 150), true, "Diamond center hits");
assert.equal(hitTestElement(diamond, 300, 150), true, "Diamond right point hits");
assert.equal(hitTestElement(diamond, 200, 100), true, "Diamond top point hits");
// Bounding box top-left corner (100, 100): |dx| + |dy| = 100/(100+8) + 50/(50+8) = 1.78 > 1 -> MISS!
assert.equal(hitTestElement(diamond, 100, 100), false, "Bounding box corner outside diamond misses");
pass("Diamond equation |x/w| + |y/h| <= 1 hit testing verified");

// 2.4 Triangle, Star, Cloud (Conservative Bounds)
const tri: TriangleElement = { ...rect, type: "triangle", id: "tri-test" };
const star: StarElement = { ...rect, type: "star", id: "star-test" };
const cloud: CloudElement = { ...rect, type: "cloud", id: "cloud-test" };

for (const el of [tri, star, cloud]) {
  assert.equal(hitTestElement(el, 200, 150), true, `${el.type} center hits`);
  assert.equal(hitTestElement(el, 50, 50), false, `${el.type} far outside misses`);
}
pass("Triangle, Star, Cloud shape hit testing verified");

// 2.5 Line & Arrow (Segment Distance with strokeWidth & tolerance)
// Line from (100, 100) to (300, 100) (horizontal dx=200, dy=0), strokeWidth=4, tol=8 -> hit dist <= 10
const line: LineElement = {
  id: "line-test",
  type: "line",
  x: 100,
  y: 100,
  x2: 200,
  y2: 0,
  rotation: 0,
  opacity: 1,
  strokeColor: "#000",
  strokeWidth: 4,
  dashStyle: "solid",
  createdAt: Date.now(),
  updatedAt: Date.now(),
};
assert.equal(hitTestElement(line, 200, 100), true, "Line midpoint hits");
assert.equal(hitTestElement(line, 200, 110), true, "Line perpendicular distance 10px hits");
assert.equal(hitTestElement(line, 200, 111), false, "Line perpendicular distance 11px misses");
assert.equal(hitTestElement(line, 90, 100), true, "Line start endpoint with 10px tolerance hits");
assert.equal(hitTestElement(line, 89, 100), false, "Line before start beyond tolerance misses");

// Arrow
const arrow: ArrowElement = {
  ...line,
  id: "arrow-test",
  type: "arrow",
  startArrowhead: "none",

```

```

    endArrowhead: "arrow",
    arrowheadSize: 10,
  };
  assert.equal(hitTestElement(arrow, 200, 100), true, "Arrow shaft hits");
  assert.equal(hitTestElement(arrow, 300, 100), true, "Arrow tip hits");
  assert.equal(hitTestElement(arrow, 300, 110), true, "Arrow tip with 10px tolerance hits");
  assert.equal(hitTestElement(arrow, 300, 112), false, "Arrow tip beyond tolerance misses");

  // Degenerate zero-length line (x2=0, y2=0)
  const zeroLine: LineElement = {
    ...line,
    x2: 0,
    y2: 0,
  };
  assert.equal(hitTestElement(zeroLine, 100, 100), true, "Zero length line at start point hits");
  assert.equal(hitTestElement(zeroLine, 110, 100), true, "Zero length line within tolerance hits");
  assert.equal(hitTestElement(zeroLine, 111, 100), false, "Zero length line beyond tolerance misses");
  pass("Line and Arrow hit testing along segments and zero-length degenerate cases verified");

  // -----
  // Section 3: Smart Anchor Ports & 3 Routing Engines Stress
  // -----
  console.log("\n[Section 3] Smart Anchor Ports & 3 Routing Engines...");

  // 3.1 Anchor Ports with Rotation
  const nodeRect: RectangleElement = {
    ...rect,
    x: 100,
    y: 100,
    width: 100,
    height: 50,
    rotation: Math.PI / 2, // 90 deg rotation around center (150, 125)
  };
  const portsRot = getNodeAnchorPorts(nodeRect);
  // With 90 deg rotation, unrotated Top (150, 100) rotated around (150, 125) by 90 deg -> (150 - 0, 125 - 25) -
  // > becomes (175, 125)
  assert.equal(Math.round(portsRot.top.x), 175, "Rotated top port X correct");
  assert.equal(Math.round(portsRot.top.y), 125, "Rotated top port Y correct");
  pass("Anchor ports rotation affine transformation verified");

  // 3.2 3 Routing Engines Stress
  const p1 = { x: 100, y: 100, nx: 1, ny: 0, anchor: "right" as const };
  const p2 = { x: 300, y: 200, nx: -1, ny: 0, anchor: "left" as const };

  // Curved
  const curvedPath = getCurvedBezierPath(p1, p2);
  assert.ok(curvedPath.startsWith("M 100,100 C"), "Curved path starts with M...C");
  assert.ok(curvedPath.endsWith("300,200"), "Curved path ends at p2");

  // Curved with degenerate distance (< 2px)
  const pClose = { x: 100.5, y: 100.5, nx: 1, ny: 0, anchor: "right" as const };
  const curvedClose = getCurvedBezierPath(p1, pClose);
  assert.equal(curvedClose, "M 100,100 L 100.5,100.5", "Close curved points (<2px) fallback to linear segment");

  // Orthogonal
  const orthoPath = getOrthogonalPath(p1, p2);
  assert.ok(orthoPath.startsWith("M 100,100 L"), "Orthogonal path generated");

  // Orthogonal with degenerate distance (< 4px)
  const orthoClose = getOrthogonalPath(p1, pClose);
  assert.equal(orthoClose, "M 100,100 L 100.5,100.5", "Close orthogonal points (<4px) fallback to linear segment");

  // Straight
  const straightPath = getStraightPath(p1, p2);
  assert.equal(straightPath, "M 100,100 L 300,200", "Straight path is exact line");

  // getConnectionPath dispatch
  assert.equal(getConnectionPath(p1, p2, "straight"), straightPath);
  assert.equal(getConnectionPath(p1, p2, "orthogonal"), orthoPath);
  assert.equal(getConnectionPath(p1, p2, "curved"), curvedPath);
  pass("All 3 routing engines (Curved, Orthogonal, Straight) and degenerate distances verified");

  // 3.3 All Arrowhead Types
  const arrowTypes = ["none", "triangle", "arrow", "stealth", "diamond", "circle"] as const;
  for (const at of arrowTypes) {
    const head = getArrowheadPath(100, 100, 50, 100, at, 12);
    if (at === "none") {

```

```
    assert.equal(head, null, "None arrowhead is null");
  } else {
    assert.ok(head !== null, `${at} arrowhead produced`);
    assert.ok(!head.path.includes("NaN"), `${at} arrowhead has no NaN`);
  }
}

// Arrowhead with zero distance (tip == from) -> returns null
const nullHead = getArrowheadPath(100, 100, 100, 100, "arrow", 12);
assert.equal(nullHead, null, "Zero distance arrowhead safely returns null");
pass("All arrowhead polygon generators and zero-distance guard verified");

// -----
// Section 4: Screen <-> World Transforms & Sidebar Offset Invariance
// -----
console.log("\n[Section 4] Screen <-> World Transformations Invertibility & Viewport Math...");

// 4.1 Invertibility Stress: 1,000 points across zoom ranges [0.01 to 100.0]
const testZooms = [0.01, 0.1, 0.5, 1.0, 2.5, 10.0, 50.0, 100.0];
const testVps = [
  { width: 1280, height: 800 },
  { width: 375, height: 667 }, // mobile
  { width: 3840, height: 2160 }, // 4K
];

for (const zoom of testZooms) {
  for (const vp of testVps) {
    const cam = { x: 1234.56, y: -789.12, zoom };
    for (let i = 0; i < 20; i++) {
      const worldX = (Math.random() - 0.5) * 20000;
      const worldY = (Math.random() - 0.5) * 20000;
      const screen = worldToScreen(worldX, worldY, cam, vp);
      const recovered = screenToWorld(screen.x, screen.y, cam, vp);

      assert.ok(Math.abs(recovered.x - worldX) < 1e-6, `World X invertibility at zoom ${zoom}`);
      assert.ok(Math.abs(recovered.y - worldY) < 1e-6, `World Y invertibility at zoom ${zoom}`);
    }
  }
}
pass("Screen <-> World transforms strictly invertible across all zoom scales and viewports");

// 4.2 Delta transforms
assert.equal(worldToScreenDelta(10, 2.5), 25);
assert.equal(screenToWorldDelta(25, 2.5), 10);
pass("Delta transformations verified");

// -----
// Section 5: Alignment & Distribution Stress
// -----
console.log("\n[Section 5] Alignment & Distribution Math Stress...");

const r1: RectangleElement = { ...rect, id: "r1", x: 10, y: 10, width: 20, height: 20 };
const r2: RectangleElement = { ...rect, id: "r2", x: 50, y: 30, width: 40, height: 30 };
const r3: RectangleElement = { ...rect, id: "r3", x: 120, y: 80, width: 30, height: 40 };

// Align Center
const alignedCenter = alignElements([r1, r2, r3], "center");
// Union x: 10 to 150 -> union center = 80
// r1 width 20 -> x = 80 - 10 = 70
// r2 width 40 -> x = 80 - 20 = 60
// r3 width 30 -> x = 80 - 15 = 65
assert.equal(alignedCenter[0].x, 70);
assert.equal(alignedCenter[1].x, 60);
assert.equal(alignedCenter[2].x, 65);

// Align Middle
const alignedMiddle = alignElements([r1, r2, r3], "middle");
// Union y: 10 to 120 -> union center = 65
// r1 height 20 -> y = 65 - 10 = 55
// r2 height 30 -> y = 65 - 15 = 50
// r3 height 40 -> y = 65 - 20 = 45
assert.equal(alignedMiddle[0].y, 55);
assert.equal(alignedMiddle[1].y, 50);
assert.equal(alignedMiddle[2].y, 45);

// Align with single element -> no-op
assert.deepEqual(alignElements([r1], "left"), [r1]);
```

```
// Distribute Vertical with 3 elements
const distributedV = distributeElements([r1, r2, r3], "vertical");
// Total height = 20 + 30 + 40 = 90
// Span = 120 - 10 = 110
// Gap = (110 - 90) / 2 = 10
// r1 y = 10
// r2 y = 10 + 20 + 10 = 40
// r3 y = 40 + 30 + 10 = 80
assert.equal(distributedV[0].y, 10);
assert.equal(distributedV[1].y, 40);
assert.equal(distributedV[2].y, 80);

// Distribute with < 3 elements -> returns original
assert.deepEqual(distributeElements([r1, r2], "horizontal"), [r1, r2]);
pass("Alignment (left, center, right, top, middle, bottom) and Distribution math verified");

console.log(`\n=====`);
console.log(`=== GEOMETRY STRESS SUITE: ALL ${totalChecks} ASSERTIONS PASSED (100%) ===`);
console.log(`=====\n`);

=====
// === END FILE: tests/challenger_geometry_stress.test.ts ===
=====

=====
// === START FILE: tests/challenger_stroke_stress.test.ts ===
// File 140 of 194 | Lines: 337 | Size: 16834 bytes
=====

import assert from "node:assert/strict";
import {
  getSvgPathFromStroke,
  buildStrokePath,
  pruneTerminalHooks,
  computePressureFromSpeed,
  getStrokeOptionsForStyle,
  getArrowheadPath,
} from "../src/lib/whiteboard/stroke";
import { PEN_TYPE_PRESETS, type StrokeElement } from "../src/lib/whiteboard/types";

console.log("=====");
console.log("=== RUNNING CHALLENGER STRESS SUITE: STROKE & B-SPLINE ENGINE ===");
console.log("=====");

let totalChecks = 0;
function pass(msg: string) {
  totalChecks++;
  // console.log(` ✓ ${msg}`);
}

// -----
// Section 1: getSvgPathFromStroke Adversarial Edge Cases
// -----
console.log(`\n[Section 1] getSvgPathFromStroke Boundary & Topology Stress Tests...`);

// 1.1 Empty array
assert.equal(getSvgPathFromStroke([]), "", "Empty point array must return empty string");
assert.equal(getSvgPathFromStroke([], false), "", "Empty point array (open) must return empty string");
pass("0 points returns empty string");

// 1.2 Single point
const p1_default = getSvgPathFromStroke([[50, 60]]);
assert.ok(p1_default.startsWith("M 49.00,60.00"), "Single point must return mini circle starting at (x-1, y)");
assert.ok(p1_default.endsWith("Z"), "Single point must end with Z");
assert.ok(!p1_default.includes("NaN"), "Single point path must not contain NaN");

const p1_neg = getSvgPathFromStroke([[-100.5, -200.75]]);
assert.ok(!p1_neg.includes("NaN"), "Negative single point must not contain NaN");
pass("1 point generates valid SVG circle path without NaN");

// 1.3 Two points
const p2_closed = getSvgPathFromStroke([[10, 20], [30, 40]], true);
assert.equal(p2_closed, "M 10.00,20.00 L 30.00,40.00 Z", "2 points closed must return M p0 L p1 Z");

const p2_open = getSvgPathFromStroke([[10, 20], [30, 40]], false);
assert.equal(p2_open, "M 10.00,20.00 L 30.00,40.00 Z", "2 points open must return M p0 L p1 Z");
```

```

pass("2 points returns valid closed line M...L...Z");

// 1.4 Three points (Triangle polygon)
const tri = [[0, 0], [100, 0], [50, 80]];
const triClosed = getSvgPathFromStroke(tri, true);
// Start midpoint: (tri[0] + tri[2])/2 = (25, 40)
assert.ok(triClosed.startsWith("M 25.00,40.00"), "Triangle closed must start at midpoint of p2 and p0");
// A triangle's corners are far sharper than the 130-degree threshold, so the
// path must be routed THROUGH each vertex. Rounding them (the old behaviour)
// cut the corners off, which is what blunted the end of every stroke.
assert.ok(triClosed.includes("L 0.00,0.00"), "sharp vertex p0 reached, not rounded away");
assert.ok(triClosed.includes("L 100.00,0.00"), "sharp vertex p1 reached, not rounded away");
assert.ok(triClosed.includes("L 50.00,80.00"), "sharp vertex p2 reached, not rounded away");
assert.ok(triClosed.endsWith("Z"), "Triangle closed must end with Z");

const triOpen = getSvgPathFromStroke(tri, false);
assert.ok(triOpen.startsWith("M 0.00,0.00"), "Open 3-pt path must start at p0");
assert.ok(triOpen.includes("L 50.00,80.00"), "Open path must end with L pLast");
pass("3 points closed and open: sharp vertices preserved, ends with Z");

// 1.5 Collinear Points
const horizPoints = Array.from({ length: 20 }, (_, i) => [i * 10, 50]);
const horizPath = getSvgPathFromStroke(horizPoints, true);
assert.ok(!horizPath.includes("NaN") && !horizPath.includes("Infinity"), "Horizontal collinear must have no NaN/Infinity");
assert.ok(horizPath.endsWith("Z"), "Horizontal collinear must end with Z");

const vertPoints = Array.from({ length: 20 }, (_, i) => [50, i * 10]);
const vertPath = getSvgPathFromStroke(vertPoints, true);
assert.ok(!vertPath.includes("NaN") && !vertPath.includes("Infinity"), "Vertical collinear must have no NaN/Infinity");

const diagPoints = Array.from({ length: 50 }, (_, i) => [i * 5.5, i * 5.5]);
const diagPath = getSvgPathFromStroke(diagPoints, true);
assert.ok(!diagPath.includes("NaN"), "Diagonal collinear must have no NaN");
pass("Collinear points (horizontal, vertical, diagonal) generate smooth valid paths");

// 1.6 Duplicate Coordinates & Zero Distance
const dupPoints = [[10, 10], [10, 10], [10, 10], [10, 10]];
const dupPath = getSvgPathFromStroke(dupPoints, true);
assert.ok(!dupPath.includes("NaN"), "Duplicate coordinates must not produce NaN");
assert.equal(dupPath.match(/Q 10\.\d\d,10\.\d\d 10\.\d\d,10\.\d\d/g)?.length, 4, "Duplicate coordinates produce 4 quad segments smoothly");

const mixedDup = [[0, 0], [0, 0], [100, 100], [100, 100], [200, 0]];
const mixedDupPath = getSvgPathFromStroke(mixedDup, true);
assert.ok(!mixedDupPath.includes("NaN"), "Mixed duplicate coordinates produce valid path");

const microPoints = [[0, 0], [1e-8, 1e-8], [2e-8, 2e-8], [3e-8, 3e-8]];
const microPath = getSvgPathFromStroke(microPoints, true);
assert.ok(!microPath.includes("NaN"), "Micro-distance coordinates produce valid path");
pass("Duplicate and micro-distance coordinates handled with zero division error");

// 1.7 Self-Intersecting Polygons (Figure-8, Star, Spirals, Zigzag)
const figure8 = [[0, 0], [100, 100], [100, 0], [0, 100]];
const fig8Path = getSvgPathFromStroke(figure8, true);
assert.ok(!fig8Path.includes("NaN") && fig8Path.endsWith("Z"), "Figure-8 path generated cleanly");

// 5-Point Star
const starPoly = [
  [50, 0], [20, 100], [95, 35], [5, 35], [80, 100]
];
const starPath = getSvgPathFromStroke(starPoly, true);
assert.ok(!starPath.includes("NaN") && starPath.endsWith("Z"), "Self-intersecting star polygon generated cleanly");

// Tight zigzag with 180-deg reversal
const zigzag: number[][] = [];
for (let i = 0; i < 50; i++) {
  zigzag.push([i * 4, (i % 2 === 0) ? 0 : 50]);
}
const zigzagPath = getSvgPathFromStroke(zigzag, true);
assert.ok(!zigzagPath.includes("NaN") && zigzagPath.endsWith("Z"), "Zigzag path generated cleanly");
pass("Self-intersecting loops and tight reversals generate valid SVG paths");

// 1.8 Massive Point Sets (1,000 to 10,000 points - Stress & Benchmark)
const largePts1k: number[][] = Array.from({ length: 1000 }, (_, i) => {
  const theta = (i / 1000) * Math.PI * 2;

```

```

    return [500 + Math.cos(theta) * 200, 500 + Math.sin(theta) * 200];
  });

const t0 = performance.now();
const path1k = getSvgPathFromStroke(largePts1k, true);
const t1 = performance.now();
assert.ok(!path1k.includes("NaN"), "1k points must have no NaN");
assert.ok(t1 - t0 < 50, `1k points B-spline generation took ${t1 - t0}.toFixed(2)}ms (expected < 50ms)`);

const largePts10k: number[][] = Array.from({ length: 10000 }, (_, i) => [i, Math.sin(i * 0.05) * 50]);
const t2 = performance.now();
const path10k = getSvgPathFromStroke(largePts10k, true);
const t3 = performance.now();
assert.ok(!path10k.includes("NaN"), "10k points must have no NaN");
assert.ok(t3 - t2 < 100, `10k points B-spline generation took ${t3 - t2}.toFixed(2)}ms (expected < 100ms)`);
pass(`Massive point sets (1k & 10k points) passed benchmark in ${t3 - t2}.toFixed(2)}ms`);

// -----
// Section 2: buildStrokePath - Single-Tap Dots & Micro-Jitter Stress
// -----
console.log(`\n[Section 2] buildStrokePath Single-Tap Dots, Jitter & Stroke Width Stress...`);

const makeStroke = (points: [number, number, number][], strokeWidth: number, strokeStyle = "pen"): StrokeElement => ({
  id: "test-stroke",
  type: "stroke",
  x: 100,
  y: 100,
  rotation: 0,
  opacity: 1,
  points,
  color: "#000000",
  strokeWidth,
  strokeStyle: strokeStyle as any,
  createdAt: Date.now(),
  updatedAt: Date.now(),
});

// 2.1 Empty stroke
assert.equal(buildStrokePath(makeStroke([], 2)), "", "Empty stroke must return empty string");
pass("Empty stroke element returns empty string");

// 2.2 Stroke Width Matrix for 1-point tap: [1, 2, 4, 10, 24, 50, 100]
const widths = [1, 2, 4, 10, 24, 50, 100];
for (const w of widths) {
  const el = makeStroke([[0, 0, 0.5]], w);
  const path = buildStrokePath(el);
  const expectedRadius = Math.max(w / 2, 2);
  const expectedRStr = expectedRadius.toFixed(2);
  const expectedDiamStr = (expectedRadius * 2).toFixed(2);

  assert.ok(path.startsWith(`M -${expectedRStr},0.00`), `Width ${w} should start at -${expectedRStr}`);
  assert.ok(path.includes(`a ${expectedRStr},${expectedRStr} 0 1,0 ${expectedDiamStr},0`), `Width ${w} should have arc with diameter ${expectedDiamStr}`);
  assert.ok(path.endsWith("Z"), `Width ${w} must end with Z`);
  assert.ok(!path.includes("NaN"), `Width ${w} must have no NaN`);
}
pass("Single-tap dots render exact circles for all stroke widths (1px to 100px)");

// 2.3 Degenerate stroke widths (0, negative)
const elZero = makeStroke([[0, 0, 0.5]], 0);
const pathZero = buildStrokePath(elZero);
assert.ok(pathZero.includes("a 2.00,2.00"), "0 strokeWidth clamps to radius 2");

const elNeg = makeStroke([[0, 0, 0.5]], -10);
const pathNeg = buildStrokePath(elNeg);
assert.ok(pathNeg.includes("a 2.00,2.00"), "Negative strokeWidth clamps to radius 2");
pass("Degenerate stroke widths (0, negative) safely clamped to minimum radius 2");

// 2.4 Micro-Jitter Tap Testing (0.5px, 1.5px, 3.0px, 10.0px)
// Jitter: 2 points with 0.5px distance
const elJitter05 = makeStroke([[0, 0, 0.5], [0.3, 0.4, 0.5]], 4); // dist = 0.5
const path05 = buildStrokePath(elJitter05);
assert.ok(path05.includes("a 2.00,2.00"), "0.5px jitter renders solid circle");

// Jitter: 2 points with 1.5px distance
const elJitter15 = makeStroke([[0, 0, 0.5], [0.9, 1.2, 0.5]], 4); // dist = 1.5
const path15 = buildStrokePath(elJitter15);

```

```

assert.ok(path15.includes("a 2.00,2.00"), "1.5px jitter renders solid circle");

// Jitter: 3 points within 3.0px bounding span
const elJitter30 = makeStroke([[0, 0, 0.5], [1.5, 1.0, 0.5], [2.0, 2.0, 0.5]], 4); // span ~2.83px
const path30 = buildStrokePath(elJitter30);
assert.ok(path30.includes("a 2.00,2.00"), "3.0px jitter renders solid circle");

// Jitter with large strokeWidth (e.g. strokeWidth = 50, dotThreshold = max(4, 50*0.6) = 30px)
const elJitterLargeW = makeStroke([[0, 0, 0.5], [5, 8, 0.5], [10, 10, 0.5]], 50); // span ~14.14px < 30px
const pathLargeW = buildStrokePath(elJitterLargeW);
assert.ok(pathLargeW.includes("a 25.00,25.00"), "14px jitter with 50px stroke width renders solid circle radius 25");

// High jitter: 2 points with 10.0px distance with 2px stroke (span 10 > 4, but length <= 2 and span < 6 -> false, getStroke called)
const elJitter10 = makeStroke([[0, 0, 0.5], [6, 8, 0.5]], 2);
const path10 = buildStrokePath(elJitter10);
assert.ok(path10.length > 0 && path10.startsWith("M ") && path10.endsWith("Z"), "10px movement generates full stroke path or fallback circle");
pass("Micro-jitter taps (0.5px, 1.5px, 3.0px, 10px) correctly resolved without collapsing to 0 area");

// -----
// Section 3: All 6 Pen Presets Stress Testing
// -----
console.log("\n[Section 3] All 6 Pen Presets (Pen, Highlighter, Marker, Brush, Calligraphy, Pencil)...");

const allPresets = ["pen", "highlighter", "marker", "brush", "calligraphy", "pencil"] as const;

for (const preset of allPresets) {
  // Check options
  const opt = getStrokeOptionsForStyle(preset, 4, true);
  assert.ok(opt.size !== undefined && opt.size > 0, `${preset} size must be > 0`);
  assert.ok(opt.start?.taper !== undefined, `${preset} start taper defined`);
  assert.ok(opt.end?.taper !== undefined, `${preset} end taper defined`);

  if (preset === "highlighter") {
    assert.equal(opt.size, 12, "Highlighter size multiplier 3x applied");
  }

  // 1. Single dot for preset
  const dot = makeStroke([[0, 0, 0.5]], 4, preset);
  const dotPath = buildStrokePath(dot);
  assert.ok(dotPath.includes("a 2.00,2.00") && dotPath.endsWith("Z"), `${preset} single dot must render circle`);

  // 2. Short 3-point stroke
  const shortStroke = makeStroke([[0, 0, 0.5], [10, 5, 0.6], [25, 12, 0.7]], 6, preset);
  const shortPath = buildStrokePath(shortStroke);
  assert.ok(shortPath.length > 0 && shortPath.startsWith("M ") && shortPath.endsWith("Z"), `${preset} short stroke valid`);

  // 3. Medium S-curve stroke
  const medStroke = makeStroke([
    [0, 0, 0.5],
    [30, 20, 0.6],
    [70, 80, 0.8],
    [120, 60, 0.5],
    [180, 100, 0.3],
    [240, 150, 0.2],
  ], 8, preset);
  const medPath = buildStrokePath(medStroke);
  assert.ok(medPath.length > 0 && medPath.startsWith("M ") && medPath.endsWith("Z"), `${preset} medium stroke valid`);
  assert.ok(!medPath.includes("NaN"), `${preset} medium stroke has no NaN`);

  // 4. Draft mode (isDraft = true)
  const draftPath = buildStrokePath(medStroke, true);
  assert.ok(draftPath.length > 0 && draftPath.startsWith("M ") && draftPath.endsWith("Z"), `${preset} draft mode valid`);
}

// Fallback for unknown preset
const unknownOpt = getStrokeOptionsForStyle("unknown_fancy_pen", 5);
assert.equal(unknownOpt.size, 5, "Unknown preset falls back to pen size");
const unknownStroke = makeStroke([[0, 0, 0.5], [20, 20, 0.5], [50, 50, 0.5]], 5, "unknown_fancy_pen");
const unknownPath = buildStrokePath(unknownStroke);
assert.ok(unknownPath.length > 0 && unknownPath.endsWith("Z"), "Unknown preset produces valid stroke");
pass("All 6 pen presets and unknown fallback tested under dots, short, medium, and draft modes");

```

```
// -----
// Section 4: Terminal Hook Pruning & Speed-Based Pressure
// -----
console.log("\n[Section 4] Terminal Hook Pruning & Speed-Based Pressure Math...");

// 4.1 pruneTerminalHooks
assert.deepEqual(pruneTerminalHooks([], [], "Empty array safe"));
assert.deepEqual(pruneTerminalHooks([[0, 0, 0.5]]), [[0, 0, 0.5]], "1 point safe");
assert.deepEqual(pruneTerminalHooks([[0, 0, 0.5], [10, 0, 0.5]]), [[0, 0, 0.5], [10, 0, 0.5]], "2 points safe");
assert.deepEqual(pruneTerminalHooks([[0, 0, 0.5], [10, 0, 0.5], [20, 0, 0.5]]), [[0, 0, 0.5], [10, 0, 0.5], [20, 0, 0.5]], "3 points safe");

// 4 points with sharp 180-deg reversal hook (< 45px tail)
const hookedPoints: [number, number, number][] = [
  [0, 0, 0.5],
  [50, 0, 0.5],
  [100, 0, 0.5],
  [85, 0, 0.5], // turns back by 15px
];
const unhooked = pruneTerminalHooks(hookedPoints);
assert.equal(unhooked.length, 3, "Sharp trailing tail hook pruned");
assert.deepEqual(unhooked[2], [100, 0, 0.5], "Last point is now the true peak");

// 4 points continuing forward (smooth line) -> NOT pruned
const straightPts: [number, number, number][] = [
  [0, 0, 0.5],
  [50, 0, 0.5],
  [100, 0, 0.5],
  [150, 0, 0.5],
];
const keptStraight = pruneTerminalHooks(straightPts);
assert.equal(keptStraight.length, 4, "Straight stroke tail is NOT pruned");

// Smooth 45-deg curve -> NOT pruned
const curvePts: [number, number, number][] = [
  [0, 0, 0.5],
  [50, 0, 0.5],
  [100, 0, 0.5],
  [120, 20, 0.5],
];
const keptCurve = pruneTerminalHooks(curvePts);
assert.equal(keptCurve.length, 4, "Smooth curve tail is NOT pruned");
pass("Terminal hook filter prunes sharp reversals while preserving smooth curves and forward lines");

// 4.2 computePressureFromSpeed
assert.equal(computePressureFromSpeed([], 0.5, "Empty points returns 0.5"));
assert.equal(computePressureFromSpeed([[0, 0, 0.5]], 0.5, "1 point returns 0.5"));

// Stationary points (dist = 0) -> pressure = 0.52 - 0 = 0.52
const pStationary = computePressureFromSpeed([[0, 0, 0.5], [0, 0, 0.5]]);
assert.equal(pStationary, 0.52, "Zero speed returns maximum stationary pressure 0.52");

// Very high speed (dist = 100px >= 10px maxSpeed) -> t = 1 -> pressure = 0.52 - 0.08 = 0.44
const pFast = computePressureFromSpeed([[0, 0, 0.5], [100, 0, 0.5]]);
assert.equal(pFast, 0.44, "High speed returns minimum pressure 0.44");

// Moderate speed (dist = 5px = half maxSpeed) -> t = 0.5 -> pressure = 0.52 - 0.04 = 0.48
const pMed = computePressureFromSpeed([[0, 0, 0.5], [3, 4, 0.5]]); // dist = 5
assert.ok(Math.abs(pMed - 0.48) < 1e-6, "Medium speed returns interpolated pressure ~0.48");
pass("Speed-based pressure calculation is continuous and bounded in [0.44, 0.52]");

// 4.3 Arrowhead helper in stroke.ts
const arrowPath = getArrowheadPath(100, 100, 0, 10);
assert.ok(arrowPath.startsWith("M 100,100 L"), "Arrowhead helper produces valid polygon");
pass("Arrowhead path generator in stroke.ts verified");

console.log(`\n=====`);
console.log(`=== STROKE STRESS SUITE: ALL ${totalChecks} ASSERTIONS PASSED (100%) ===`);
console.log(`=====\n`);

=====
// === END FILE: tests/challenger_stroke_stress.test.ts ===
=====
=====
```

```
// === START FILE: tests/dictation_continuation.test.ts ===
// File 141 of 194 | Lines: 295 | Size: 11340 bytes
=====

/**
 * Voice dictation: cumulative finals, language choice, mic setup.
 *
 * The reported bug: saying "how to do this" writes
 * "how how to how to do how to do this".
 * The tablet engine re-sends the sentence-so-far as a new FINAL
 * ("how", then "how to", then "how to do", ...). The old guard only
 * dropped byte-identical repeats, so every longer copy was appended.
 * The fix: a final that CONTINUES the previous one revises it in place.
 */

import assert from "node:assert/strict";
import {
  VoiceDictationController,
  isContinuationOfFinal,
  spliceDictationChunk,
  SUPPORTED_DICTATION_LANGS,
  resolveDictationLang,
} from "../src/lib/whiteboard/speechRecognition";

const NL = String.fromCharCode(10);

let passed = 0;
let failed = 0;
async function test(name: string, fn: () => void | Promise<void>) {
  try {
    await fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`      ${(err as Error).message}`);
  }
}

// -----
// Stand-in speech engine (same shape as review_fixes.test.ts).
// -----

type Handler = ((ev: unknown) => void) | null;
class FakeRecognition {
  static last: FakeRecognition | null = null;
  continuous = false;
  interimResults = false;
  lang = "";
  maxAlternatives = 1;
  onstart: Handler = null;
  onresult: Handler = null;
  onerror: Handler = null;
  onend: Handler = null;
  onaudiostart: Handler = null;
  onaudioend: Handler = null;
  onsoundstart: Handler = null;
  onsoundend: Handler = null;
  onspeechstart: Handler = null;
  onspeechend: Handler = null;
  onnomatch: Handler = null;
  constructor() {
    FakeRecognition.last = this;
  }
  start() {
    this.onstart?.({});
  }
  /** Engine silence timeout: ends the session, triggering auto-restart. */
  end() {
    this.onend?.({});
  }
  stop() {}
  abort() {}
  addEventListener() {}
  removeEventListener() {}
  dispatchEvent() {
    return true;
  }
}
```

```

    }
    emitFinal(text: string) {
      const alt = { transcript: text, confidence: 0 };
      const result = Object.assign([alt], { isFinal: true, item: () => alt });
      const results = Object.assign([result], { item: () => result });
      this.onresult?.({ resultIndex: 0, results });
    }
  }
}

async function main() {
  console.log(NL + "■■■ Dictation continuation ■■■");

  await test("isContinuationOfFinal: growing prefixes continue, new words do not", () => {
    assert.equal(isContinuationOfFinal("how", 1000, "how to", 1500), true);
    assert.equal(isContinuationOfFinal("how to", 1500, "how to do this", 2000), true);
    assert.equal(isContinuationOfFinal("how", 1000, "second sentence", 1500), false);
    assert.equal(isContinuationOfFinal("buy milk", 1000, "buy milk", 1500), false);
    assert.equal(isContinuationOfFinal("yes", 1000, "yesterday", 1500), false);
    assert.equal(isContinuationOfFinal("how", 1000, "how to", 9000), false);
    assert.equal(isContinuationOfFinal("", 1000, "how", 1500), false);
  });

  await test("isContinuationOfFinal: a revised word still revises, a shrink appends", () => {
    // Tablet case: "are you want to fix the" -> "are you going to fix the problem".
    assert.equal(
      isContinuationOfFinal("are you want to fix the", 1000, "are you going to fix the problem", 1500),
      true
    );
    // A shorter re-send is a new phrase – revising would delete confirmed words.
    assert.equal(isContinuationOfFinal("how to do this", 1000, "how to", 1500), false);
  });

  await test("tablet sentence revises to exactly what was said", async () => {
    (globalThis as unknown as { window: unknown }).window = { webkitSpeechRecognition: FakeRecognition };
    const calls: Array<{ text: string; revisesPrevious: boolean }> = [];
    const c = new VoiceDictationController({
      onFinal: (t, opts) => calls.push({ text: t, revisesPrevious: opts?.revisesPrevious ?? false }),
    });
    assert.equal(await c.start(), true);
    const rec = FakeRecognition.last!;
    rec.emitFinal("are");
    rec.emitFinal("are you");
    rec.emitFinal("are you want to fix");
    rec.emitFinal("are you want to fix the");
    rec.emitFinal("are you going to fix the problem");
    assert.deepEqual(
      calls.map((x) => x.revisesPrevious),
      [false, true, true, true, true]
    );
    let text = "";
    let cursor = 0;
    let range: { start: number; length: number } | null = null;
    for (const { text: chunk, revisesPrevious } of calls) {
      const out = spliceDictationChunk(text, cursor, cursor, chunk, range, revisesPrevious);
      text = out.text;
      cursor = out.cursor;
      range = out.range;
    }
    assert.equal(text, "are you going to fix the problem ");
    c.stop();
  });

  await test("controller flags cumulative finals as revisions, not appends", async () => {
    (globalThis as unknown as { window: unknown }).window = { webkitSpeechRecognition: FakeRecognition };
    const calls: Array<{ text: string; revisesPrevious: boolean }> = [];
    const c = new VoiceDictationController({
      onFinal: (t, opts) => calls.push({ text: t, revisesPrevious: opts?.revisesPrevious ?? false }),
    });
    assert.equal(await c.start(), true);
    const rec = FakeRecognition.last!;
    rec.emitFinal("how");
    rec.emitFinal("how to");
    rec.emitFinal("how to do");
    rec.emitFinal("how to do this");
    assert.deepEqual(calls, [
      { text: "how", revisesPrevious: false },
      { text: "how to", revisesPrevious: true },
      { text: "how to do", revisesPrevious: true },
    ]);
  });
}

```

```

    { text: "how to do this", revisesPrevious: true },
  ]);
  c.stop();
});

await test("revised chunks splice to exactly what was said", () => {
  // Simulates the editor: append first chunk, revise in place after that.
  let text = "";
  let cursor = 0;
  let range: { start: number; length: number } | null = null;
  const chunks: Array<{ chunk: string; revises: boolean }> = [
    { chunk: "how", revises: false },
    { chunk: "how to", revises: true },
    { chunk: "how to do", revises: true },
    { chunk: "how to do this", revises: true },
  ];
  for (const { chunk, revises } of chunks) {
    const out = spliceDictationChunk(text, cursor, cursor, chunk, range, revises);
    text = out.text;
    cursor = out.cursor;
    range = out.range;
  }
  assert.equal(text, "how to do this ");
});

await test("a genuinely new sentence still appends after a revision", () => {
  let text = "";
  let cursor = 0;
  let range: { start: number; length: number } | null = null;
  for (const step of [
    { chunk: "how to do this", revises: false },
    { chunk: "second sentence", revises: false },
  ]) {
    const out = spliceDictationChunk(text, cursor, cursor, step.chunk, range, step.revises);
    text = out.text;
    cursor = out.cursor;
    range = out.range;
  }
  assert.equal(text, "how to do this second sentence ");
});

await test("revision is ignored when the user moved the cursor (falls back to append)", () => {
  const first = spliceDictationChunk("", 0, 0, "how", null, false);
  // User taps back to the start before the revised final arrives.
  const out = spliceDictationChunk(first.text, 0, 0, "how to", first.range, true);
  assert.equal(out.text, "how to how ");
});

await test("pause then continue: a restarted session does not rewrite everything", async () => {
  (globalThis as unknown as { window: unknown }).window = { webkitSpeechRecognition: FakeRecognition };
  const calls: Array<{ text: string; revisesPrevious: boolean }> = [];
  const c = new VoiceDictationController({
    onFinal: (t, opts) => calls.push({ text: t, revisesPrevious: opts?.revisesPrevious ?? false }),
  });
  assert.equal(await c.start(), true);
  const rec = FakeRecognition.last!;
  rec.emitFinal("are you going");
  // A long pause: the engine session ends and auto-restarts behind us.
  const realNow = Date.now;
  try {
    Date.now = () => realNow() + 30000;
    rec.end();
    await new Promise((r) => setTimeout(r, 150));
    const rec2 = FakeRecognition.last!;
    assert.notEqual(rec2, rec, "auto-restart should open a new engine session");
    // Engine catch-up: exact re-send, then a grown re-send, then new words.
    rec2.emitFinal("are you going");
    rec2.emitFinal("are you going home");
    rec2.emitFinal("and then we deploy");
  } finally {
    Date.now = realNow;
  }
  assert.deepEqual(calls, [
    { text: "are you going", revisesPrevious: false },
    { text: "are you going home", revisesPrevious: true },
    { text: "and then we deploy", revisesPrevious: false },
  ]);
  let text = "";

```

```

    let cursor = 0;
    let range: { start: number; length: number } | null = null;
    for (const { text: chunk, revisesPrevious } of calls) {
      const out = spliceDictationChunk(text, cursor, cursor, chunk, range, revisesPrevious);
      text = out.text;
      cursor = out.cursor;
      range = out.range;
    }
    assert.equal(text, "are you going home and then we deploy ");
    c.stop();
  });

  await test("same segment re-sent mid-session after a pause is skipped", async () => {
    (globalThis as unknown as { window: unknown }).window = { webkitSpeechRecognition: FakeRecognition };
    const calls: string[] = [];
    const c = new VoiceDictationController({ onFinal: (t) => calls.push(t) });
    assert.equal(await c.start(), true);
    const rec = FakeRecognition.last!;
    rec.emitFinal("hello there");
    const realNow = Date.now;
    try {
      Date.now = () => realNow() + 30000;
      // Same session, same result index, same words: a list re-send, not speech.
      rec.emitFinal("hello there");
    } finally {
      Date.now = realNow;
    }
    assert.deepEqual(calls, ["hello there"]);
    c.stop();
  });

  await test("a fresh Dictate press repeats nothing: same words append", async () => {
    (globalThis as unknown as { window: unknown }).window = { webkitSpeechRecognition: FakeRecognition };
    const calls: Array<{ text: string; revisesPrevious: boolean }> = [];
    const c = new VoiceDictationController({
      onFinal: (t, opts) => calls.push({ text: t, revisesPrevious: opts?.revisesPrevious ?? false }),
    });
    assert.equal(await c.start(), true);
    FakeRecognition.last!.emitFinal("hello");
    c.stop();
    assert.equal(await c.start(), true);
    FakeRecognition.last!.emitFinal("hello");
    assert.deepEqual(calls, [
      { text: "hello", revisesPrevious: false },
      { text: "hello", revisesPrevious: false },
    ]);
    c.stop();
  });

  await test("supported languages include Hebrew and Arabic, default resolves sensibly", () => {
    const ids = SUPPORTED_DICTATION_LANGS.map((l) => l.id);
    assert.ok(ids.includes("en-US"));
    assert.ok(ids.includes("he-IL"));
    assert.ok(ids.includes("ar-SA"));
    assert.equal(resolveDictationLang("he-IL"), "he-IL");
    assert.equal(resolveDictationLang("he"), "he-IL");
    assert.equal(resolveDictationLang("xx-YY"), "en-US");
    assert.equal(resolveDictationLang(undefined), "en-US");
  });

  console.log(` ${passed} passed, ${failed} failed`);
  if (failed > 0) process.exit(1);
}

void main();

=====
// === END FILE: tests/dictation_continuation.test.ts ===
=====

=====
// === START FILE: tests/dictation_robustness.test.ts ===
// File 142 of 194 | Lines: 304 | Size: 9958 bytes
=====

/**
 * Voice dictation robustness — the "weak and buggy" report.

```

```

*
* - Stop mid-sentence used to unhook the engine at once: the last words
*   were lost. They are written now, and a late final revises them.
* - Every silent pause used an auto-restart and 5 of them switched
*   dictation off (~30 s of thinking). Pauses are limited by time now.
* - One "network" blip ended dictation for good; it is retried now.
* - Words appear in the note while you speak (interim written in place).
*/

import assert from "node:assert/strict";
import {
  VoiceDictationController,
  SILENCE_LIMIT_MS,
  placeInterim,
  spliceDictationChunk,
  stripInterim,
  type DictationRange,
} from "../src/lib/whiteboard/speechRecognition";

const NL = String.fromCharCode(10);

let passed = 0;
let failed = 0;
async function test(name: string, fn: () => void | Promise<void>) {
  try {
    await fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`   ${err as Error}.message`);
  }
}

type Handler = ((ev: unknown) => void) | null;
class FakeRecognition {
  static last: FakeRecognition | null = null;
  static made = 0;
  continuous = false;
  interimResults = false;
  lang = "";
  maxAlternatives = 1;
  stopped = false;
  onstart: Handler = null;
  onresult: Handler = null;
  onerror: Handler = null;
  onend: Handler = null;
  onaudiostart: Handler = null;
  onaudioend: Handler = null;
  onsoundstart: Handler = null;
  onsoundend: Handler = null;
  onspeechstart: Handler = null;
  onspeechend: Handler = null;
  onnomatch: Handler = null;
  constructor() {
    FakeRecognition.last = this;
    FakeRecognition.made++;
  }
  start() {
    this.onstart?.({});
  }
  stop() {
    this.stopped = true;
  }
  abort() {}
  end() {
    this.onend?.({});
  }
  error(code: string) {
    this.onerror?.({ error: code });
  }
  /** One event: results as [isFinal, text] pairs from index `from`. */
  emit(results: Array<[boolean, string]>, from = 0) {
    const list = results.map(([isFinal, text]) => {
      const alt = { transcript: text, confidence: 0 };
      return Object.assign([alt], { isFinal, item: () => alt });
    });
  }
}

```

```

    this.onresult?.({ resultIndex: from, results: Object.assign(list, { item: (i: number) => list[i] }) });
  }
}

const tick = (ms = 0) => new Promise((r) => setTimeout(r, ms));

function install() {
  (globalThis as unknown as { window: unknown }).window = { webkitSpeechRecognition: FakeRecognition };
}

/** Plays controller output into a text the way the editor does. */
function editorSim() {
  let text = "";
  let cursor = 0;
  let last: DictationRange | null = null;
  let interim: DictationRange | null = null;
  return {
    onInterim(words: string) {
      if (!interim && !words.trim()) return;
      const out = placeInterim(text, cursor, cursor, interim, words);
      text = out.text;
      cursor = out.cursor;
      interim = out.range;
    },
    onFinal(chunk: string, revises: boolean) {
      const base = stripInterim(text, cursor, cursor, interim);
      interim = null;
      const out = spliceDictationChunk(base.value, base.selStart, base.selEnd, chunk, last, revises);
      text = out.text;
      cursor = out.cursor;
      last = out.range;
    },
    get text() {
      return text;
    },
  };
}

async function main() {
  console.log(NL + "■■ Dictation robustness ■■");

  await test("Stop mid-sentence keeps the words being said", async () => {
    install();
    const ed = editorSim();
    const c = new VoiceDictationController({
      onInterim: (w) => ed.onInterim(w),
      onFinal: (t, o) => ed.onFinal(t, o?.revisesPrevious ?? false),
    });
    await c.start();
    const rec = FakeRecognition.last!;
    rec.emit([[true, "first sentence"]]);
    rec.emit([[true, "first sentence"], [false, "and the last"]], 1);
    c.stop();
    assert.equal(ed.text, "first sentence and the last ");
    assert.equal(rec.stopped, true);
  });

  await test("the engine's own final after Stop revises the flushed words, never doubles them", async () => {
    install();
    const ed = editorSim();
    const c = new VoiceDictationController({
      onInterim: (w) => ed.onInterim(w),
      onFinal: (t, o) => ed.onFinal(t, o?.revisesPrevious ?? false),
    });
    await c.start();
    const rec = FakeRecognition.last!;
    rec.emit([[false, "see you tomor"]]);
    c.stop();
    // Moments later the engine delivers the real final for that phrase.
    rec.emit([[true, "see you tomorrow"]]);
    assert.equal(ed.text, "see you tomorrow ");
    rec.end();
  });

  await test("a final exactly equal to the flushed words is not written twice", async () => {
    install();
    const ed = editorSim();
    const c = new VoiceDictationController({

```

```

    onInterim: (w) => ed.onInterim(w),
    onFinal: (t, o) => ed.onFinal(t, o?.revisesPrevious ?? false),
  });
  await c.start();
  const rec = FakeRecognition.last!;
  rec.emit([[false, "buy milk"]]);
  c.stop();
  rec.emit([[true, "buy milk"]]);
  assert.equal(ed.text, "buy milk ");
});

await test("words appear in the note while speaking, then settle to the final", async () => {
  install();
  const seen: string[] = [];
  const ed = editorSim();
  const c = new VoiceDictationController({
    onInterim: (w) => {
      ed.onInterim(w);
      seen.push(ed.text);
    },
    onFinal: (t, o) => ed.onFinal(t, o?.revisesPrevious ?? false),
  });
  await c.start();
  const rec = FakeRecognition.last!;
  rec.emit([[false, "how"]]);
  rec.emit([[false, "how are"]]);
  rec.emit([[false, "how are you"]]);
  rec.emit([[true, "how are you"]]);
  assert.ok(seen.includes("how are"), `live text: ${JSON.stringify(seen)}`);
  assert.equal(ed.text, "how are you ");
  c.abort();
});

await test("thinking in silence does not switch dictation off after 5 pauses", async () => {
  install();
  let status = "";
  const c = new VoiceDictationController({ onStatusChange: (s) => (status = s) });
  await c.start();
  const realNow = Date.now;
  let offset = 0;
  try {
    Date.now = () => realNow() + offset;
    for (let i = 0; i < 10; i++) {
      offset += 6000; // each silent session lasts ~6 s before Android ends it
      FakeRecognition.last!.end();
      await tick(1050);
    }
    assert.equal(status, "listening", "still listening after 60 s of pauses");
  } finally {
    Date.now = realNow;
    c.abort();
  }
});

await test("long silence ends dictation with a message, not silently", async () => {
  install();
  let status = "";
  const notes: string[] = [];
  const c = new VoiceDictationController({ onStatusChange: (s) => (status = s), onError: (m) => notes.push(m) });
  await c.start();
  const realNow = Date.now;
  try {
    Date.now = () => realNow() + SILENCE_LIMIT_MS + 1000;
    FakeRecognition.last!.end();
  } finally {
    Date.now = realNow;
  }
  assert.equal(status, "idle");
  assert.ok(notes.some((n) => /silence/i.test(n)), JSON.stringify(notes));
});

await test("an engine that dies instantly is given up on, with a message", async () => {
  install();
  const notes: string[] = [];
  let status = "";
  const c = new VoiceDictationController({ onStatusChange: (s) => (status = s), onError: (m) => notes.push(m) });
});

```

```

    await c.start();
    for (let i = 0; i < 6 && status === "listening"; i++) {
      FakeRecognition.last!.end(); // ends at once, nothing heard
      await tick(1050);
    }
    assert.equal(status, "idle");
    assert.ok(notes.some((n) => /keeps stopping/i.test(n)), JSON.stringify(notes));
  });

  await test("one network blip is retried; dictation keeps going", async () => {
    install();
    let status = "";
    const c = new VoiceDictationController({ onStatusChange: (s) => (status = s) });
    await c.start();
    const before = FakeRecognition.made;
    FakeRecognition.last!.error("network");
    FakeRecognition.last!.end();
    await tick(1050);
    assert.equal(status, "listening");
    assert.ok(FakeRecognition.made > before, "a new session was started");
    c.abort();
  });

  await test("repeated network failures are reported", async () => {
    install();
    let status = "";
    const notes: string[] = [];
    const c = new VoiceDictationController({ onStatusChange: (s) => (status = s), onError: (m) => notes.push(m)
    });
    await c.start();
    for (let i = 0; i < 4; i++) FakeRecognition.last!.error("network");
    assert.equal(status, "error");
    assert.ok(notes.length > 0);
    c.abort();
  });

  await test("a session ending mid-phrase keeps the words heard so far", async () => {
    install();
    const ed = editorSim();
    const c = new VoiceDictationController({
      onInterim: (w) => ed.onInterim(w),
      onFinal: (t, o) => ed.onFinal(t, o?.revisesPrevious ?? false),
    });
    await c.start();
    const rec = FakeRecognition.last!;
    rec.emit([[false, "the meeting is at"]]);
    rec.end(); // Android cut the session before sending a final
    assert.equal(ed.text, "the meeting is at ");
    c.abort();
  });

  await test("typed text around the live words is never eaten", () => {
    // Existing text, caret at the end; interim shown, then replaced by final.
    const a = placeInterim("Notes:", 6, 6, null, "hello");
    assert.equal(a.text, "Notes: hello");
    const b = placeInterim(a.text, a.cursor, a.cursor, a.range, "hello world");
    assert.equal(b.text, "Notes: hello world");
    const s = stripInterim(b.text, b.cursor, b.cursor, b.range);
    assert.equal(s.value, "Notes:");
    assert.equal(s.selStart, 6);
    const cleared = placeInterim(b.text, b.cursor, b.cursor, b.range, "");
    assert.equal(cleared.text, "Notes:");
    assert.equal(cleared.range, null);
  });

  console.log(` ${passed} passed, ${failed} failed`);
  if (failed > 0) process.exit(1);
  process.exit(0);
}

main();

=====
// === END FILE: tests/dictation_robustness.test.ts ===
=====
=====

```

```
// === START FILE: tests/duplicate_notebook.test.ts ===
// File 143 of 194 | Lines: 496 | Size: 17813 bytes
=====

/**
 * Tests for Duplicating a Notebook with Page Range & Destination,
 * and Single-Step Undo/Redo for Notebook Page Merges.
 */

import assert from "node:assert/strict";
import { useWhiteboard } from "../src/store/whiteboard";
import { getPages } from "../src/lib/whiteboard/notebook";
import type {
  Board,
  ConnectionElement,
  NotebookPage,
  StrokeElement,
} from "../src/lib/whiteboard/types";
import { cleanHistoryEntry } from "../src/lib/whiteboard/persistence";

let passed = 0;
let failed = 0;
function test(name: string, fn: () => void) {
  try {
    fn();
    passed++;
    console.log(` ✓ ${name}`);
  } catch (err) {
    failed++;
    console.error(` ✗ ${name}`);
    console.error(`${(err as Error).message}`);
  }
}

const S = () => useWhiteboard.getState();
const getBoard = (id: string): Board => S().boards.find((b) => b.id === id!);

let seq = 0;
function makeBlankNotebook(name: string, pageCount: number): Board {
  const id = `nb_${Date.now()}_${++seq}`;
  const pages: NotebookPage[] = [];
  for (let i = 0; i < pageCount; i++) {
    pages.push({
      id: `page_${id}_${i}`,
      elements: [],
      createdAt: 1000 + i,
      updatedAt: 1000 + i,
    });
  }
  return {
    id,
    name,
    kind: "notebook",
    pageSize: "a4",
    folderId: null,
    elements: [],
    pages,
    activePageId: pages[0]?.id ?? "",
    camera: { x: 0, y: 0, zoom: 1 },
    createdAt: 1000,
    updatedAt: 1000,
  };
}

function makeStroke(id: string, x: number, y: number): StrokeElement {
  return {
    id,
    type: "stroke",
    x,
    y,
    rotation: 0,
    opacity: 1,
    color: "#000000",
    strokeWidth: 2,
    strokeStyle: "ballpoint",
    points: [[0, 0, 0.5], [10, 10, 0.5]],
    createdAt: 1000,
    updatedAt: 1000,
  };
}
```

```

    };
  }

function makeConnection(id: string, fromId: string, toId: string, boardId: string): ConnectionElement {
  return {
    id,
    type: "connection",
    x: 0,
    y: 0,
    rotation: 0,
    opacity: 1,
    source: { boardId, elementId: fromId, anchor: "center" },
    target: { boardId, elementId: toId, anchor: "center" },
    routing: "straight",
    linkType: "relates",
    dashStyle: "solid",
    startArrowhead: "none",
    endArrowhead: "arrow",
    strokeWidth: 2,
    createdAt: 1000,
    updatedAt: 1000,
  } as ConnectionElement;
}

function resetStoreWithBoards(boards: Board[], activeId?: string) {
  useWhiteboard.setState({
    boards,
    activeBoardId: activeId ?? boards[0]?.id ?? null,
    folders: [
      {
        id: "folder_1",
        name: "Semester 1",
        parentId: null,
        expanded: true,
        createdAt: 1000,
      },
    ],
    selectedIds: [],
    editingId: null,
    history: { past: [], future: [] },
  });
}

console.log("\n■■ Duplicate Notebook & Single-Step Merge Undo Tests ■■");

test("duplicateNotebook copies only chosen page indices into a new notebook in target folder", () => {
  const nb = makeBlankNotebook("Calculus", 5);
  const nbPages = getPages(nb);
  nbPages.forEach((p, idx) => {
    p.elements.push(makeStroke(`el_${idx}`, 10 * idx, 10 * idx));
  });

  resetStoreWithBoards([nb], nb.id);

  // Duplicate pages 1 and 3 (0-based indices 1 and 3) to folder_1
  const copyId = S().duplicateNotebook(nb.id, {
    name: "Calculus Subset",
    folderId: "folder_1",
    pageIndices: [1, 3],
  });

  assert.ok(copyId, "Must return new notebook id");
  const copy = getBoard(copyId!);
  assert.ok(copy, "New notebook must exist in store");
  assert.equal(copy.name, "Calculus Subset");
  assert.equal(copy.folderId, "folder_1");
  assert.equal(copy.kind, "notebook");
  assert.equal(copy.elements.length, 0, "board.elements must be strictly empty for notebooks");

  const copyPages = getPages(copy);
  assert.equal(copyPages.length, 2, "Must contain exactly 2 pages");
  assert.equal(copyPages[0].elements[0].id.startsWith("el_1"), false, "Element ID must be remapped");
  assert.equal(copyPages[0].elements[0].x, 10, "First copied page should have page index 1's content");
  assert.equal(copyPages[1].elements[0].x, 30, "Second copied page should have page index 3's content");

  // Verify active board was NOT switched
  assert.equal(S().activeBoardId, nb.id, "Active board must not switch on duplicate");
});

```

```

test("duplicateNotebook ensures every copied element receives a fresh id", () => {
  const nb = makeBlankNotebook("Physics", 2);
  const nbPages = getPages(nb);
  nbPages[0].elements.push(makeStroke("orig_stroke_1", 0, 0));

  resetStoreWithBoards([nb], nb.id);

  const copyId = S().duplicateNotebook(nb.id, {
    name: "Physics Copy",
    folderId: null,
    pageIndices: [0],
  });

  const copy = getBoard(copyId);
  const copyPages = getPages(copy);
  const copyElem = copyPages[0].elements[0];
  assert.notEqual(copyElem.id, "orig_stroke_1", "Copied element must have a fresh id");
  assert.equal(copyElem.type, "stroke");
});

test("duplicateNotebook remaps connections between copied elements and prunes dangling connections", () => {
  const nb = makeBlankNotebook("Diagrams", 2);
  const strokeA = makeStroke("stroke_a", 10, 10);
  const strokeB = makeStroke("stroke_b", 100, 100);
  const internalConn = makeConnection("conn_ab", "stroke_a", "stroke_b", nb.id);
  const strokeOnPage2 = makeStroke("stroke_page2", 200, 200);
  const crossPageConn = makeConnection("conn_cross", "stroke_a", "stroke_page2", nb.id);

  const nbPages = getPages(nb);
  nbPages[0].elements.push(strokeA, strokeB, internalConn, crossPageConn);
  nbPages[1].elements.push(strokeOnPage2);

  resetStoreWithBoards([nb], nb.id);

  // Duplicate ONLY Page 0
  const copyId = S().duplicateNotebook(nb.id, {
    name: "Diagrams Page 1 Only",
    folderId: null,
    pageIndices: [0],
  });

  const copy = getBoard(copyId);
  const copyPages = getPages(copy);
  const elements = copyPages[0].elements;

  // The internal connection between A and B must be remapped and kept
  const remappedConn = elements.find((el) => el.type === "connection") as ConnectionElement;
  assert.ok(remappedConn, "Internal connection must be preserved");
  assert.notEqual(remappedConn.id, "conn_ab", "Connection must have new id");
  assert.notEqual(remappedConn.source.elementId, "stroke_a");
  assert.notEqual(remappedConn.target.elementId, "stroke_b");
  assert.equal(remappedConn.source.boardId, copyId);
  assert.equal(remappedConn.target.boardId, copyId);

  // The cross-page connection to Page 2 must be pruned
  const crossConnInCopy = elements.find((el) => el.id === "conn_cross");
  assert.equal(crossConnInCopy, undefined, "Dangling connection must be pruned");
  assert.equal(elements.filter((el) => el.type === "connection").length, 1);
});

test("duplicateNotebook preserves referenced pdfDocIds on the new board and omits unreferenced ones", () => {
  const nb = makeBlankNotebook("Lecture", 2);
  nb.pdfDocIds = ["pdf_lecture_notes_1", "pdf_unreferenced_page2"];
  const nbPages = getPages(nb);
  nbPages[0].pdf = { docId: "pdf_lecture_notes_1", pageNumber: 3 };
  nbPages[1].pdf = { docId: "pdf_unreferenced_page2", pageNumber: 1 };

  resetStoreWithBoards([nb], nb.id);

  const copyId = S().duplicateNotebook(nb.id, {
    name: "Lecture Page 1",
    folderId: null,
    pageIndices: [0],
  });

  const copy = getBoard(copyId);
  // Only the referenced PDF doc must be preserved, not unreferenced ones

```

```
assert.deepEqual(copy.pdfDocIds, ["pdf_lecture_notes_1"]);
const copyPages = getPages(copy);
assert.equal(copyPages[0].pdf?.docId, "pdf_lecture_notes_1");
assert.equal(copyPages[0].pdf?.pageNumber, 3);
});

test("duplicateNotebook leaves the source notebook strictly unmodified (byte-identical)", () => {
  const nb = makeBlankNotebook("Original", 3);
  const nbPages = getPages(nb);
  nbPages[0].elements.push(makeStroke("el1", 5, 5));

  resetStoreWithBoards([nb], nb.id);
  const snapshotBefore = JSON.stringify(getBoard(nb.id));

  S().duplicateNotebook(nb.id, {
    name: "Copy",
    folderId: null,
    pageIndices: [0, 1],
  });

  const snapshotAfter = JSON.stringify(getBoard(nb.id));
  assert.equal(snapshotBefore, snapshotAfter, "Source notebook must be byte-identical after duplicate");
});

test("insertPagesFromBoard into arbitrary target notebook is undoable in exactly ONE step", () => {
  const source = makeBlankNotebook("Source", 3);
  const sourcePages = getPages(source);
  sourcePages[0].pdf = { docId: "pdf_from_source", pageNumber: 1 };
  const target = makeBlankNotebook("Target", 2);
  target.pdfDocIds = [];

  resetStoreWithBoards([source, target], target.id);

  const initialTargetPages = JSON.stringify(getPages(getBoard(target.id)));

  // Merge pages 0 and 2 from source into target at index 1 (between page 1 and page 2)
  const insertedCount = S().insertPagesFromBoard(
    source.id,
    { indices: [0, 2] },
    1,
    target.id
  );

  assert.equal(insertedCount, 2, "Must insert 2 pages");
  const postMergePages = getPages(getBoard(target.id));
  assert.equal(postMergePages.length, 4, "Target must now have 4 pages");
  assert.deepEqual(getBoard(target.id).pdfDocIds, ["pdf_from_source"]);

  // Verify exactly 1 history entry was added
  const history = S().history;
  assert.equal(history.past.length, 1, "Exactly one history entry must be pushed for the merge");
  assert.equal(history.past[0].boardId, target.id);
  assert.ok(history.past[0].pages, "History entry must carry pages snapshot");
  assert.equal(history.past[0].pages?.length, 2, "Snapshot must preserve 2 pre-merge pages");

  // Verify source notebook does NOT appear in history at all
  assert.ok(
    history.past.every((e) => e.boardId !== source.id),
    "Source notebook must not appear in history"
  );

  // Press UNDO once: must completely revert the merge AND the pdfDocIds
  S().undo();

  const undoneTargetPages = JSON.stringify(getPages(getBoard(target.id)));
  assert.equal(undoneTargetPages, initialTargetPages, "Target notebook must revert to exact pre-merge pages");
  assert.deepEqual(getBoard(target.id).pdfDocIds, [], "Undo must restore pre-merge pdfDocIds");
  assert.equal(S().history.past.length, 0, "Past history must be empty after undo");
  assert.equal(S().history.future.length, 1, "Future history must have 1 redo entry");

  // Press REDO: must symmetrically restore the merged pages AND pdfDocIds
  S().redo();

  const redoneTargetPages = getPages(getBoard(target.id));
  assert.equal(redoneTargetPages.length, 4, "Redo must restore 4 pages");
  assert.deepEqual(getBoard(target.id).pdfDocIds, ["pdf_from_source"]);

  // Press UNDO again: reverts again
```

```

S().undo();
assert.equal(
  JSON.stringify(getPages(getBoard(target.id))),
  initialTargetPages,
  "Second undo must revert back to pre-merge pages"
);
assert.deepEqual(getBoard(target.id).pdfDocIds, []);
});

test("merging into an inactive notebook leaves active board unchanged, and undo cleanly reverts the inactive notebook", () => {
  const activeNb = makeBlankNotebook("ActiveNotebook", 2);
  const inactiveTarget = makeBlankNotebook("InactiveTarget", 2);
  const sourceNb = makeBlankNotebook("SourceNotebook", 3);

  resetStoreWithBoards([activeNb, inactiveTarget, sourceNb], activeNb.id);

  // Set camera and selection on active board
  useWhiteboard.setState({
    camera: { x: 42, y: 99, zoom: 1.5 },
    selectedIds: ["dummy_sel"],
  });

  const preMergeInactive = JSON.stringify(getPages(getBoard(inactiveTarget.id)));

  // Merge into inactiveTarget
  S().insertPagesFromBoard(sourceNb.id, { indices: [1] }, 2, inactiveTarget.id);

  // Verify active board state is completely untouched
  assert.equal(S().activeBoardId, activeNb.id, "Active board ID must remain untouched");
  assert.deepEqual(S().camera, { x: 42, y: 99, zoom: 1.5 }, "Camera must remain untouched");
  assert.deepEqual(S().selectedIds, ["dummy_sel"], "Selected IDs must remain untouched");

  // Verify inactive target received the page
  assert.equal(getPages(getBoard(inactiveTarget.id)).length, 3);

  // Press undo
  S().undo();

  // Inactive target reverted
  assert.equal(
    JSON.stringify(getPages(getBoard(inactiveTarget.id))),
    preMergeInactive,
    "Inactive target must revert to pre-merge state in background"
  );
  // Active board still untouched
  assert.equal(S().activeBoardId, activeNb.id);
  assert.deepEqual(S().camera, { x: 42, y: 99, zoom: 1.5 });
  assert.deepEqual(S().selectedIds, ["dummy_sel"]);
});

test("duplicateNotebook on a canvas remaps connections to the new board with fresh ids", () => {
  const canvasId = `canvas_${Date.now()}_${++seq}`;
  const strokeA = makeStroke("c_stroke_a", 10, 10);
  const strokeB = makeStroke("c_stroke_b", 100, 100);
  const conn = makeConnection("c_conn_ab", "c_stroke_a", "c_stroke_b", canvasId);
  const canvas: Board = {
    id: canvasId,
    name: "Sketch",
    kind: "canvas",
    folderId: null,
    elements: [strokeA, strokeB, conn],
    camera: { x: 0, y: 0, zoom: 1 },
    createdAt: 1000,
    updatedAt: 1000,
  };

  resetStoreWithBoards([canvas], canvas.id);

  const copyId = S().duplicateNotebook(canvas.id, {
    name: "Sketch Copy",
    folderId: null,
  });

  const copy = getBoard(copyId);
  assert.equal(copy.kind, "notebook");
  assert.equal(copy.elements.length, 0, "board.elements must be strictly empty for notebooks");
  const copyPages = getPages(copy);

```

```

assert.equal(copyPages.length, 1, "Canvas duplicate must become a single-page notebook");
const elements = copyPages[0].elements;
assert.equal(elements.length, 3, "All canvas elements must be carried over");

// Every element id must be fresh – none of the canvas ids may survive.
const ids = new Set(elements.map((el) => el.id));
assert.equal(ids.size, 3, "Copied element ids must be unique");
for (const oldId of ["c_stroke_a", "c_stroke_b", "c_conn_ab"]) {
  assert.ok(!ids.has(oldId), `Old id ${oldId} must not survive the copy`);
}

// The connection must point at the NEW board and the NEW element ids.
const copyConn = elements.find((el) => el.type === "connection") as ConnectionElement;
assert.ok(copyConn, "Connection must survive the canvas copy");
assert.equal(copyConn.source.boardId, copyId, "Connection source must target the new board");
assert.equal(copyConn.target.boardId, copyId, "Connection target must target the new board");
assert.ok(ids.has(copyConn.source.elementId), "Connection source must reference a copied element");
assert.ok(ids.has(copyConn.target.elementId), "Connection target must reference a copied element");

// Source canvas must be untouched.
assert.equal(getBoard(canvasId).elements.length, 3, "Source canvas must be unmodified");
});

test("duplicateNotebook dedupes repeated page indices instead of colliding element ids", () => {
  const nb = makeBlankNotebook("Repeat", 2);
  const nbPages = getPages(nb);
  nbPages[0].elements.push(makeStroke("rep_el_0", 1, 1));
  nbPages[1].elements.push(makeStroke("rep_el_1", 2, 2));

  resetStoreWithBoards([nb], nb.id);

  const copyId = S().duplicateNotebook(nb.id, {
    name: "Repeat Copy",
    folderId: null,
    pageIndices: [0, 0, 1],
  })!;

  const copy = getBoard(copyId);
  const copyPages = getPages(copy);
  assert.equal(copyPages.length, 2, "Repeated indices must be deduped to 2 pages");
  const allIds = copyPages.flatMap((p) => p.elements.map((el) => el.id));
  assert.equal(new Set(allIds).size, allIds.length, "Element ids must be unique across copied pages");
});

test("insertPagesFromBoard dedupes repeated indices into a single insertion", () => {
  const source = makeBlankNotebook("DupSource", 2);
  const target = makeBlankNotebook("DupTarget", 1);

  resetStoreWithBoards([source, target], target.id);
  const before = S().history.past.length;

  const inserted = S().insertPagesFromBoard(source.id, { indices: [0, 0, 1] }, 1, target.id);

  assert.equal(inserted, 2, "Repeated indices must insert 2 pages, not 3");
  assert.equal(getPages(getBoard(target.id)).length, 3, "Target must grow by exactly 2 pages");
  assert.equal(S().history.past.length, before + 1, "Deduped merge must still push exactly one history entry");
  ;

  S().undo();
  assert.equal(getPages(getBoard(target.id)).length, 1, "Single undo must revert the deduped merge");
});

test("persistence cleanHistoryEntry validates both legacy elements-only entries and pages entries", () => {
  // Legacy entry with elements only
  const legacyRaw = {
    boardId: "board_123",
    pageId: "page_1",
    elements: [
      { id: "e1", type: "stroke" },
      { id: "e2", type: "text" },
    ],
    at: 5000,
  };
  const legacyCleaned = cleanHistoryEntry(legacyRaw);
  assert.ok(legacyCleaned, "Legacy entry must be valid");
  assert.equal(legacyCleaned?.boardId, "board_123");
  assert.equal(legacyCleaned?.elements.length, 2);
  assert.equal(legacyCleaned?.pages, undefined);

```

```
// New entry with pages snapshot
const newRaw = {
  boardId: "board_456",
  pageId: "page_2",
  elements: [],
  pages: [
    {
      id: "p1",
      elements: [{ id: "el_p1", type: "stroke" }],
    },
    {
      id: "p2",
      elements: [],
    },
  ],
  at: 6000,
};
const newCleaned = cleanHistoryEntry(newRaw);
assert.ok(newCleaned, "Pages history entry must be valid");
assert.equal(newCleaned?.boardId, "board_456");
assert.equal(newCleaned?.pages?.length, 2);
assert.equal(newCleaned?.pages?.[0].elements[0].id, "el_p1");

// Corrupt entries must be rejected safely
assert.equal(cleanHistoryEntry(null), null);
assert.equal(cleanHistoryEntry({}), null);
assert.equal(cleanHistoryEntry({ boardId: 123 }), null);
assert.equal(cleanHistoryEntry({ boardId: "b", elements: "not_array" }), null);
assert.equal(cleanHistoryEntry({ boardId: "b", elements: [{ id: 123 }] }), null);
});

console.log(`\n${passed} passed, ${failed} failed.`);
if (failed > 0) process.exit(1);

=====
// === END FILE: tests/duplicate_notebook.test.ts ===
=====

=====
// === START FILE: tests/e2e/m3-pwa-branding.test.js ===
// File 144 of 194 | Lines: 1080 | Size: 37790 bytes
=====

/**
 * Milestone 3 Empirical Test Suite: Branding & PWA Integration
 * =====
 * Comprehensive, multi-tiered test suite verifying:
 * 1. Service Worker caching strategies, SWR, offline fallbacks, filters (public/sw.js)
 * 2. PWA registration lifecycle, beforeinstallprompt, appinstalled (PWABuilder.tsx)
 * 3. Next.js 15 metadata, viewport, icon declarations (src/app/layout.tsx)
 * 4. PWA Web App Manifest schema & installability (public/manifest.json)
 * 5. Physical image asset dimensions, color channels, binary ICO headers (public/*)
 * 6. Global ProNotes branding consistency across UI, configs, and scripts
 * 7. Stress Tests, Fault Injection & Edge Cases
 */

const fs = require('fs');
const path = require('path');
const { execSync } = require('child_process');
const sharp = require('sharp');
const React = require('react');
const {
  describe,
  it,
  expect,
  beforeEach,
  afterEach,
  setupMockDom,
  loadTsModule,
  clearModuleCache,
  ctx
} = require('./test-helpers');

// The service worker's cache key, read from sw.js itself. These tests used to
// hardcode the very first key; the worker is bumped on every deploy (that is how it
// evicts stale chunks), so the frozen key failed every SW test once the suite
```

```
// stopped being run - and nobody noticed, because nothing ran it.
const SW_CACHE = (() => {
  const src = fs.readFileSync(path.join(__dirname, '..', '..', 'public', 'sw.js'), 'utf-8');
  const m = src.match(/const CACHE_VERSION = '([^']+)'/);
  if (!m) throw new Error('sw.js no longer declares CACHE_VERSION - update this suite');
  return m[1];
})();

const ROOT_DIR = path.resolve(__dirname, '../..');
const PUBLIC_DIR = path.resolve(ROOT_DIR, 'public');

// =====
// Mock Service Worker Environment
// =====

class MockCache {
  constructor(name) {
    this.name = name;
    this.store = new Map();
  }

  async add(url) {
    this.store.set(url, new MockResponse(`content-for-${url}`, { status: 200 }));
  }

  async addAll(urls) {
    for (const url of urls) {
      await this.add(url);
    }
  }

  async match(request) {
    const key = typeof request === 'string' ? request : request.url;
    if (this.store.has(key)) return this.store.get(key);
    try {
      const u = new URL(key, 'http://localhost:3000');
      if (this.store.has(u.pathname)) return this.store.get(u.pathname);
      if (this.store.has(u.href)) return this.store.get(u.href);
    } catch {
      // ignore
    }
    return undefined;
  }

  async put(request, response) {
    const key = typeof request === 'string' ? request : request.url;
    this.store.set(key, response);
  }

  async delete(request) {
    const key = typeof request === 'string' ? request : request.url;
    return this.store.delete(key);
  }
}

class MockCacheStorage {
  constructor() {
    this.caches = new Map();
  }

  async open(cacheName) {
    if (!this.caches.has(cacheName)) {
      this.caches.set(cacheName, new MockCache(cacheName));
    }
    return this.caches.get(cacheName);
  }

  async match(request) {
    for (const cache of this.caches.values()) {
      const match = await cache.match(request);
      if (match) return match;
    }
    return undefined;
  }

  async keys() {
    return Array.from(this.caches.keys());
  }
}
```

```
    async delete(cacheName) {
      return this.caches.delete(cacheName);
    }
  }

class MockResponse {
  constructor(body = '', options = {}) {
    this.body = body;
    this.status = options.status !== undefined ? options.status : 200;
    this.statusText = options.statusText || 'OK';
    this.headers = options.headers || {};
  }

  clone() {
    return new MockResponse(this.body, {
      status: this.status,
      statusText: this.statusText,
      headers: { ...this.headers }
    });
  }

  async text() {
    return this.body;
  }
}

class MockRequest {
  constructor(input, init = {}) {
    this.url = typeof input === 'string' ? input : input.url;
    this.method = (init.method || 'GET').toUpperCase();
    this.mode = init.mode || 'cors';
    // A real Request's headers are a Headers object. sw.js now reads
    // `request.headers.get('RSC')` to keep App Router payloads out of the
    // cache, and a plain object made every fetch-strategy test throw.
    const raw = init.headers || {};
    const lower = new Map(Object.entries(raw).map(([k, v]) => [k.toLowerCase(), String(v)]));
    this.headers = {
      get: (name) => (lower.has(name.toLowerCase()) ? lower.get(name.toLowerCase()) : null),
      has: (name) => lower.has(name.toLowerCase()),
    };
  }
}

class MockExtendableEvent {
  constructor(type) {
    this.type = type;
    this.promises = [];
  }

  waitUntil(p) {
    this.promises.push(Promise.resolve(p));
  }

  async wait() {
    await Promise.all(this.promises);
  }
}

class MockFetchEvent extends MockExtendableEvent {
  constructor(request) {
    super('fetch');
    this.request = request;
    this.responsePromise = null;
    this.responded = false;
  }

  respondWith(p) {
    this.responded = true;
    this.responsePromise = Promise.resolve(p);
  }

  async getResponse() {
    if (!this.responsePromise) return null;
    return await this.responsePromise;
  }
}
```

```
function createSwEnvironment() {
  const listeners = new Map();
  const caches = new MockCacheStorage();
  let skipWaitingCalled = false;
  let claimCalled = false;

  const mockSelf = {
    addEventListener: (type, fn) => {
      if (!listeners.has(type)) listeners.set(type, []);
      listeners.get(type).push(fn);
    },
    skipWaiting: () => {
      skipWaitingCalled = true;
      return Promise.resolve();
    },
    clients: {
      claim: () => {
        claimCalled = true;
        return Promise.resolve();
      }
    }
  };
};

const swCode = fs.readFileSync(path.join(PUBLIC_DIR, 'sw.js'), 'utf-8');

// Helper to run SW
const runInSw = (fetchMock) => {
  const fn = new Function(
    'self',
    'caches',
    'fetch',
    'Request',
    'Response',
    'URL',
    swCode
  );
  fn(
    mockSelf,
    caches,
    fetchMock,
    MockRequest,
    MockResponse,
    URL
  );
};

return {
  listeners,
  caches,
  mockSelf,
  getSkipWaitingCalled: () => skipWaitingCalled,
  getClaimCalled: () => claimCalled,
  runInSw
};
}

// Helper to run React component effects
function mountComponent(compFn) {
  let cleanupFn = null;
  const originalUseEffect = React.useEffect;
  React.useEffect = (effectCallback) => {
    cleanupFn = effectCallback();
  };
  try {
    compFn();
  } finally {
    React.useEffect = originalUseEffect;
  }
  return {
    unmount: () => {
      if (typeof cleanupFn === 'function') {
        cleanupFn();
      }
    }
  };
};
}

// =====
```

```
// TEST SUITES
// =====

describe('M3.1 – Service Worker Implementation (public/sw.js)', () => {
  it('sw.js file exists and has non-empty content', () => {
    const swPath = path.join(PUBLIC_DIR, 'sw.js');
    expect(fs.existsSync(swPath)).toBe(true);
    const content = fs.readFileSync(swPath, 'utf-8');
    expect(content.length).toBeGreaterThan(100);
    expect(content).toContain(SW_CACHE);
  });

  it('declares all required STATIC_ASSETS and all files exist on disk', () => {
    const swContent = fs.readFileSync(path.join(PUBLIC_DIR, 'sw.js'), 'utf-8');
    const requiredAssets = [
      '/',
      '/manifest.json',
      '/favicon.ico',
      '/favicon.png',
      '/favicon-32x32.png',
      '/favicon-16x16.png',
      '/icon-192.png',
      '/icon-512.png',
      '/icon-192-maskable.png',
      '/icon-512-maskable.png',
      '/apple-touch-icon.png',
      '/logo.png',
    ];

    for (const asset of requiredAssets) {
      expect(swContent).toContain(`${asset}`);
      if (asset !== '/') {
        const filePath = path.join(PUBLIC_DIR, asset.replace('/', ''));
        expect(fs.existsSync(filePath)).toBe(true);
      }
    }
  });

  it('handles "install" event: pre-caches STATIC_ASSETS and calls skipWaiting', async () => {
    const env = createSwEnvironment();
    env.runInSw(async () => new MockResponse());

    const installListeners = env.listeners.get('install') || [];
    expect(installListeners.length).toBeGreaterThan(0);

    const installEvent = new MockExtendableEvent('install');
    installListeners[0](installEvent);
    await installEvent.wait();

    expect(env.getSkipWaitingCalled()).toBe(true);

    const cache = await env.caches.open(SW_CACHE);
    const rootCached = await cache.match('/');
    expect(rootCached).toBeDefined();
    const manifestCached = await cache.match('/manifest.json');
    expect(manifestCached).toBeDefined();
    const iconCached = await cache.match('/icon-192.png');
    expect(iconCached).toBeDefined();
  });

  it('handles "activate" event: purges stale caches and claims clients', async () => {
    const env = createSwEnvironment();
    env.runInSw(async () => new MockResponse());

    // Populate old caches
    const oldCache1 = await env.caches.open('pronotes-v0');
    await oldCache1.put('/old.js', new MockResponse('old'));
    const oldCache2 = await env.caches.open('legacy-cache');
    await oldCache2.put('/legacy.js', new MockResponse('legacy'));
    const currentCache = await env.caches.open(SW_CACHE);
    await currentCache.put('/current.js', new MockResponse('current'));

    const activateListeners = env.listeners.get('activate') || [];
    expect(activateListeners.length).toBeGreaterThan(0);

    const activateEvent = new MockExtendableEvent('activate');
    activateListeners[0](activateEvent);
    await activateEvent.wait();
  });
});
```

```

    expect(env.getClaimCalled()).toBe(true);

    const remainingKeys = await env.caches.keys();
    expect(remainingKeys).toContain(SW_CACHE);
    expect(remainingKeys.includes('pronotes-v0')).toBe(false);
    expect(remainingKeys.includes('legacy-cache')).toBe(false);
  });

  it('fetch strategy: ignores non-GET requests (POST, PUT, DELETE)', async () => {
    const env = createSwEnvironment();
    let networkFetchCalled = false;
    env.runInSw(async () => {
      networkFetchCalled = true;
      return new MockResponse('ok');
    });

    const fetchListeners = env.listeners.get('fetch') || [];
    const postReq = new MockRequest('http://localhost:3000/api/save', { method: 'POST' });
    const fetchEvent = new MockFetchEvent(postReq);

    fetchListeners[0](fetchEvent);
    expect(fetchEvent.responded).toBe(false);
    expect(networkFetchCalled).toBe(false);
  });

  it('fetch strategy: ignores /api/* requests and non-http protocols', async () => {
    const env = createSwEnvironment();
    env.runInSw(async () => new MockResponse('ok'));
    const fetchListeners = env.listeners.get('fetch') || [];

    const apiReq = new MockRequest('http://localhost:3000/api/auth/session', { method: 'GET' });
    const apiEvent = new MockFetchEvent(apiReq);
    fetchListeners[0](apiEvent);
    expect(apiEvent.responded).toBe(false);

    const chromeExtReq = new MockRequest('chrome-extension://abcdefg/inject.js', { method: 'GET' });
    const extEvent = new MockFetchEvent(chromeExtReq);
    fetchListeners[0](extEvent);
    expect(extEvent.responded).toBe(false);
  });

  it('fetch strategy (Navigation): Network-First succeeds and updates cache on 200', async () => {
    const env = createSwEnvironment();
    let networkCalled = false;
    env.runInSw(async (req) => {
      networkCalled = true;
      return new MockResponse('<!DOCTYPE html><html>Fresh Network HTML</html>', { status: 200 });
    });

    const fetchListeners = env.listeners.get('fetch') || [];
    const navReq = new MockRequest('http://localhost:3000/whiteboard/123', {
      method: 'GET',
      mode: 'navigate'
    });
    const fetchEvent = new MockFetchEvent(navReq);
    fetchListeners[0](fetchEvent);

    expect(fetchEvent.responded).toBe(true);
    const res = await fetchEvent.getResponse();
    expect(networkCalled).toBe(true);
    expect(await res.text()).toContain('Fresh Network HTML');

    // Verify it updated the cache
    const cache = await env.caches.open(SW_CACHE);
    const cachedNav = await cache.match(navReq);
    expect(cachedNav).toBeDefined();
    expect(await cachedNav.text()).toContain('Fresh Network HTML');
  });

  it('fetch strategy (Navigation): Network-First falls back to cached "/" when offline', async () => {
    const env = createSwEnvironment();
    // Pre-cache root '/'
    const cache = await env.caches.open(SW_CACHE);
    await cache.put('/', new MockResponse('<!DOCTYPE html><html>Cached Offline App Shell</html>', { status: 200 }));

    // Simulate network error (offline)
  });

```

```

    env.runInSw(async () => {
      throw new Error('Failed to fetch (offline)');
    });

    const fetchListeners = env.listeners.get('fetch') || [];
    const navReq = new MockRequest('http://localhost:3000/some/deep/route', {
      method: 'GET',
      mode: 'navigate'
    });
    const fetchEvent = new MockFetchEvent(navReq);
    fetchListeners[0](fetchEvent);

    expect(fetchEvent.responded).toBe(true);
    const res = await fetchEvent.getResponse();
    expect(res).toBeDefined();
    expect(await res.text()).toContain('Cached Offline App Shell');
  });

  it('fetch strategy (Static Assets): Cache-First returns cached item immediately and triggers SWR revalidatio
    n', async () => {
    const env = createSwEnvironment();
    const cache = await env.caches.open(SW_CACHE);
    const assetUrl = 'http://localhost:3000/static/bundle.js';
    await cache.put(assetUrl, new MockResponse('console.log("Cached v1");', { status: 200 }));

    let backgroundFetchTriggered = false;
    env.runInSw(async (req) => {
      backgroundFetchTriggered = true;
      return new MockResponse('console.log("Fresh v2");', { status: 200 });
    });

    const fetchListeners = env.listeners.get('fetch') || [];
    const assetReq = new MockRequest(assetUrl, { method: 'GET', mode: 'cors' });
    const fetchEvent = new MockFetchEvent(assetReq);
    fetchListeners[0](fetchEvent);

    expect(fetchEvent.responded).toBe(true);
    const res = await fetchEvent.getResponse();
    // Instant response is cached v1
    expect(await res.text()).toBe('console.log("Cached v1");');

    // Wait microtask for SWR background promise
    await new Promise((r) => setTimeout(r, 10));
    expect(backgroundFetchTriggered).toBe(true);

    // Cache updated to v2
    const updated = await cache.match(assetUrl);
    expect(await updated.text()).toBe('console.log("Fresh v2");');
  });

  it('fetch strategy (Static Assets): Cache-Miss fetches from network, caches, and returns response', async ()
    => {
    const env = createSwEnvironment();
    const assetUrl = 'http://localhost:3000/logo.png';
    let networkFetched = false;

    env.runInSw(async (req) => {
      networkFetched = true;
      return new MockResponse('PNG_IMAGE_DATA_BYTES', { status: 200 });
    });

    const fetchListeners = env.listeners.get('fetch') || [];
    const assetReq = new MockRequest(assetUrl, { method: 'GET', mode: 'cors' });
    const fetchEvent = new MockFetchEvent(assetReq);
    fetchListeners[0](fetchEvent);

    expect(fetchEvent.responded).toBe(true);
    const res = await fetchEvent.getResponse();
    expect(networkFetched).toBe(true);
    expect(await res.text()).toBe('PNG_IMAGE_DATA_BYTES');

    // Cache now holds the newly fetched item
    const cache = await env.caches.open(SW_CACHE);
    const cachedItem = await cache.match(assetUrl);
    expect(cachedItem).toBeDefined();
    expect(await cachedItem.text()).toBe('PNG_IMAGE_DATA_BYTES');
  });
});

```

```

describe('M3.2 — PWA Client Lifecycle & Component (PWARegister.tsx)', () => {
  // These tests model the ONE environment the service worker is meant for: a
  // production web build served over http(s). PWARegister now refuses to
  // register anywhere else (dev server, the APK, Electron's pronotes://), so
  // the mock DOM has to say which environment it is - it used to have no
  // `location` at all, and no NODE_ENV.
  let savedNodeEnv;
  beforeEach(() => {
    clearModuleCache();
    setupMockDom();
    Object.defineProperty(global, 'navigator', {
      value: global.window.navigator,
      configurable: true,
      writable: true
    });
    global.window.location = { protocol: 'https:', pathname: '/', search: '', hash: '' };
    savedNodeEnv = process.env.NODE_ENV;
    process.env.NODE_ENV = 'production';
  });
  afterEach(() => {
    process.env.NODE_ENV = savedNodeEnv;
  });

  it('PWARegister source exists and compiles without TypeScript errors', () => {
    const compPath = path.join(ROOT_DIR, 'src/components/pwa/PWARegister.tsx');
    expect(fs.existsSync(compPath)).toBe(true);
    const mod = loadTsModule('src/components/pwa/PWARegister.tsx');
    expect(typeof mod.PWARegister).toBe('function');
  });

  it('Registers /sw.js on window load and sets up update lifecycle handler', async () => {
    const eventMap = {};
    let registeredSwPath = null;

    global.window.addEventListener = (event, handler) => {
      eventMap[event] = handler;
    };
    global.window.removeEventListener = (event, handler) => {
      delete eventMap[event];
    };

    let onUpdateFoundHandler = null;
    let stateChangeHandler = null;
    const mockInstallingWorker = {
      state: 'installing',
      set onstatechange(fn) {
        stateChangeHandler = fn;
      },
      get onstatechange() {
        return stateChangeHandler;
      }
    };

    const mockRegistration = {
      scope: '/',
      get installing() {
        return mockInstallingWorker;
      },
      set onupdatefound(fn) {
        onUpdateFoundHandler = fn;
      },
      get onupdatefound() {
        return onUpdateFoundHandler;
      }
    };

    const mockNavigator = {
      serviceWorker: {
        register: async (path) => {
          registeredSwPath = path;
          return mockRegistration;
        },
        controller: {}
      }
    };
    global.window.navigator = mockNavigator;
    Object.defineProperty(global, 'navigator', {

```

```
    value: mockNavigator,
    configurable: true,
    writable: true
  });

const mod = loadTsModule('src/components/pwa/PWARegister.tsx');
mountComponent(mod.PWARegister);

// Verify window.load was subscribed
expect(typeof eventMap['load']).toBe('function');

// Trigger load event
eventMap['load']();
await new Promise((r) => setTimeout(r, 20));

expect(registeredSwPath).toBe('/sw.js');

// Test onupdatefound handler
expect(typeof mockRegistration.onupdatefound).toBe('function');
mockRegistration.onupdatefound();
expect(typeof mockInstallingWorker.onstatechange).toBe('function');

// Fire state change to installed
mockInstallingWorker.state = 'installed';
mockInstallingWorker.onstatechange();
});

it('Captures beforeinstallprompt and dispatches "pronotes:can-install"', () => {
  const eventMap = {};
  const dispatchedEvents = [];

  global.window.addEventListener = (event, handler) => {
    eventMap[event] = handler;
  };
  global.window.removeEventListener = (event, handler) => {
    delete eventMap[event];
  };
  global.window.dispatchEvent = (event) => {
    dispatchedEvents.push(event);
  };

  const mod = loadTsModule('src/components/pwa/PWARegister.tsx');
  mountComponent(mod.PWARegister);

  const fakePromptEvent = {
    platforms: ['web', 'windows'],
    prompt: async () => {},
    userChoice: Promise.resolve({ outcome: 'accepted', platform: 'windows' })
  };

  expect(typeof eventMap['beforeinstallprompt']).toBe('function');
  eventMap['beforeinstallprompt'](fakePromptEvent);

  expect(global.window.__PWA_INSTALL_PROMPT__).toBe(fakePromptEvent);
  expect(dispatchedEvents.some((e) => e.type === 'pronotes:can-install')).toBe(true);
});

it('Handles appinstalled, clears prompt state, and dispatches "pronotes:installed"', () => {
  const eventMap = {};
  const dispatchedEvents = [];

  global.window.addEventListener = (event, handler) => {
    eventMap[event] = handler;
  };
  global.window.removeEventListener = (event, handler) => {
    delete eventMap[event];
  };
  global.window.dispatchEvent = (event) => {
    dispatchedEvents.push(event);
  };

  const mod = loadTsModule('src/components/pwa/PWARegister.tsx');
  mountComponent(mod.PWARegister);
  global.window.__PWA_INSTALL_PROMPT__ = { fake: true };

  expect(typeof eventMap['appinstalled']).toBe('function');
  eventMap['appinstalled']();
});
```

```

    expect(global.window.__PWA_INSTALL_PROMPT__).toBeNull();
    expect(dispatchedEvents.some((e) => e.type === 'pronotes:installed')).toBe(true);
  });

  it('Cleans up event listeners on unmount', () => {
    const eventMap = {};

    global.window.addEventListener = (event, handler) => {
      eventMap[event] = handler;
    };
    global.window.removeEventListener = (event, handler) => {
      delete eventMap[event];
    };

    const mod = loadTsModule('src/components/pwa/PWARegister.tsx');
    const instance = mountComponent(mod.PWARegister);

    expect(eventMap['beforeinstallprompt']).toBeDefined();
    expect(eventMap['appinstalled']).toBeDefined();

    instance.unmount();

    expect(eventMap['beforeinstallprompt']).toBeUndefined();
    expect(eventMap['appinstalled']).toBeUndefined();
  });

  // --- where the service worker must NOT run -----
  // A cache-first worker inside the APK could serve an old, unhashed
  // pdf.worker.min.mjs after an update; under `npm run dev` it served stale
  // unhashed dev chunks. In both places it must stay unregistered and clean up
  // any worker an earlier build installed.
  const expectNoRegistration = async (label) => {
    let registered = false;
    let unregistered = 0;
    const eventMap = {};
    global.window.addEventListener = (event, handler) => { eventMap[event] = handler; };
    global.window.removeEventListener = (event) => { delete eventMap[event]; };
    const nav = {
      serviceWorker: {
        register: async () => { registered = true; return {}; },
        getRegistrations: async () => [{ unregister: async () => { unregistered++; return true; } }],
        controller: null,
      },
    };
    global.window.navigator = nav;
    Object.defineProperty(global, 'navigator', { value: nav, configurable: true, writable: true });

    const mod = loadTsModule('src/components/pwa/PWARegister.tsx');
    const instance = mountComponent(mod.PWARegister);
    if (eventMap['load']) eventMap['load']();
    await new Promise((r) => setTimeout(r, 20));
    expect(registered).toBe(false);
    expect(unregistered).toBe(1);
    instance.unmount();
  };

  it('does NOT register under the dev server, and removes an old worker', async () => {
    process.env.NODE_ENV = 'development';
    await expectNoRegistration('dev');
  });

  it('does NOT register outside http(s) (Electron pronotes://), and removes an old worker', async () => {
    global.window.location = { protocol: 'pronotes:', pathname: '/index.html', search: '', hash: '' };
    await expectNoRegistration('electron');
  });
});

describe('M3.3 — Next.js Layout Metadata & Viewport Exports (src/app/layout.tsx)', () => {
  it('src/app/layout.tsx exists and contains valid metadata and viewport exports', () => {
    const layoutPath = path.join(ROOT_DIR, 'src/app/layout.tsx');
    expect(fs.existsSync(layoutPath)).toBe(true);

    const layoutContent = fs.readFileSync(layoutPath, 'utf-8');
    expect(layoutContent).toContain('export const metadata');
    expect(layoutContent).toContain('export const viewport');
    expect(layoutContent).toContain('<PWARegister');
  });
});

```

```
it('Metadata contains accurate ProNotes title, manifest, and icons', () => {
  const layoutContent = fs.readFileSync(path.join(ROOT_DIR, 'src/app/layout.tsx'), 'utf-8');

  expect(layoutContent).toContain('title: "ProNotes – Infinite Whiteboard & Knowledge Canvas"');
  expect(layoutContent).toContain('manifest: "/manifest.json"');
  expect(layoutContent).toContain('applicationName: "ProNotes"');
  expect(layoutContent).toContain('capable: true');
  expect(layoutContent).toContain('statusBarStyle: "black-translucent"');
  expect(layoutContent).toContain('url: "/favicon.ico"');
  expect(layoutContent).toContain('url: "/favicon.png"');
  expect(layoutContent).toContain('url: "/icon-192.png"');
  expect(layoutContent).toContain('url: "/icon-512.png"');
  expect(layoutContent).toContain('url: "/apple-touch-icon.png"');
});

it('Viewport exports Next.js 15 themeColor and scale constraints', () => {
  const layoutContent = fs.readFileSync(path.join(ROOT_DIR, 'src/app/layout.tsx'), 'utf-8');

  expect(layoutContent).toContain('themeColor: "#4f46e5"');
  expect(layoutContent).toContain('width: "device-width"');
  expect(layoutContent).toContain('initialScale: 1');
  expect(layoutContent).toContain('maximumScale: 1');
  expect(layoutContent).toContain('userScalable: false');
});
});

describe('M3.4 – PWA Web App Manifest Compliance (public/manifest.json)', () => {
  const manifestPath = path.join(PUBLIC_DIR, 'manifest.json');

  it('manifest.json exists and is valid JSON', () => {
    expect(fs.existsSync(manifestPath)).toBe(true);
    const content = fs.readFileSync(manifestPath, 'utf-8');
    const parsed = JSON.parse(content);
    expect(typeof parsed).toBe('object');
  });

  it('Conforms to PWA installation standards (name, display, colors, start_url)', () => {
    const manifest = JSON.parse(fs.readFileSync(manifestPath, 'utf-8'));

    expect(manifest.name).toBe('ProNotes – Infinite Whiteboard & Knowledge Canvas');
    expect(manifest.short_name).toBe('ProNotes');
    expect(manifest.start_url).toBe('/');
    expect(manifest.scope).toBe('/');
    expect(manifest.display).toBe('standalone');
    expect(manifest.theme_color.toLowerCase()).toBe('#4f46e5');
    expect(manifest.background_color.toLowerCase()).toBe('#0f172a');
    expect(Array.isArray(manifest.display_override)).toBe(true);
    expect(manifest.display_override).toContain('window-controls-overlay');
  });

  it('Contains all required icons including standard and maskable variants', () => {
    const manifest = JSON.parse(fs.readFileSync(manifestPath, 'utf-8'));
    expect(Array.isArray(manifest.icons)).toBe(true);
    expect(manifest.icons.length).toBeGreaterThanOrEqual(4);

    const hasStandard192 = manifest.icons.some(
      (i) => i.sizes === '192x192' && (i.purpose === 'any' || !i.purpose) && i.src === '/icon-192.png'
    );
    const hasStandard512 = manifest.icons.some(
      (i) => i.sizes === '512x512' && (i.purpose === 'any' || !i.purpose) && i.src === '/icon-512.png'
    );
    const hasMaskable192 = manifest.icons.some(
      (i) => i.sizes === '192x192' && i.purpose === 'maskable' && i.src === '/icon-192-maskable.png'
    );
    const hasMaskable512 = manifest.icons.some(
      (i) => i.sizes === '512x512' && i.purpose === 'maskable' && i.src === '/icon-512-maskable.png'
    );

    expect(hasStandard192).toBe(true);
    expect(hasStandard512).toBe(true);
    expect(hasMaskable192).toBe(true);
    expect(hasMaskable512).toBe(true);

    // Verify all icon src paths resolve to actual files
    for (const icon of manifest.icons) {
      const relPath = icon.src.startsWith('/') ? icon.src.slice(1) : icon.src;
      expect(fs.existsSync(path.join(PUBLIC_DIR, relPath))).toBe(true);
    }
  });
});
```

```

});

it('Defines shortcuts for New Whiteboard and Knowledge Graph with valid targets', () => {
  const manifest = JSON.parse(fs.readFileSync(manifestPath, 'utf-8'));
  expect(Array.isArray(manifest.shortcuts)).toBe(true);
  expect(manifest.shortcuts.length).toBeGreaterThanOrEqual(2);

  const newBoard = manifest.shortcuts.find((s) => s.url === '/?action=new');
  expect(newBoard).toBeDefined();
  expect(newBoard.name).toBe('New Whiteboard');

  const graph = manifest.shortcuts.find((s) => s.url === '/?view=graph');
  expect(graph).toBeDefined();
  expect(graph.name).toBe('Knowledge Graph');
});
});

describe('M3.5 – Physical Brand Assets & Image Verification (Sharp & Binary)', () => {
  it('Validates binary structure of favicon.ico (32x32 and 16x16 formats embedded)', () => {
    const icoPath = path.join(PUBLIC_DIR, 'favicon.ico');
    expect(fs.existsSync(icoPath)).toBe(true);
    const buf = fs.readFileSync(icoPath);

    expect(buf.length).toBeGreaterThan(100);
    // Reserved 0x0000
    expect(buf.readUInt16LE(0)).toBe(0);
    // Type 1 (ICO)
    expect(buf.readUInt16LE(2)).toBe(1);
    // Count of images = 2
    const count = buf.readUInt16LE(4);
    expect(count).toBe(2);

    // Entry 1: 32x32
    expect(buf.readUInt8(6)).toBe(32);
    expect(buf.readUInt8(7)).toBe(32);

    // Entry 2: 16x16
    expect(buf.readUInt8(22)).toBe(16);
    expect(buf.readUInt8(23)).toBe(16);
  });

  it('Validates dimensions and RGBA color format of all raster icon assets', async () => {
    const expectedAssets = [
      { file: 'favicon.png', width: 32, height: 32 },
      { file: 'favicon-32x32.png', width: 32, height: 32 },
      { file: 'favicon-16x16.png', width: 16, height: 16 },
      { file: 'apple-touch-icon.png', width: 180, height: 180 },
      { file: 'icon-192.png', width: 192, height: 192 },
      { file: 'icon-512.png', width: 512, height: 512 },
      { file: 'icon-192-maskable.png', width: 192, height: 192 },
      { file: 'icon-512-maskable.png', width: 512, height: 512 },
      { file: 'logo.png', width: 512, height: 512 },
    ];

    for (const asset of expectedAssets) {
      const filePath = path.join(PUBLIC_DIR, asset.file);
      expect(fs.existsSync(filePath)).toBe(true);

      const metadata = await sharp(filePath).metadata();
      expect(metadata.format).toBe('png');
      expect(metadata.width).toBe(asset.width);
      expect(metadata.height).toBe(asset.height);
      expect(metadata.channels).toBeGreaterThanOrEqual(3);
    }
  });

  it('Verifies safe zone padding on maskable icons (padded with background #0f172a)', async () => {
    const maskable512 = path.join(PUBLIC_DIR, 'icon-512-maskable.png');
    const { data, info } = await sharp(maskable512)
      .raw()
      .toBuffer({ resolveWithObject: true });

    // Check top-left corner pixel (0, 0) is background slate #0f172a (r=15, g=23, b=42)
    const r = data[0];
    const g = data[1];
    const b = data[2];

    expect(r).toBe(15);
  });

```

```
    expect(g).toBe(23);
    expect(b).toBe(42);
  });
});

describe('M3.6 – Global ProNotes Branding Consistency', () => {
  it('Sidebar header includes ProNotes logo and brand title', () => {
    const sidebarContent = fs.readFileSync(path.join(ROOT_DIR, 'src/components/whiteboard/Sidebar.tsx'), 'utf-8');
    expect(sidebarContent).toContain('src="/favicon.png"');
    expect(sidebarContent).toContain('alt="ProNotes Logo"');
    expect(sidebarContent).toContain('>ProNotes</h2>');
  });

  it('Tutorial modal welcomes user to ProNotes', () => {
    const tutContent = fs.readFileSync(path.join(ROOT_DIR, 'src/components/whiteboard/TutorialModal.tsx'), 'utf-8');
    expect(tutContent).toContain('Welcome to ProNotes');
    expect(tutContent).toContain('ProNotes is an infinite canvas');
  });

  it('package.json sets package name to "pronotes"', () => {
    const pkg = JSON.parse(fs.readFileSync(path.join(ROOT_DIR, 'package.json'), 'utf-8'));
    expect(pkg.name).toBe('pronotes');
  });

  it('README.md document has ProNotes as headline title', () => {
    const readmeContent = fs.readFileSync(path.join(ROOT_DIR, 'README.md'), 'utf-8');
    // Split on \r?\n: a Windows checkout (core.autocrlf=true) ends lines with
    // CRLF, which left a trailing \r on the title and failed in a fresh worktree.
    expect(readmeContent.split(/\r?\n/)[0]).toBe('# ProNotes');
  });
});

describe('M3.7 – Stress Tests, Fault Injection & Edge Cases', () => {
  it('SW Navigation Fault: Handles offline deep routes with query parameters gracefully', async () => {
    const env = createSwEnvironment();
    const cache = await env.caches.open(SW_CACHE);
    await cache.put('/', new MockResponse('<!DOCTYPE html><html>Offline Fallback Root</html>', { status: 200 }));

    env.runInSw(async () => {
      throw new Error('Network Connection Refused');
    });

    const fetchListeners = env.listeners.get('fetch') || [];
    const queryNavReq = new MockRequest('http://localhost:3000/board/matrix-123?action=new&theme=dark', {
      method: 'GET',
      mode: 'navigate'
    });

    const fetchEvent = new MockFetchEvent(queryNavReq);
    fetchListeners[0](fetchEvent);

    expect(fetchEvent.responded).toBe(true);
    const res = await fetchEvent.getResponse();
    expect(res).toBeDefined();
    expect(await res.text()).toBe('<!DOCTYPE html><html>Offline Fallback Root</html>');
  });

  it('SW Resilience: HTTP 500 server errors are NOT cached into STATIC cache', async () => {
    const env = createSwEnvironment();
    env.runInSw(async (req) => {
      return new MockResponse('Internal Server Error', { status: 500 });
    });

    const fetchListeners = env.listeners.get('fetch') || [];
    const assetReq = new MockRequest('http://localhost:3000/dynamic-chunk.js', { method: 'GET' });
    const fetchEvent = new MockFetchEvent(assetReq);
    fetchListeners[0](fetchEvent);

    expect(fetchEvent.responded).toBe(true);
    const res = await fetchEvent.getResponse();
    expect(res.status).toBe(500);

    const cache = await env.caches.open(SW_CACHE);
    const cachedItem = await cache.match(assetReq);
    expect(cachedItem).toBeUndefined();
  });
});
```

```

});

it('SW Resilience: SWR background network failure does not disrupt cached delivery', async () => {
  const env = createSwEnvironment();
  const cache = await env.caches.open(SW_CACHE);
  const assetUrl = 'http://localhost:3000/app.css';
  await cache.put(assetUrl, new MockResponse('body { color: #fff; }', { status: 200 }));

  // Simulate network error during background revalidate
  env.runInSw(async () => {
    throw new Error('Network Timeout during SWR');
  });

  const fetchListeners = env.listeners.get('fetch') || [];
  const req = new MockRequest(assetUrl, { method: 'GET' });
  const fetchEvent = new MockFetchEvent(req);
  fetchListeners[0](fetchEvent);

  expect(fetchEvent.responded).toBe(true);
  const res = await fetchEvent.getResponse();
  expect(await res.text()).toBe('body { color: #fff; }');

  // Wait microtask
  await new Promise((r) => setTimeout(r, 15));
  // Cache remains intact
  const intact = await cache.match(assetUrl);
  expect(await intact.text()).toBe('body { color: #fff; }');
});

it('Icon Pipeline Idempotence: Running generate-icons.js regenerates identical valid assets', () => {
  const scriptPath = path.join(ROOT_DIR, 'scripts/generate-icons.js');
  expect(fs.existsSync(scriptPath)).toBe(true);

  const out = execSync(`node "${scriptPath}"`, {
    cwd: ROOT_DIR,
    env: { ...process.env, PATH: `C:\\Program Files\\nodejs;${process.env.PATH}` }
  }).toString();

  expect(out).toContain('All brand assets successfully generated!');
});

// =====
// RUNNER
// =====

async function runTests() {
  console.log('\n=====');
  console.log(' M3 EMPIRICAL CHALLENGER TEST SUITE: Branding & PWA Integration');
  console.log('=====\\n');

  let passedSuites = 0;
  let failedSuites = 0;
  let totalTests = 0;
  let passedTests = 0;
  let failedTests = 0;

  for (const suite of ctx.suites) {
    console.log(`\\n• [SUITE] ${suite.name}`);
    let suiteFailed = false;

    for (const test of suite.tests) {
      totalTests++;
      ctx.currentTest = test;

      // Run beforeEach hooks
      for (const beforeFn of suite.beforeEachFns) {
        await beforeFn();
      }

      const start = Date.now();
      try {
        await test.fn();
        test.passed = true;
        test.durationMs = Date.now() - start;
        passedTests++;
        console.log(` ✓ ${test.name} (${test.durationMs}ms)`);
      } catch (err) {

```

```

    test.passed = false;
    test.error = err;
    test.durationMs = Date.now() - start;
    failedTests++;
    suiteFailed = true;
    console.error(`  X ${test.name} (${test.durationMs}ms)`);
    console.error(`    Error: ${err.message}`);
  }

  // Run afterEach hooks
  for (const afterFn of suite.afterEachFns) {
    await afterFn();
  }
}

if (suiteFailed) {
  failedSuites++;
} else {
  passedSuites++;
}
}

console.log(`\n=====`);
console.log('  TEST EXECUTION SUMMARY');
console.log('=====');
console.log(` Suites:      ${passedSuites} passed, ${failedSuites} failed, ${ctx.suites.length} total`);
console.log(` Tests:       ${passedTests} passed, ${failedTests} failed, ${totalTests} total`);
console.log(` Assertions: ${ctx.passedAssertions} passed, ${ctx.failedAssertions} failed, ${ctx.totalAssert
ions} total`);
console.log('=====\\n');

if (failedTests > 0) {
  process.exit(1);
}
}

runTests().catch((err) => {
  console.error('Fatal Runner Error:', err);
  process.exit(1);
});

=====

// === END FILE: tests/e2e/m3-pwa-branding.test.js ===
=====

=====

// === START FILE: tests/e2e/mutation-challenge.js ===
// File 145 of 194 | Lines: 183 | Size: 6994 bytes
=====

const fs = require('fs');
const path = require('path');
const { execSync } = require('child_process');

const ROOT = path.resolve(__dirname, '../..');

const mutations = [
  {
    id: 'MUT_01_STROKE_DOT',
    desc: 'Single-tap dot returns empty string instead of circular arc',
    file: 'src/lib/whiteboard/stroke.ts',
    find: 'if (cleanPoints.length === 1) {',
    replace: 'if (cleanPoints.length === 1) { return "";'
  },
  {
    id: 'MUT_02_HIGHLIGHTER_SCALE',
    desc: 'Highlighter size multiplier changed from 3x to 1x',
    file: 'src/lib/whiteboard/stroke.ts',
    find: 'const sizeMultiplier = style === "highlighter" ? 3 : 1;',
    replace: 'const sizeMultiplier = style === "highlighter" ? 1 : 1;'
  },
  {
    id: 'MUT_03_COORDINATE_SCREEN_TO_WORLD',
    desc: 'screenToWorld multiplies by zoom instead of dividing',
    file: 'src/lib/whiteboard/geometry.ts',
    find: '(sx - viewport.width / 2) / camera.zoom + camera.x',
    replace: '(sx - viewport.width / 2) * camera.zoom + camera.x'
  }
]

```

```
},
{
  id: 'MUT_04_RECT_HIT_TEST',
  desc: 'Rectangle hit testing always returns false',
  file: 'src/lib/whiteboard/geometry.ts',
  find: 'case "rectangle":\n    case "text":\n    case "sticky":\n    case "image":\n    return (',
  replace: 'case "rectangle":\n    case "text":\n    case "sticky":\n    case "image":\n    return false &
    & ('
},
{
  id: 'MUT_05_SMART_CONNECTOR_ANCHORS',
  desc: 'getOptimalConnectionAnchors returns inverted fixed anchors',
  file: 'src/lib/whiteboard/geometry.ts',
  find: 'return { source: sAnchor, target: tAnchor };',
  replace: 'return { source: { ...sAnchor, anchor: "top" }, target: { ...tAnchor, anchor: "bottom" } };',
},
{
  id: 'MUT_06_WIKILINK_PARSER',
  desc: 'WIKILINK_REGEX regex corrupted to curly braces {{...}}',
  file: 'src/lib/whiteboard/links.ts',
  find: 'const WIKILINK_REGEX = /\[\[([^\]]+)\](?:\[\[([^\]]+)\])?\]\]/g;',
  replace: 'const WIKILINK_REGEX = /\[\[([^\]]+)\]\]\]/g;',
},
{
  id: 'MUT_07_STORE_UNDO',
  desc: 'Store undo made into a no-op',
  file: 'src/store/whiteboard.ts',
  find: 'undo: () => {\n    const state = get();',
  replace: 'undo: () => {\n    return;\n    const state = get();',
},
{
  id: 'MUT_08_ALIGN_TOP',
  desc: 'alignElements top alignment corrupted with 100px shift',
  file: 'src/lib/whiteboard/geometry.ts',
  find: 'case "top":\n    dy = union.y - b.y;\n    break;',
  replace: 'case "top":\n    dy = union.y - b.y + 100;\n    break;',
},
{
  id: 'MUT_09_CLAMP_MATH',
  desc: 'clamp function returns raw unconstrained input',
  file: 'src/lib/whiteboard/geometry.ts',
  find: 'export function clamp(v: number, min: number, max: number): number {\n    return Math.max(min, Math.m
    in(max, v));\n}',
  replace: 'export function clamp(v: number, min: number, max: number): number {\n    return v;\n}',
},
{
  id: 'MUT_10_MANIFEST_BRANDING',
  desc: 'PWA manifest name corrupted from ProNotes to BogusNotes',
  file: 'public/manifest.json',
  find: '"name": "ProNotes - Infinite Whiteboard & Knowledge Canvas"',
  replace: '"name": "BogusNotes - Canvas"',
},
{
  id: 'MUT_11_TOUCH_SETTINGS',
  desc: 'DEFAULT_TOUCH_SETTINGS twoFingerUndo set to false',
  file: 'src/lib/whiteboard/types.ts',
  find: 'twoFingerUndo: true,',
  replace: 'twoFingerUndo: false,',
},
{
  id: 'MUT_12_ZOOM_MIN',
  desc: 'MIN_ZOOM changed from 0.05 to 0.5',
  file: 'src/lib/whiteboard/types.ts',
  find: 'export const MIN_ZOOM = 0.05;',
  replace: 'export const MIN_ZOOM = 0.5;',
},
{
  id: 'MUT_13_SW_FETCH_GET_ONLY',
  desc: 'Service Worker bypasses GET filtering allowing POST caching',
  file: 'public/sw.js',
  find: 'if (request.method !== 'GET') {',
  replace: 'if (request.method !== 'POST') {'
},
{
  id: 'MUT_14_PERSISTENCE_MIGRATION',
  desc: 'Persistence loadState migrate assigns wrong fallback color',
  file: 'src/lib/whiteboard/persistence.ts',
  find: 'color: typeof b.color === "string" ? b.color : "#3b82f6",',
```

```

    replace: 'color: typeof b.color === "string" ? b.color : "#ff0000",'
  },
  {
    id: 'MUT_15_BOUNDS_ELLIPSE',
    desc: 'getElementBounds on ellipse returns width 0',
    file: 'src/lib/whiteboard/geometry.ts',
    find: 'case "rectangle":\n    case "ellipse":',
    replace: 'case "rectangle":\n    return { x: el.x, y: el.y, width: el.width, height: el.height }; \n    c
      case "ellipse":\n    return { x: el.x, y: el.y, width: 0, height: el.height };'
  }
];

const results = [];

console.log('='.repeat(80));
console.log(' ■ PRONOTES ADVERSARIAL MUTATION TESTING SUITE (15 MUTATIONS)');
console.log('='.repeat(80));

for (const m of mutations) {
  const filePath = path.join(ROOT, m.file);
  const original = fs.readFileSync(filePath, 'utf8');
  if (!original.includes(m.find)) {
    console.error('■ [SETUP ERROR] Target string not found for ${m.id} in ${m.file}');
    results.push({ id: m.id, desc: m.desc, caught: false, error: 'Target not found in file' });
    continue;
  }
  const mutated = original.replace(m.find, m.replace);
  fs.writeFileSync(filePath, mutated, 'utf8');

  let caught = false;
  let output = '';
  try {
    output = execSync('node tests/e2e/runner.js', {
      cwd: ROOT,
      env: { ...process.env, PATH: 'C:\\Program Files\\nodejs;' + process.env.PATH },
      encoding: 'utf8',
      stdio: 'pipe'
    });
    caught = false; // If it exited 0, test suite missed the bug!
  } catch (err) {
    caught = true; // Exited non-zero -> bug was caught!
    output = (err.stdout || '') + '\n' + (err.stderr || '');
  } finally {
    fs.writeFileSync(filePath, original, 'utf8'); // ALWAYS RESTORE ORIGINAL
  }

  // Count failed tests in output
  const failMatches = output.match(/X\s+([^\n]+)/g) || [];
  results.push({
    id: m.id,
    desc: m.desc,
    file: m.file,
    caught,
    failedTestCount: failMatches.length,
    sampleFailures: failMatches.slice(0, 3).map(s => s.trim())
  });

  const status = caught ? '■ CAUGHT' : '■ MISSED';
  console.log(`\n${status} [${m.id}] ${m.desc}`);
  console.log(`  Target File: ${m.file}`);
  console.log(`  Failed Test Assertions Triggered: ${failMatches.length}`);
  if (failMatches.length > 0) {
    console.log(`  Sample Caught Tests:`);
    failMatches.slice(0, 3).forEach(f => console.log(`    - ${f.trim()}`));
  }
}

console.log('\n' + '='.repeat(80));
console.log(' ■ MUTATION SCORE SUMMARY');
console.log('='.repeat(80));
const caughtCount = results.filter(r => r.caught).length;
const totalCount = results.length;
const score = ((caughtCount / totalCount) * 100).toFixed(1);

console.log(`  Total Injected Mutations : ${totalCount}`);
console.log(`  Mutations Caught          : ${caughtCount}`);
console.log(`  Mutations Missed          : ${totalCount - caughtCount}`);
console.log(`  Mutation Score             : ${score}%`);

```

```
console.log('='.repeat(80));

process.exit(caughtCount === totalCount ? 0 : 1);

=====
// === END FILE: tests/e2e/mutation-challenge.js ===
=====

=====
// === START FILE: tests/e2e/runner.js ===
// File 146 of 194 | Lines: 183 | Size: 6969 bytes
=====

#!/usr/bin/env node

/**
 * ProNotes E2E Test Suite Runner
 * =====
 * Master test runner executing Tiers 1-4 across all 22 features, boundary cases,
 * pairwise combinations, and real-world application scenarios.
 *
 * Exit Codes:
 * 0: All tests and assertions passed successfully.
 * 1: One or more tests or assertions failed.
 */

const path = require('path');
const { ctx, clearModuleCache, getAllBeforeEachFns, getAllAfterEachFns } = require('./test-helpers');

// Color formatting for console output
const colors = {
  reset: '\x1b[0m',
  bold: '\x1b[1m',
  dim: '\x1b[2m',
  green: '\x1b[32m',
  red: '\x1b[31m',
  yellow: '\x1b[33m',
  blue: '\x1b[34m',
  cyan: '\x1b[36m',
  magenta: '\x1b[35m',
  gray: '\x1b[90m',
  bgGreen: '\x1b[42m\x1b[30m',
  bgRed: '\x1b[41m\x1b[37m'
};

const TIERS = [
  { file: 'tier1-features.test.js', name: 'Tier 1: Feature Coverage (22 Features)', minAssertions: 110 },
  { file: 'tier2-boundaries.test.js', name: 'Tier 2: Boundary & Corner Cases (22 Features)', minAssertions: 110 },
  { file: 'tier3-pairwise.test.js', name: 'Tier 3: Pairwise Combinations (>=25 Tests)', minTests: 25 },
  { file: 'tier4-scenarios.test.js', name: 'Tier 4: Real-World Scenarios (>=5 Workflows)', minTests: 5 },
  // Written for milestone 3 but never added here, so it had quietly rotted
  // (it read the deleted start.bat). Wired in so it stays true.
  { file: 'm3-pwa-branding.test.js', name: 'Milestone 3: PWA, Icons & Branding', minTests: 1 }
];

async function runTestSuite() {
  const startTime = Date.now();
  console.log('\n' + colors.bold + colors.cyan + '='.repeat(80) + colors.reset);
  console.log(colors.bold + colors.cyan + ' ■ ProNotes — Automated End-to-End Test Suite Runner' + colors.reset);
  console.log(colors.cyan + '='.repeat(80) + colors.reset + '\n');

  let totalTestsPassed = 0;
  let totalTestsFailed = 0;
  let totalAssertionsCount = 0;
  const tierResults = [];

  for (let tIdx = 0; tIdx < TIERS.length; tIdx++) {
    const tier = TIERS[tIdx];
    const tierStart = Date.now();
    console.log(colors.bold + colors.blue + `[Tier ${tIdx + 1}/${TIERS.length}] Loading ${tier.name}...` + colors.reset);

    // Reset test context for clean isolation
    ctx.reset();
    clearModuleCache();
  }
}
```

```

try {
  // Require and evaluate the test file
  const fullPath = path.resolve(__dirname, tier.file);
  delete require.cache[require.resolve(fullPath)];
  require(fullPath);
} catch (loadErr) {
  console.error(colors.red + `[ERROR] Failed to load test suite ${tier.file}: ${loadErr.message}` + colors
    .reset);
  console.error(loadErr.stack);
  process.exit(1);
}

let tierPassedTests = 0;
let tierFailedTests = 0;
let tierAssertions = 0;

for (const suite of ctx.suites) {
  if (suite.tests.length === 0) continue;
  console.log(`\n  ${colors.bold}${suite.name}${colors.reset}`);

  for (const testItem of suite.tests) {
    ctx.currentTest = testItem;
    const testStart = Date.now();

    try {
      // Execute all beforeEach hooks (ancestors -> current)
      const beforeFns = getAllBeforeEachFns(testItem.suite);
      for (const fn of beforeFns) {
        await fn();
      }

      // Execute test body
      await testItem.fn();

      // Execute all afterEach hooks (current -> ancestors)
      const afterFns = getAllAfterEachFns(testItem.suite);
      for (const fn of afterFns) {
        await fn();
      }

      testItem.durationMs = Date.now() - testStart;
      testItem.passed = true;
      tierPassedTests++;
      totalTestsPassed++;
      console.log(`    ${colors.green}✓${colors.reset} ${colors.gray}${testItem.name}${colors.reset} ${col
        ors.dim}(${testItem.durationMs}ms)${colors.reset}`);
    } catch (err) {
      testItem.durationMs = Date.now() - testStart;
      testItem.passed = false;
      testItem.error = err;
      tierFailedTests++;
      totalTestsFailed++;
      console.log(`    ${colors.red}✗ ${testItem.name}${colors.reset} ${colors.dim}(${testItem.durationMs}
        ms)${colors.reset}`);
      console.log(`    ${colors.red}Error: ${err.message}${colors.reset}`);
      if (err.stack) {
        const stackLines = err.stack.split('\n').slice(1, 4).join('\n');
        console.log(`    ${colors.gray}${stackLines}${colors.reset}`);
      }
    }
  }
}

tierAssertions = ctx.totalAssertions;
totalAssertionsCount += tierAssertions;
const tierDuration = Date.now() - tierStart;

tierResults.push({
  tierName: tier.name,
  passed: tierPassedTests,
  failed: tierFailedTests,
  total: tierPassedTests + tierFailedTests,
  assertions: tierAssertions,
  durationMs: tierDuration
});

console.log(`\n  ${colors.dim}--> ${tier.name}: ${tierPassedTests} passed, ${tierFailedTests} failed (${ti

```

```

        erAssertions} assertions, ${tierDuration}ms){colors.reset}\n`);
    }

    const totalDuration = Date.now() - startTime;

    // Print Comprehensive Summary Table
    console.log('\n' + colors.bold + colors.cyan + '='.repeat(80) + colors.reset);
    console.log(colors.bold + ' ■ E2E TEST EXECUTION SUMMARY' + colors.reset);
    console.log(colors.cyan + '='.repeat(80) + colors.reset);

    console.log(
        ` ${'Tier Name'.padEnd(52)} | ${'Tests'.padStart(7)} | ${'Pass'.padStart(5)} | ${'Fail'.padStart(5)} | ${
            'Assertions'.padStart(10)} `
    );
    console.log(' ' + '-'.repeat(88));

    for (const res of tierResults) {
        const statusColor = res.failed === 0 ? colors.green : colors.red;
        console.log(
            ` ${statusColor}${res.tierName.padEnd(52)}${colors.reset} | ` +
            `${String(res.total).padStart(7)} | ` +
            `${colors.green}${String(res.passed).padStart(5)}${colors.reset} | ` +
            `${res.failed > 0 ? colors.red : colors.gray}${String(res.failed).padStart(5)}${colors.reset} | ` +
            `${String(res.assertions).padStart(10)} `
        );
    }

    console.log(' ' + '-'.repeat(88));
    console.log(
        ` ${colors.bold}${'TOTAL'.padEnd(52)}${colors.reset} | ` +
        `${String(totalTestsPassed + totalTestsFailed).padStart(7)} | ` +
        `${colors.green}${String(totalTestsPassed).padStart(5)}${colors.reset} | ` +
        `${totalTestsFailed > 0 ? colors.red : colors.green}${String(totalTestsFailed).padStart(5)}${colors.reset}
        | ` +
        `${String(totalAssertionsCount).padStart(10)} `
    );

    console.log(colors.cyan + '='.repeat(80) + colors.reset);
    console.log(` ■ Total Duration: ${totalDuration}ms`);

    if (totalTestsFailed === 0) {
        console.log('\n' + colors.bgGreen + colors.bold + ' ✓ ALL E2E TESTS PASSED SUCCESSFULLY (Exit Code 0) ' +
            colors.reset + '\n');
        process.exit(0);
    } else {
        console.log('\n' + colors.bgRed + colors.bold + ` ✗ ${totalTestsFailed} TEST(S) FAILED (Exit Code 1) ` +
            colors.reset + '\n');
        process.exit(1);
    }
}

runTestSuite();

=====
// === END FILE: tests/e2e/runner.js ===
=====

=====
// === START FILE: tests/e2e/test-helpers.js ===
// File 147 of 194 | Lines: 459 | Size: 13379 bytes
=====

/**
 * E2E Test Framework & Helpers for ProNotes
 * =====
 * Provides lightweight test harness primitives (describe, it/test, expect, beforeEach, afterEach),
 * transparent TypeScript module loader using ts.transpileModule, DOM/Window/Storage simulation,
 * and vector math assertion utilities.
 */

const fs = require('fs');
const path = require('path');
const ts = require('typescript');

// =====
// Test Runner Harness Primitives
// =====

```

```
class TestContext {
  constructor() {
    this.suites = [];
    this.currentSuite = null;
    this.currentTest = null;
    this.totalAssertions = 0;
    this.passedAssertions = 0;
    this.failedAssertions = 0;
    this.featureCoverage = new Map(); // featureKey -> count
  }

  reset() {
    this.suites = [];
    this.currentSuite = null;
    this.currentTest = null;
    this.totalAssertions = 0;
    this.passedAssertions = 0;
    this.failedAssertions = 0;
    this.featureCoverage.clear();
  }
}

const ctx = new TestContext();

function describe(suiteName, fn) {
  const parentSuite = ctx.currentSuite;
  const suite = {
    name: suiteName,
    parent: parentSuite,
    tests: [],
    beforeEachFns: [],
    afterEachFns: [],
    passed: 0,
    failed: 0,
    durationMs: 0
  };
  ctx.suites.push(suite);
  ctx.currentSuite = suite;
  try {
    fn();
  } finally {
    ctx.currentSuite = parentSuite;
  }
}

function beforeEach(fn) {
  if (ctx.currentSuite) {
    ctx.currentSuite.beforeEachFns.push(fn);
  }
}

function afterEach(fn) {
  if (ctx.currentSuite) {
    ctx.currentSuite.afterEachFns.push(fn);
  }
}

function it(testName, fn, options = {}) {
  if (!ctx.currentSuite) {
    throw new Error(`Test "${testName}" must be inside a describe() block`);
  }
  const test = {
    name: testName,
    suite: ctx.currentSuite,
    fn,
    options,
    assertions: [],
    passed: true,
    error: null,
    durationMs: 0
  };
  ctx.currentSuite.tests.push(test);
}

const test = it;

function getAllBeforeEachFns(suite) {
```

```

const fns = [];
let curr = suite;
while (curr) {
  if (curr.beforeEachFns && curr.beforeEachFns.length > 0) {
    fns.unshift(...curr.beforeEachFns);
  }
  curr = curr.parent;
}
return fns;
}

function getAllAfterEachFns(suite) {
  const fns = [];
  let curr = suite;
  while (curr) {
    if (curr.afterEachFns && curr.afterEachFns.length > 0) {
      fns.push(...curr.afterEachFns);
    }
    curr = curr.parent;
  }
  return fns;
}

function trackFeature(featureKey) {
  const count = ctx.featureCoverage.get(featureKey) || 0;
  ctx.featureCoverage.set(featureKey, count + 1);
}

// =====
// Assertions & Matchers
// =====

class Expectation {
  constructor(actual) {
    this.actual = actual;
    this.isNot = false;
  }

  get not() {
    this.isNot = !this.isNot;
    return this;
  }

  _record(pass, message, expected) {
    ctx.totalAssertions++;
    const finalPass = this.isNot ? !pass : pass;
    if (finalPass) {
      ctx.passedAssertions++;
    } else {
      ctx.failedAssertions++;
      const err = new Error(message || `Assertion failed: expected ${JSON.stringify(this.actual)} ${this.isNot
        ? 'NOT ' : ''}to match ${JSON.stringify(expected)}`);
      if (ctx.currentTest) {
        ctx.currentTest.passed = false;
        ctx.currentTest.error = err;
      }
      throw err;
    }
  }

  toBe(expected) {
    const pass = this.actual === expected || Object.is(this.actual, expected);
    this._record(pass, `Expected ${this.actual} to be ${expected}`, expected);
  }

  toEqual(expected) {
    const pass = deepEqual(this.actual, expected);
    this._record(pass, `Expected deep equality:\nActual:   ${JSON.stringify(this.actual)}\nExpected: ${JSON.st
      ringify(expected)}`, expected);
  }

  toBeTruthy() {
    const pass = Boolean(this.actual);
    this._record(pass, `Expected ${this.actual} to be truthy`);
  }

  toBeFalsy() {
    const pass = !Boolean(this.actual);

```

```
    this._record(pass, `Expected ${this.actual} to be falsy`);
  }

  toBeNull() {
    const pass = this.actual === null;
    this._record(pass, `Expected ${this.actual} to be null`);
  }

  toBeUndefined() {
    const pass = this.actual === undefined;
    this._record(pass, `Expected ${this.actual} to be undefined`);
  }

  toBeDefined() {
    const pass = this.actual !== undefined;
    this._record(pass, `Expected value to be defined`);
  }

  toBeGreaterThan(expected) {
    const pass = typeof this.actual === 'number' && this.actual > expected;
    this._record(pass, `Expected ${this.actual} > ${expected}`, expected);
  }

  toBeGreaterThanOrEqual(expected) {
    const pass = typeof this.actual === 'number' && this.actual >= expected;
    this._record(pass, `Expected ${this.actual} >= ${expected}`, expected);
  }

  toBeLessThan(expected) {
    const pass = typeof this.actual === 'number' && this.actual < expected;
    this._record(pass, `Expected ${this.actual} < ${expected}`, expected);
  }

  toBeLessThanOrEqual(expected) {
    const pass = typeof this.actual === 'number' && this.actual <= expected;
    this._record(pass, `Expected ${this.actual} <= ${expected}`, expected);
  }

  toBeCloseTo(expected, precision = 2) {
    const pass = typeof this.actual === 'number' && Math.abs(this.actual - expected) < Math.pow(10, -precision) / 2;
    this._record(pass, `Expected ${this.actual} to be close to ${expected} (precision ${precision})`, expected);
  }

  toContain(expected) {
    let pass = false;
    if (typeof this.actual === 'string' || Array.isArray(this.actual)) {
      pass = this.actual.includes(expected);
    } else if (this.actual instanceof Set || this.actual instanceof Map) {
      pass = this.actual.has(expected);
    }
    this._record(pass, `Expected ${JSON.stringify(this.actual)} to contain ${JSON.stringify(expected)}`, expected);
  }

  toHaveLength(expected) {
    const len = this.actual ? this.actual.length : undefined;
    const pass = len === expected;
    this._record(pass, `Expected length ${len} to be ${expected}`, expected);
  }

  toMatch(regexOrString) {
    let pass = false;
    if (typeof regexOrString === 'string') {
      pass = typeof this.actual === 'string' && this.actual.includes(regexOrString);
    } else if (regexOrString instanceof RegExp) {
      pass = typeof this.actual === 'string' && regexOrString.test(this.actual);
    }
    this._record(pass, `Expected "${this.actual}" to match ${regexOrString}`, regexOrString);
  }

  toThrow(expectedError) {
    let threw = false;
    let thrownError = null;
    if (typeof this.actual !== 'function') {
      throw new Error('toThrow requires a function');
    }
  }
```

```

    try {
      this.actual();
    } catch (e) {
      threw = true;
      thrownError = e;
    }
    if (!threw) {
      this._record(false, 'Expected function to throw an error, but it did not');
      return;
    }
    if (expectedError) {
      if (typeof expectedError === 'string') {
        const pass = thrownError.message.includes(expectedError);
        this._record(pass, `Expected error message "${thrownError.message}" to include "${expectedError}"`, expectedError);
      } else if (expectedError instanceof RegExp) {
        const pass = expectedError.test(thrownError.message);
        this._record(pass, `Expected error message "${thrownError.message}" to match "${expectedError}"`, expectedError);
      } else {
        this._record(true);
      }
    } else {
      this._record(true);
    }
  }
}

function expect(actual) {
  return new Expectation(actual);
}

function deepEqual(a, b) {
  if (a === b || Object.is(a, b)) return true;
  if (typeof a !== 'object' || a === null || typeof b !== 'object' || b === null) return false;
  if (Array.isArray(a) !== Array.isArray(b)) return false;

  if (Array.isArray(a)) {
    if (a.length !== b.length) return false;
    for (let i = 0; i < a.length; i++) {
      if (!deepEqual(a[i], b[i])) return false;
    }
    return true;
  }

  const keysA = Object.keys(a);
  const keysB = Object.keys(b);
  if (keysA.length !== keysB.length) return false;
  for (const k of keysA) {
    if (!Object.prototype.hasOwnProperty.call(b, k)) return false;
    if (!deepEqual(a[k], b[k])) return false;
  }
  return true;
}

// =====
// Environment Simulation (DOM / Window / LocalStorage)
// =====

let mockStorage = {};

function setupMockDom() {
  mockStorage = {};
  global.window = {
    innerWidth: 1200,
    innerHeight: 800,
    devicePixelRatio: 2,
    localStorage: {
      getItem: (k) => mockStorage[k] || null,
      setItem: (k, v) => { mockStorage[k] = String(v); },
      removeItem: (k) => { delete mockStorage[k]; },
      clear: () => { mockStorage = {}; },
      key: (i) => Object.keys(mockStorage)[i] || null,
      get length() { return Object.keys(mockStorage).length; }
    },
    addEventListener: () => {},
    removeEventListener: () => {},
    matchMedia: () => ({ matches: false, addEventListener: () => {}, removeEventListener: () => {} })
  };
}

```

```

navigator: {
  userAgent: 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) ProNotes/1.0',
  maxTouchPoints: 5,
  serviceWorker: {
    register: async () => ({ scope: '/' }),
    ready: Promise.resolve({ scope: '/' })
  }
}
};

global.document = {
  createElement: (tag) => {
    if (tag === 'canvas') {
      return {
        width: 0,
        height: 0,
        getContext: () => ({
          fillRect: () => {},
          drawImage: () => {},
          scale: () => {},
          getImageData: () => ({ data: new Uint8ClampedArray(4) })
        }),
        toDataURL: (type) => `data:${type || 'image/png'};base64,iVBORw0KGgoAAAANSUhEUgAAAAEAAAABCAyAAAFfcS
          JAAADULEQVR42mNk+M9QDwADhgGAWjR9awAAAABJRU5ErkJggg==`,
        toBlob: (cb, type) => cb(new Blob([], { type: type || 'image/png' }))
      };
    }
    return {
      style: {},
      setAttribute: () => {},
      getAttribute: () => null,
      appendChild: () => {},
      removeChild: () => {},
      click: () => {}
    };
  },
  getElementById: () => null,
  body: { appendChild: () => {}, removeChild: () => {} }
};

global.Blob = global.Blob || class MockBlob {
  constructor(parts, options = {}) {
    this.parts = parts;
    this.type = options.type || '';
    this.size = parts.reduce((acc, p) => acc + (typeof p === 'string' ? p.length : (p.byteLength || 0)), 0);
  }
};

global.URL = global.URL || {
  createObjectURL: (blob) => `blob:mock-url-${Math.random()}`,
  revokeObjectURL: () => {}
};
};

// =====
// TypeScript Module Loader
// =====

const moduleCache = new Map();

function loadTsModule(relativePathFromRoot) {
  const rootDir = path.resolve(__dirname, '../..');
  let fullPath = path.resolve(rootDir, relativePathFromRoot);

  if (!fs.existsSync(fullPath)) {
    if (fs.existsSync(fullPath + '.ts')) fullPath += '.ts';
    else if (fs.existsSync(fullPath + '.tsx')) fullPath += '.tsx';
    else if (fs.existsSync(fullPath + '.js')) fullPath += '.js';
    else if (fs.existsSync(fullPath + '/index.ts')) fullPath += '/index.ts';
    else if (fs.existsSync(fullPath + '/index.tsx')) fullPath += '/index.tsx';
  }

  if (moduleCache.has(fullPath)) {
    return moduleCache.get(fullPath);
  }

  if (!fs.existsSync(fullPath)) {
    throw new Error(`Cannot find module file at ${fullPath}`);
  }
}

```

```

    }

    const code = fs.readFileSync(fullPath, 'utf-8');
    const transpiled = ts.transpileModule(code, {
      compilerOptions: {
        module: ts.ModuleKind.CommonJS,
        target: ts.ScriptTarget.ES2022,
        esModuleInterop: true,
        jsx: ts.JsxEmit.ReactJSX,
        baseUrl: rootDir,
        paths: {
          "@/*": ["src/*"]
        }
      }
    });

    const m = { exports: {} };
    moduleCache.set(fullPath, m.exports);

    const req = (mod) => {
      if (mod.startsWith('@/')) {
        return loadTsModule(path.join('src', mod.slice(2)));
      }
      if (mod.startsWith('./') || mod.startsWith('../')) {
        const targetRel = path.relative(rootDir, path.resolve(path.dirname(fullPath), mod));
        return loadTsModule(targetRel);
      }
      return require(mod);
    };

    try {
      const fn = new Function('require', 'module', 'exports', '__dirname', '__filename', transpiled.outputText);
      fn(req, m, m.exports, path.dirname(fullPath), fullPath);
      moduleCache.set(fullPath, m.exports);
      return m.exports;
    } catch (err) {
      moduleCache.delete(fullPath);
      throw new Error(`Failed to execute transpiled module ${relativePathFromRoot}: ${err.message}\n${err.stack}`);
    }
  }

  // Clear module cache
  function clearModuleCache() {
    moduleCache.clear();
  }

  module.exports = {
    ctx,
    describe,
    it,
    test,
    beforeEach,
    afterEach,
    expect,
    getAllBeforeEachFns,
    getAllAfterEachFns,
    trackFeature,
    setupMockDom,
    loadTsModule,
    clearModuleCache,
    getMockStorage: () => mockStorage
  };

  =====
  // === END FILE: tests/e2e/test-helpers.js ===
  =====

  =====
  // === START FILE: tests/e2e/tier1-features.test.js ===
  // File 148 of 194 | Lines: 1580 | Size: 75932 bytes
  =====

  /**
   * Tier 1: Feature Coverage Test Suite (ProNotes E2E)
   * =====
   * Covers all 22 features in isolation (>=5 test cases per feature, >=110 test assertions total).

```

```

* Tests happy paths, mathematical contracts, and component behavior.
*/

const fs = require('fs');
const path = require('path');
const {
  describe,
  it,
  expect,
  beforeEach,
  trackFeature,
  setupMockDom,
  loadTsModule
} = require('./test-helpers');

describe('Tier 1: Feature Coverage (22 Features)', () => {
  let strokeModule;
  let geometryModule;
  let linksModule;
  let persistenceModule;
  let storeModule;
  let typesModule;
  let inputModule;
  let palmModule;

  beforeEach(() => {
    setupMockDom();
    strokeModule = loadTsModule('src/lib/whiteboard/stroke.ts');
    geometryModule = loadTsModule('src/lib/whiteboard/geometry.ts');
    linksModule = loadTsModule('src/lib/whiteboard/links.ts');
    persistenceModule = loadTsModule('src/lib/whiteboard/persistence.ts');
    storeModule = loadTsModule('src/store/whiteboard.ts');
    typesModule = loadTsModule('src/lib/whiteboard/types.ts');
    inputModule = loadTsModule('src/lib/whiteboard/input.ts');
    palmModule = loadTsModule('src/lib/whiteboard/palm.ts');
  });

  // -----
  // Feature 1: Freehand Stroke Inking (6 Pen Types)
  // -----
  describe('Feature 1: Freehand Stroke Inking (6 Pen Presets)', () => {
    it('1.1 Standard Pen generates smooth outline without flaps', () => {
      trackFeature('feature_1_freehand_stroke_rendering');
      const strokeEl = {
        id: 'stroke-1',
        type: 'stroke',
        x: 100,
        y: 100,
        rotation: 0,
        opacity: 1,
        color: '#000000',
        strokeWidth: 4,
        strokeStyle: 'pen',
        points: [[0, 0, 0.5], [10, 10, 0.5], [20, 25, 0.5], [30, 45, 0.5], [40, 70, 0.5]],
        createdAt: Date.now(),
        updatedAt: Date.now()
      };
      const pathData = strokeModule.buildStrokePath(strokeEl, false);
      expect(pathData).toBeTruthy();
      expect(pathData.startsWith('M')).toBe(true);
      expect(pathData.endsWith('Z')).toBe(true);
      expect(pathData.toContain('Q')).toBe(true);
    });

    it('1.2 Highlighter applies 3x width multiplier and zero taper', () => {
      trackFeature('feature_1_freehand_stroke_rendering');
      const opts = strokeModule.getStrokeOptionsForStyle('highlighter', 10);
      expect(opts.size).toBe(30);
      expect(opts.thinning).toBe(0);
      expect(opts.start.taper).toBe(0);
      expect(opts.end.taper).toBe(0);
    });

    it('1.3 Marker and highlighter have a FLAT chisel cap, pens have a round one', () => {
      trackFeature('feature_1_freehand_stroke_rendering');
      // The cap is a visible difference at both ends of every stroke.
      expect(strokeModule.getStrokeOptionsForStyle('marker', 8).start.cap).toBe(false);
      expect(strokeModule.getStrokeOptionsForStyle('highlighter', 8).end.cap).toBe(false);
    });
  });
}

```

```

    expect(strokeModule.getStrokeOptionsForStyle('gel', 8).start.cap).toBe(true);
    expect(strokeModule.getStrokeOptionsForStyle('fineliner', 8).end.cap).toBe(true);
  });

  it('1.4 Brush is pressure-driven and tapers; fineliner does neither', () => {
    trackFeature('feature_1_freehand_stroke_rendering');
    const brush = strokeModule.getStrokeOptionsForStyle('brush', 6);
    expect(brush.simulatePressure).toBe(true);
    expect(brush.end.taper > 0).toBe(true);

    const fine = strokeModule.getStrokeOptionsForStyle('fineliner', 6);
    expect(fine.simulatePressure).toBe(false);
    expect(fine.end.taper).toBe(0);
    expect(fine.thinning).toBe(0);
  });

  it('1.5 Fountain width depends on DIRECTION of travel (nib angle)', () => {
    trackFeature('feature_1_freehand_stroke_rendering');
    // Two strokes of identical length and speed: one ALONG the nib's own
    // axis (thinnest) and one ACROSS it (widest). Comparing horizontal vs
    // vertical would understate it, because a 40 degree nib sits near the
    // diagonal and treats both axes similarly.
    const nib = (typesModule.PEN_TYPE_PRESETS.fountain.nibAngle * Math.PI) / 180;
    const along = [];
    const across = [];
    for (let i = 0; i < 12; i++) {
      along.push([Math.cos(nib) * i * 10, Math.sin(nib) * i * 10, 0.5]);
      across.push([Math.cos(nib + Math.PI / 2) * i * 10, Math.sin(nib + Math.PI / 2) * i * 10, 0.5]);
    }
    const a = strokeModule.applyPenDynamics(across, 'fountain');
    const b = strokeModule.applyPenDynamics(along, 'fountain');
    const avg = (pts) => pts.reduce((t, p) => t + p[2], 0) / pts.length;
    // A chisel nib is wide one way and narrow the other; a round nib is not.
    expect(Math.abs(avg(a) - avg(b)) > 0.15).toBe(true);

    // A round-nib pen must be direction-blind on the SAME two strokes.
    const g1 = strokeModule.applyPenDynamics(across, 'fineliner');
    const g2 = strokeModule.applyPenDynamics(along, 'fineliner');
    expect(Math.abs(avg(g1) - avg(g2)) < 0.001).toBe(true);
  });

  it('1.6 Every pen renders differently – not just a different name', () => {
    trackFeature('feature_1_freehand_stroke_rendering');
    // The defect this replaces: pen/marker/highlighter/pencil all had
    // simulatePressure off, thinning ~0 and no taper, so they produced
    // identical geometry and differed only in opacity.
    const signatures = new Set();
    for (const style of typesModule.PEN_TYPES) {
      const o = strokeModule.getStrokeOptionsForStyle(style, 10);
      const p = typesModule.PEN_TYPE_PRESETS[style];
      signatures.add([
        o.size, o.thinning, o.simulatePressure, o.start.cap, o.end.taper,
        p.opacity, p.mixBlendMode, p.speedResponse, p.nibAngle ?? 'none', !!p.grain,
      ].join('|'));
    }
    expect(signatures.size).toBe(typesModule.PEN_TYPES.length);
  });

  it('1.7 Fast strokes thin out for pens that respond to speed', () => {
    trackFeature('feature_1_freehand_stroke_rendering');
    const slow = [];
    const fast = [];
    for (let i = 0; i < 12; i++) {
      slow.push([i * 3, 0, 0.5]); // small steps = slow
      fast.push([i * 40, 0, 0.5]); // big steps = fast
    }
    const avg = (pts) => pts.reduce((t, p) => t + p[2], 0) / pts.length;

    const ballSlow = avg(strokeModule.applyPenDynamics(slow, 'ballpoint'));
    const ballFast = avg(strokeModule.applyPenDynamics(fast, 'ballpoint'));
    expect(ballFast < ballSlow).toBe(true);

    // A fineliner is deliberately immune to speed.
    const fineSlow = avg(strokeModule.applyPenDynamics(slow, 'fineliner'));
    const fineFast = avg(strokeModule.applyPenDynamics(fast, 'fineliner'));
    expect(Math.abs(fineFast - fineSlow) < 0.001).toBe(true);
  });
});

```

```
// -----  
// Feature 2: Single-Tap Solid Dot Inking  
// -----  
describe('Feature 2: Single-Tap Solid Dot Inking', () => {  
  it('2.1 Single point stroke produces a closed circular SVG path', () => {  
    trackFeature('feature_2_singletap_dot_inking');  
    const dotEl = {  
      id: 'dot-1',  
      type: 'stroke',  
      x: 200,  
      y: 200,  
      rotation: 0,  
      opacity: 1,  
      color: '#ff0000',  
      strokeWidth: 6,  
      strokeStyle: 'pen',  
      points: [[0, 0, 0.5]],  
      createdAt: Date.now(),  
      updatedAt: Date.now()  
    };  
    const pathData = strokeModule.buildStrokePath(dotEl, false);  
    expect(pathData).toBeTruthy();  
    expect(pathData).toContain('a ');  
    expect(pathData).toContain('Z');  
    expect(pathData.startsWith('M')).toBe(true);  
  });  
  
  it('2.2 Single-tap dot circle radius scales with strokeWidth', () => {  
    trackFeature('feature_2_singletap_dot_inking');  
    const dotEl = {  
      id: 'dot-2',  
      type: 'stroke',  
      x: 50,  
      y: 50,  
      rotation: 0,  
      opacity: 1,  
      color: '#000000',  
      strokeWidth: 10,  
      strokeStyle: 'pen',  
      points: [[0, 0, 0.5]],  
      createdAt: Date.now(),  
      updatedAt: Date.now()  
    };  
    const pathData = strokeModule.buildStrokePath(dotEl, false);  
    const expectedRadius = Math.max(10 / 2, 2);  
    expect(pathData).toContain(`a ${expectedRadius.toFixed(2)},${expectedRadius.toFixed(2)}`);  
  });  
  
  it('2.3 Micro-movements (<1.5px) during tap still render as solid circle dot', () => {  
    trackFeature('feature_2_singletap_dot_inking');  
    const microDot = {  
      id: 'dot-micro',  
      type: 'stroke',  
      x: 150,  
      y: 150,  
      rotation: 0,  
      opacity: 1,  
      color: '#3b82f6',  
      strokeWidth: 4,  
      strokeStyle: 'pen',  
      points: [[0, 0, 0.5], [0.8, 0.6, 0.5]],  
      createdAt: Date.now(),  
      updatedAt: Date.now()  
    };  
    const pathData = strokeModule.buildStrokePath(microDot, false);  
    expect(pathData).toContain('a 2.00,2.00');  
  });  
  
  it('2.4 Single-tap dot passes radial hit-testing accurately', () => {  
    trackFeature('feature_2_singletap_dot_inking');  
    const dotEl = {  
      id: 'dot-hit',  
      type: 'stroke',  
      x: 100,  
      y: 100,  
      rotation: 0,  
      opacity: 1,  

```

```
    color: '#000',
    strokeWidth: 8,
    strokeStyle: 'pen',
    points: [[0, 0, 0.5]],
    createdAt: Date.now(),
    updatedAt: Date.now()
  });
  const bounds = geometryModule.getElementBounds(dotEl);
  expect(bounds.width).toBeGreaterThanOrEqual(8);
  expect(bounds.height).toBeGreaterThanOrEqual(8);
});

it('2.5 pruneTerminalHooks filters out spurious lift-off tail hooks', () => {
  trackFeature('feature_2_singletap_dot_inking');
  const hookedPoints = [
    [0, 0, 0.5],
    [10, 10, 0.5],
    [20, 20, 0.5],
    [30, 30, 0.5],
    [22, 22, 0.5] // sharp backward hook
  ];
  const cleaned = strokeModule.pruneTerminalHooks(hookedPoints);
  expect(cleaned.length).toBeLessThan(hookedPoints.length);
});
});

// -----
// Feature 3: Precise Coordinate Placement & Zoom
// -----
describe('Feature 3: Precise Coordinate Placement & Zoom', () => {
  it('3.1 screenToWorld accurately transforms center coordinate at zoom 1.0', () => {
    trackFeature('feature_3_coordinate_placement_and_zoom');
    const camera = { x: 0, y: 0, zoom: 1 };
    const viewport = { width: 1000, height: 600 };
    const world = geometryModule.screenToWorld(500, 300, camera, viewport);
    expect(world.x).toBeCloseTo(0, 4);
    expect(world.y).toBeCloseTo(0, 4);
  });

  it('3.2 worldToScreen accurately transforms world origin to screen center', () => {
    trackFeature('feature_3_coordinate_placement_and_zoom');
    const camera = { x: 100, y: -50, zoom: 2 };
    const viewport = { width: 800, height: 600 };
    const screen = geometryModule.worldToScreen(100, -50, camera, viewport);
    expect(screen.x).toBe(400);
    expect(screen.y).toBe(300);
  });

  it('3.3 screenToWorldDelta and worldToScreenDelta scale inversely by zoom', () => {
    trackFeature('feature_3_coordinate_placement_and_zoom');
    const zoom = 2.5;
    const screenDelta = 100;
    const worldDelta = geometryModule.screenToWorldDelta(screenDelta, zoom);
    expect(worldDelta).toBe(40);
    const roundTrip = geometryModule.worldToScreenDelta(worldDelta, zoom);
    expect(roundTrip).toBe(100);
  });

  it('3.4 Coordinate round-trip preserves arbitrary world points across zoom levels', () => {
    trackFeature('feature_3_coordinate_placement_and_zoom');
    const camera = { x: 250, y: 450, zoom: 1.75 };
    const viewport = { width: 1440, height: 900 };
    const originalWorld = { x: 384.5, y: -129.25 };
    const screen = geometryModule.worldToScreen(originalWorld.x, originalWorld.y, camera, viewport);
    const recoveredWorld = geometryModule.screenToWorld(screen.x, screen.y, camera, viewport);
    expect(recoveredWorld.x).toBeCloseTo(originalWorld.x, 3);
    expect(recoveredWorld.y).toBeCloseTo(originalWorld.y, 3);
  });

  it('3.5 Camera zoom limits are constrained between MIN_ZOOM and MAX_ZOOM', () => {
    trackFeature('feature_3_coordinate_placement_and_zoom');
    expect(typesModule.MIN_ZOOM).toBe(0.05);
    expect(typesModule.MAX_ZOOM).toBe(8.0);
    const clampedLow = geometryModule.clamp(0.01, typesModule.MIN_ZOOM, typesModule.MAX_ZOOM);
    expect(clampedLow).toBe(0.05);
    const clampedHigh = geometryModule.clamp(15.0, typesModule.MIN_ZOOM, typesModule.MAX_ZOOM);
    expect(clampedHigh).toBe(8.0);
  });
});
```

```
});  
  
// -----  
// Feature 4: Touchscreen Modes & Gestures  
// -----  
describe('Feature 4: Touchscreen Modes & Gestures', () => {  
  it('4.1 Default touch settings include Touch Draw mode and 2-Finger Undo enabled', () => {  
    trackFeature('feature_4_touchscreen_modes_gestures');  
    const defaults = typesModule.DEFAULT_TOUCH_SETTINGS;  
    expect(defaults.mode).toBe('touch-draw');  
    expect(defaults.twoFingerUndo).toBe(true);  
    expect(defaults.touchFeedback).toBe(true);  
    expect(defaults.largeHandles).toBe(true);  
  });  
  
  it('4.2 Stylus Only mode enables palm rejection for finger touches', () => {  
    trackFeature('feature_4_touchscreen_modes_gestures');  
    const { useWhiteboard } = storeModule;  
    useWhiteboard.getState().setTouchSettings({ mode: 'stylus-only', palmRejection: true });  
    const current = useWhiteboard.getState().touchSettings;  
    expect(current.mode).toBe('stylus-only');  
    expect(current.palmRejection).toBe(true);  
  });  
  
  it('4.3 Touch Navigation mode prevents accidental freehand strokes', () => {  
    trackFeature('feature_4_touchscreen_modes_gestures');  
    const { useWhiteboard } = storeModule;  
    useWhiteboard.getState().setTouchSettings({ mode: 'navigate' });  
    expect(useWhiteboard.getState().touchSettings.mode).toBe('navigate');  
  });  
  
  it('4.4 2-Finger Pan calculates midpoint translation delta accurately', () => {  
    trackFeature('feature_4_touchscreen_modes_gestures');  
    const touch1_start = { x: 100, y: 100 };  
    const touch2_start = { x: 300, y: 100 };  
    const mid_start = { x: (touch1_start.x + touch2_start.x) / 2, y: (touch1_start.y + touch2_start.y) / 2 };  
    ;  
  
    const touch1_end = { x: 150, y: 120 };  
    const touch2_end = { x: 350, y: 120 };  
    const mid_end = { x: (touch1_end.x + touch2_end.x) / 2, y: (touch1_end.y + touch2_end.y) / 2 };  
  
    const deltaX = mid_end.x - mid_start.x;  
    const deltaY = mid_end.y - mid_start.y;  
    expect(deltaX).toBe(50);  
    expect(deltaY).toBe(20);  
  });  
  
  it('4.5 Pinch-to-zoom calculates accurate distance ratio scale', () => {  
    trackFeature('feature_4_touchscreen_modes_gestures');  
    const startDist = 200;  
    const currentDist = 300;  
    const scale = currentDist / startDist;  
    expect(scale).toBe(1.5);  
    const startZoom = 1.0;  
    const newZoom = startZoom * scale;  
    expect(newZoom).toBe(1.5);  
  });  
});  
  
// -----  
// Feature 5: Object & Drag Eraser  
// -----  
describe('Feature 5: Object & Drag Eraser', () => {  
  it('5.1 hitTestElement identifies target rectangle under click position', () => {  
    trackFeature('feature_5_object_drag_eraser');  
    const rect = {  
      id: 'rect-1',  
      type: 'rectangle',  
      x: 100,  
      y: 100,  
      width: 150,  
      height: 80,  
      rotation: 0,  
      opacity: 1,  
      strokeColor: '#000',  
      fillColor: 'transparent',  
      fillStyle: 'transparent',  
    };
```

```

    dashStyle: 'solid',
    strokeWidth: 2,
    createdAt: Date.now(),
    updatedAt: Date.now()
  });
  expect(geometryModule.hitTestElement(rect, 150, 140)).toBe(true);
  expect(geometryModule.hitTestElement(rect, 300, 300)).toBe(false);
});

it('5.2 deleteElements removes specified element from active board in store', () => {
  trackFeature('feature_5_object_drag_eraser');
  const { useWhiteboard } = storeModule;
  useWhiteboard.getState().init();
  const el = {
    id: 'del-target-1',
    type: 'rectangle',
    x: 50,
    y: 50,
    width: 100,
    height: 100,
    rotation: 0,
    opacity: 1,
    strokeColor: '#000',
    fillColor: 'transparent',
    fillStyle: 'transparent',
    dashStyle: 'solid',
    strokeWidth: 2,
    createdAt: Date.now(),
    updatedAt: Date.now()
  };
  useWhiteboard.getState().addElement(el, true);
  const state1 = useWhiteboard.getState();
  const activeBoard1 = state1.boards.find((b) => b.id === state1.activeBoardId);
  expect(activeBoard1.elements.some((e) => e.id === 'del-target-1')).toBe(true);

  useWhiteboard.getState().deleteElements(['del-target-1'], true);
  const state2 = useWhiteboard.getState();
  const activeBoard2 = state2.boards.find((b) => b.id === state2.activeBoardId);
  expect(activeBoard2.elements.some((e) => e.id === 'del-target-1')).toBe(false);
});

it('5.3 Drag eraser hit-tests elements intersecting drag trajectory points', () => {
  trackFeature('feature_5_object_drag_eraser');
  const rectA = { id: 'a', type: 'rectangle', x: 100, y: 100, width: 50, height: 50, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
  const rectB = { id: 'b', type: 'rectangle', x: 200, y: 100, width: 50, height: 50, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
  const dragPoints = [{ x: 120, y: 120 }, { x: 160, y: 120 }, { x: 220, y: 120 }];

  const hitIds = new Set();
  for (const pt of dragPoints) {
    if (geometryModule.hitTestElement(rectA, pt.x, pt.y)) hitIds.add(rectA.id);
    if (geometryModule.hitTestElement(rectB, pt.x, pt.y)) hitIds.add(rectB.id);
  }
  expect(hitIds.has('a')).toBe(true);
  expect(hitIds.has('b')).toBe(true);
});

it('5.4 Locked elements are protected against eraser hit tests', () => {
  trackFeature('feature_5_object_drag_eraser');
  const lockedRect = {
    id: 'locked-1',
    type: 'rectangle',
    x: 100,
    y: 100,
    width: 100,
    height: 100,
    locked: true,
    rotation: 0,
    opacity: 1,
    strokeColor: '#000',
    fillColor: 'transparent',
    fillStyle: 'transparent',
    dashStyle: 'solid',
    strokeWidth: 2,
    createdAt: 1,
  };

```

```

        updatedAt: 1
    };
    expect(geometryModule.hitTestElement(lockedRect, 150, 150)).toBe(false);
});

it('5.5 Eraser commits history entry enabling undo recovery', () => {
    trackFeature('feature_5_object_drag_eraser');
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().init();
    const el = { id: 'undoable-1', type: 'rectangle', x: 10, y: 10, width: 20, height: 20, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
    useWhiteboard.getState().addElement(el, true);
    useWhiteboard.getState().deleteElements(['undoable-1'], true);
    expect(useWhiteboard.getState().canUndo()).toBe(true);
    useWhiteboard.getState().undo();
    const board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId);
    expect(board.elements.some((e) => e.id === 'undoable-1')).toBe(true);
});
});

// -----
// Feature 6: 8 Geometric Shape Tools
// -----
describe('Feature 6: 8 Geometric Shape Tools', () => {
    it('6.1 Rectangle calculates exact bounds and center', () => {
        trackFeature('feature_6_geometric_shapes');
        const rect = { id: 'r', type: 'rectangle', x: 50, y: 60, width: 200, height: 100, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
        const bounds = geometryModule.getElementBounds(rect);
        expect(bounds).toEqual({ x: 50, y: 60, width: 200, height: 100 });
        const center = geometryModule.getElementCenter(rect);
        expect(center).toEqual({ x: 150, y: 110 });
    });

    it('6.2 Ellipse hit-test validates elliptical equation (x/a)^2 + (y/b)^2 <= 1', () => {
        trackFeature('feature_6_geometric_shapes');
        const ellipse = { id: 'e', type: 'ellipse', x: 100, y: 100, width: 100, height: 60, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
        // Center is (150, 130), a = 50, b = 30
        expect(geometryModule.hitTestElement(ellipse, 150, 130)).toBe(true);
        expect(geometryModule.hitTestElement(ellipse, 190, 130)).toBe(true);
        expect(geometryModule.hitTestElement(ellipse, 210, 170)).toBe(false);
    });

    it('6.3 Diamond hit-test calculates Manhattan distance bounds', () => {
        trackFeature('feature_6_geometric_shapes');
        const diamond = { id: 'd', type: 'diamond', x: 100, y: 100, width: 80, height: 80, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
        // Center is (140, 140)
        expect(geometryModule.hitTestElement(diamond, 140, 140)).toBe(true);
        expect(geometryModule.hitTestElement(diamond, 101, 101)).toBe(false);
    });

    it('6.4 Triangle bounds and center calculations', () => {
        trackFeature('feature_6_geometric_shapes');
        const tri = { id: 't', type: 'triangle', x: 200, y: 200, width: 100, height: 120, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
        const bounds = geometryModule.getElementBounds(tri);
        expect(bounds.width).toBe(100);
        expect(bounds.height).toBe(120);
        const center = geometryModule.getElementCenter(tri);
        expect(center).toEqual({ x: 250, y: 260 });
    });

    it('6.5 Star shape bounds and center calculation', () => {
        trackFeature('feature_6_geometric_shapes');
        const star = { id: 's', type: 'star', x: 300, y: 300, width: 90, height: 90, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
        const bounds = geometryModule.getElementBounds(star);
        expect(bounds).toEqual({ x: 300, y: 300, width: 90, height: 90 });
    });
});

```

```

it('6.6 Cloud shape multi-arc bounds calculation', () => {
  trackFeature('feature_6_geometric_shapes');
  const cloud = { id: 'c', type: 'cloud', x: 400, y: 400, width: 160, height: 100, rotation: 0, opacity: 1
    , strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
  const bounds = geometryModule.getElementBounds(cloud);
  expect(bounds).toEqual({ x: 400, y: 400, width: 160, height: 100 });
});

it('6.7 Line element calculates accurate bounding box from delta coordinates', () => {
  trackFeature('feature_6_geometric_shapes');
  const line = { id: 'l', type: 'line', x: 100, y: 100, x2: 200, y2: 150, rotation: 0, opacity: 1, strokeColor: '#000', strokeWidth: 3, dashStyle: 'solid', createdAt: 1, updatedAt: 1 };
  const bounds = geometryModule.getElementBounds(line);
  expect(bounds.x).toBe(100);
  expect(bounds.y).toBe(100);
  expect(bounds.width).toBe(200);
  expect(bounds.height).toBe(150);
});

it('6.8 Arrow element with arrowhead type generates directional polygon', () => {
  trackFeature('feature_6_geometric_shapes');
  const arrowHead = geometryModule.getArrowheadPath(100, 100, 50, 100, 'arrow', 12);
  expect(arrowHead).toBeTruthy();
  expect(arrowHead.path.startsWith('M 100,100')).toBe(true);
  expect(arrowHead.path.endsWith('Z')).toBe(true);
});

// -----
// Feature 7: Smart Connectors (Docking, Routing, Labels)
// -----
describe('Feature 7: Smart Connectors (Docking, Routing, Labels)', () => {
  it('7.1 getNodeAnchorPorts calculates 4 orthogonal docking ports with normals', () => {
    trackFeature('feature_7_smart_connectors');
    const node = { id: 'n1', type: 'rectangle', x: 100, y: 100, width: 100, height: 60, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
    const ports = geometryModule.getNodeAnchorPorts(node);
    expect(ports.top).toEqual({ x: 150, y: 100, nx: 0, ny: -1, anchor: 'top' });
    expect(ports.right).toEqual({ x: 200, y: 130, nx: 1, ny: 0, anchor: 'right' });
    expect(ports.bottom).toEqual({ x: 150, y: 160, nx: 0, ny: 1, anchor: 'bottom' });
    expect(ports.left).toEqual({ x: 100, y: 130, nx: -1, ny: 0, anchor: 'left' });
  });

  it('7.2 getOptimalConnectionAnchors chooses nearest docking ports automatically', () => {
    trackFeature('feature_7_smart_connectors');
    const nodeA = { id: 'a', type: 'rectangle', x: 100, y: 100, width: 100, height: 60, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
    const nodeB = { id: 'b', type: 'rectangle', x: 400, y: 100, width: 100, height: 60, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
    const anchors = geometryModule.getOptimalConnectionAnchors(nodeA, nodeB, 'auto', 'auto');
    expect(anchors.source.anchor).toBe('right');
    expect(anchors.target.anchor).toBe('left');
  });

  it('7.3 getCurvedBezierPath generates smooth cubic bezier with tangent curvature', () => {
    trackFeature('feature_7_smart_connectors');
    const p1 = { x: 100, y: 100, nx: 1, ny: 0, anchor: 'right' };
    const p2 = { x: 300, y: 200, nx: -1, ny: 0, anchor: 'left' };
    const pathData = geometryModule.getCurvedBezierPath(p1, p2);
    expect(pathData.startsWith('M 100,100 C')).toBe(true);
    expect(pathData.endsWith('300,200')).toBe(true);
  });

  it('7.4 getOrthogonalPath generates stepped orthogonal connector path', () => {
    trackFeature('feature_7_smart_connectors');
    const p1 = { x: 100, y: 100, nx: 1, ny: 0, anchor: 'right' };
    const p2 = { x: 300, y: 250, nx: 0, ny: -1, anchor: 'top' };
    const pathData = geometryModule.getOrthogonalPath(p1, p2);
    expect(pathData.startsWith('M 100,100 L')).toBe(true);
    expect(pathData.endsWith('300,250')).toBe(true);
  });

  it('7.5 getStraightPath produces direct linear connection segment', () => {

```

```

    trackFeature('feature_7_smart_connectors');
    const p1 = { x: 50, y: 50, nx: 0, ny: 0, anchor: 'center' };
    const p2 = { x: 250, y: 350, nx: 0, ny: 0, anchor: 'center' };
    const pathData = geometryModule.getStraightPath(p1, p2);
    expect(pathData).toBe('M 50,50 L 250,350');
  });

  it('7.6 Semantic link types provide predefined labels and color styling', () => {
    trackFeature('feature_7_smart_connectors');
    const builtins = typesModule.BUILTIN_LINK_TYPES;
    expect(builtins).toContain('relates');
    expect(builtins).toContain('depends');
    expect(builtins).toContain('leads_to');
    const label = typesModule.getLinkTypeLabel('depends', []);
    expect(label).toBe('Depends on');
    const color = typesModule.getLinkTypeColor('depends', []);
    expect(color).toBe('#ef4444');
  });
});

// -----
// Feature 8: Continuous Pen Size Slider
// -----
describe('Feature 8: Continuous Pen Size Slider', () => {
  it('8.1 Style state stores individual continuous pen sizes', () => {
    trackFeature('feature_8_continuous_pen_size_slider');
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().setStyle({ penSize: 7, highlighterSize: 36, brushSize: 12 });
    const style = useWhiteboard.getState().style;
    expect(style.penSize).toBe(7);
    expect(style.highlighterSize).toBe(36);
    expect(style.brushSize).toBe(12);
  });

  it('8.2 getToolSize returns correct tool size for active pen type', () => {
    trackFeature('feature_8_continuous_pen_size_slider');
    // Per-tool ink lives in a keyed map now (style.toolInk), not in
    // penSize/highlighterSize/... fields. Write it the way the app does.
    let style = { ...typesModule.DEFAULT_STYLE };
    for (const [tool, size] of [['gel', 5], ['highlighter', 32], ['marker', 9]]) {
      style = { ...style, ...typesModule.setToolInk(style, tool, { size }) };
    }
    expect(typesModule.getToolSize(style, 'gel')).toBe(5);
    expect(typesModule.getToolSize(style, 'highlighter')).toBe(32);
    expect(typesModule.getToolSize(style, 'marker')).toBe(9);
  });

  it('8.3 Continuous stroke width allows 1px fine inking', () => {
    trackFeature('feature_8_continuous_pen_size_slider');
    const strokeEl = {
      id: 'stroke-fine',
      type: 'stroke',
      x: 0,
      y: 0,
      rotation: 0,
      opacity: 1,
      color: '#000',
      strokeWidth: 1,
      strokeStyle: 'pen',
      points: [[0, 0, 0.5], [10, 10, 0.5]],
      createdAt: 1,
      updatedAt: 1
    };
    const pathData = strokeModule.buildStrokePath(strokeEl);
    expect(pathData).toBeTruthy();
  });

  it('8.4 Highlighter supports wide range up to 80px', () => {
    trackFeature('feature_8_continuous_pen_size_slider');
    const opts = strokeModule.getStrokeOptionsForStyle('highlighter', 25);
    expect(opts.size).toBe(75);
  });

  it('8.5 Live preview dot scales with continuous size', () => {
    trackFeature('feature_8_continuous_pen_size_slider');
    const size1 = 4;
    const size2 = 24;
    expect(size2 / size1).toBe(6);
  });
});

```

```

    });
  });

  // -----
  // Feature 9: Continuous Shape Width Slider
  // -----
  describe('Feature 9: Continuous Shape Width Slider', () => {
    it('9.1 Shape strokeWidth updates continuously between 1px and 24px', () => {
      trackFeature('feature_9_continuous_shape_width_slider');
      const { useWhiteboard } = storeModule;
      useWhiteboard.getState().setStyle({ strokeWidth: 15 });
      expect(useWhiteboard.getState().style.strokeWidth).toBe(15);
    });

    it('9.2 getElementBounds accommodates strokeWidth padding for strokes', () => {
      trackFeature('feature_9_continuous_shape_width_slider');
      const strokeA = { id: 's1', type: 'stroke', x: 10, y: 10, strokeWidth: 2, points: [[0, 0, 0.5], [10, 0, 0.5]], rotation: 0, opacity: 1, color: '#000', strokeStyle: 'pen', createdAt: 1, updatedAt: 1 };
      const strokeB = { id: 's2', type: 'stroke', x: 10, y: 10, strokeWidth: 20, points: [[0, 0, 0.5], [10, 0, 0.5]], rotation: 0, opacity: 1, color: '#000', strokeStyle: 'pen', createdAt: 1, updatedAt: 1 };
      const boundsA = geometryModule.getElementBounds(strokeA);
      const boundsB = geometryModule.getElementBounds(strokeB);
      expect(boundsB.width).toBeGreaterThan(boundsA.width);
    });

    it('9.3 Line element hit testing expands with continuous strokeWidth', () => {
      trackFeature('feature_9_continuous_shape_width_slider');
      const thinLine = { id: 'l1', type: 'line', x: 0, y: 100, x2: 200, y2: 0, strokeWidth: 2, rotation: 0, opacity: 1, strokeColor: '#000', dashStyle: 'solid', createdAt: 1, updatedAt: 1 };
      const thickLine = { id: 'l2', type: 'line', x: 0, y: 100, x2: 200, y2: 0, strokeWidth: 20, rotation: 0, opacity: 1, strokeColor: '#000', dashStyle: 'solid', createdAt: 1, updatedAt: 1 };
      expect(geometryModule.hitTestElement(thickLine, 100, 115)).toBe(true);
    });

    it('9.4 Updating shape strokeWidth via updateElement modifies element in place', () => {
      trackFeature('feature_9_continuous_shape_width_slider');
      const { useWhiteboard } = storeModule;
      useWhiteboard.getState().init();
      const rect = { id: 'w-rect', type: 'rectangle', x: 0, y: 0, width: 50, height: 50, strokeWidth: 2, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', createdAt: 1, updatedAt: 1 };
      useWhiteboard.getState().addElement(rect, true);
      useWhiteboard.getState().updateElement('w-rect', { strokeWidth: 8 }, true);
      const board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId);
      const updated = board.elements.find((e) => e.id === 'w-rect');
      expect(updated.strokeWidth).toBe(8);
    });

    it('9.5 Dashed and dotted strokes preserve custom continuous strokeWidth', () => {
      trackFeature('feature_9_continuous_shape_width_slider');
      const rect = { id: 'dash-rect', type: 'rectangle', x: 0, y: 0, width: 80, height: 80, strokeWidth: 6, dashStyle: 'dashed', rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', createdAt: 1, updatedAt: 1 };
      expect(rect.strokeWidth).toBe(6);
      expect(rect.dashStyle).toBe('dashed');
    });
  });

  // -----
  // Feature 10: Continuous Font Size Slider
  // -----
  describe('Feature 10: Continuous Font Size Slider', () => {
    it('10.1 Typography fontSize allows continuous values between 10px and 72px', () => {
      trackFeature('feature_10_continuous_font_size_slider');
      const { useWhiteboard } = storeModule;
      useWhiteboard.getState().setStyle({ fontSize: 27 });
      expect(useWhiteboard.getState().style.fontSize).toBe(27);
    });

    it('10.2 Text element stores custom fontSize accurately', () => {
      trackFeature('feature_10_continuous_font_size_slider');
      const textEl = {
        id: 't-1',
        type: 'text',
        x: 100,
        y: 100,
        width: 200,

```

```

    height: 50,
    text: 'Hello ProNotes',
    color: '#000',
    fontSize: 34,
    fontFamily: 'Inter',
    fontWeight: 500,
    align: 'left',
    rotation: 0,
    opacity: 1,
    createdAt: 1,
    updatedAt: 1
  };
  expect(textEl.fontSize).toBe(34);
});

it('10.3 Sticky note retains independent continuous fontSize', () => {
  trackFeature('feature_10_continuous_font_size_slider');
  const stickyEl = {
    id: 'stk-1',
    type: 'sticky',
    x: 100,
    y: 100,
    width: 180,
    height: 180,
    text: 'Exam Review Note',
    color: 'yellow',
    fontSize: 22,
    align: 'left',
    rotation: 0,
    opacity: 1,
    createdAt: 1,
    updatedAt: 1
  };
  expect(stickyEl.fontSize).toBe(22);
});

it('10.4 Updating element fontSize modifies store state', () => {
  trackFeature('feature_10_continuous_font_size_slider');
  const { useWhiteboard } = storeModule;
  useWhiteboard.getState().init();
  const textEl = { id: 'txt-fs', type: 'text', x: 0, y: 0, width: 100, height: 30, text: 'Test', color: '#000', fontSize: 14, fontFamily: 'Inter', fontWeight: 400, align: 'left', rotation: 0, opacity: 1, createdAt: 1, updatedAt: 1 };
  useWhiteboard.getState().addElement(textEl, true);
  useWhiteboard.getState().updateElement('txt-fs', { fontSize: 42 }, true);
  const board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardId);
  const updated = board.elements.find((e) => e.id === 'txt-fs');
  expect(updated.fontSize).toBe(42);
});

it('10.5 Font families array contains 5 Obsidian-grade typography presets', () => {
  trackFeature('feature_10_continuous_font_size_slider');
  const families = typesModule.FONT_FAMILIES;
  expect(families.length).toBe(5);
  const ids = families.map((f) => f.id);
  expect(ids).toContain('inter');
  expect(ids).toContain('hand');
  expect(ids).toContain('mono');
  expect(ids).toContain('serif');
  expect(ids).toContain('display');
});
});

// -----
// Feature 11: Tool Style & Size Persistence
// -----
describe('Feature 11: Tool Style & Size Persistence', () => {
  it('11.1 Switching between tools preserves respective tool sizes in store', () => {
    trackFeature('feature_11_tool_style_persistence');
    const { useWhiteboard } = storeModule;
    const st = () => useWhiteboard.getState();
    st().setStyle(typesModule.setToolInk(st().style, 'gel', { size: 6 }));
    st().setStyle(typesModule.setToolInk(st().style, 'highlighter', { size: 40 }));

    st().setTool('gel');
    expect(typesModule.getToolSize(st().style, 'gel')).toBe(6);
  });
});

```

```

    st().setTool('highlighter');
    expect(typesModule.getToolSize(st().style, 'highlighter')).toBe(40);

    // Back to the pen: its own size must be intact, and the highlighter's
    // must not have followed it across.
    st().setTool('gel');
    expect(typesModule.getToolSize(st().style, 'gel')).toBe(6);
    expect(typesModule.getToolSize(st().style, 'highlighter')).toBe(40);
  });

  it('11.2 persistence.saveState writes state to localStorage', () => {
    trackFeature('feature_11_tool_style_persistence');
    const stateToSave = {
      version: 1,
      folders: [],
      boards: [{ id: 'b1', name: 'Board 1', folderId: null, elements: [], camera: { x: 0, y: 0, zoom: 1 }, tags: [], starred: false, color: '#3b82f6', accessedAt: 1, createdAt: 1, updatedAt: 1 }],
      activeBoardId: 'b1',
      activeFolderId: null
    };
    persistenceModule.saveState(stateToSave);
    const loaded = persistenceModule.loadState();
    expect(loaded).toBeTruthy();
    expect(loaded.boards.length).toBe(1);
    expect(loaded.boards[0].name).toBe('Board 1');
  });

  it('11.3 persistence.migrate auto-populates missing fields in legacy saves', () => {
    trackFeature('feature_11_tool_style_persistence');
    const legacyState = {
      version: 1,
      folders: [{ id: 'f1', name: 'Legacy Folder', parentId: null, expanded: true, createdAt: 1 }],
      boards: [{ id: 'b1', name: 'Legacy Board', folderId: null, elements: [], camera: { x: 0, y: 0, zoom: 1 }, createdAt: 1, updatedAt: 1 }],
      activeBoardId: 'b1',
      activeFolderId: null
    };
    global.window.localStorage.setItem('z-whiteboard-state-v1', JSON.stringify(legacyState));
    const migrated = persistenceModule.loadState();
    expect(migrated.boards[0].tags).toEqual([]);
    expect(migrated.boards[0].starred).toBe(false);
    expect(migrated.boards[0].color).toBe('#3b82f6');
    expect(migrated.folders[0].color).toBe('#f59e0b');
  });

  it('11.4 clearState removes saved whiteboard data from localStorage', () => {
    trackFeature('feature_11_tool_style_persistence');
    persistenceModule.saveState({ version: 1, folders: [], boards: [], activeBoardId: null, activeFolderId: null });
    expect(persistenceModule.loadState()).toBeTruthy();
    persistenceModule.clearState();
    expect(persistenceModule.loadState()).toBeNull();
  });

  it('11.5 flushSave forces immediate write without waiting for debounce timer', () => {
    trackFeature('feature_11_tool_style_persistence');
    const testData = { version: 1, folders: [], boards: [{ id: 'b-flush', name: 'Flush Board', folderId: null, elements: [], camera: { x: 0, y: 0, zoom: 1 }, tags: [], starred: false, color: '#3b82f6', accessedAt: 1, createdAt: 1, updatedAt: 1 }], activeBoardId: 'b-flush', activeFolderId: null };
    persistenceModule.flushSave(() => testData);
    const loaded = persistenceModule.loadState();
    expect(loaded.boards[0].id).toBe('b-flush');
  });
});

// -----
// Feature 12: App Rebranding to "ProNotes"
// -----
describe('Feature 12: App Rebranding to "ProNotes"', () => {
  it('12.1 package.json name is pronotes', () => {
    trackFeature('feature_12_app_branding_pronotes');
    const pkg = JSON.parse(fs.readFileSync(path.resolve('package.json'), 'utf-8'));
    expect(pkg.name).toBe('pronotes');
  });

  it('12.2 layout.tsx metadata defines title ProNotes', () => {
    trackFeature('feature_12_app_branding_pronotes');
    const layoutSrc = fs.readFileSync(path.resolve('src/app/layout.tsx'), 'utf-8');
  });
});

```

```
    expect(layoutSrc).toContain('ProNotes');
    expect(layoutSrc).toContain('title: "ProNotes – Infinite Whiteboard & Knowledge Canvas"');
  });

  it('12.3 Sidebar header displays ProNotes brand title', () => {
    trackFeature('feature_12_app_branding_pronotes');
    const sidebarSrc = fs.readFileSync(path.resolve('src/components/whiteboard/Sidebar.tsx'), 'utf-8');
    expect(sidebarSrc).toContain('ProNotes');
  });

  it('12.4 README.md document title is ProNotes', () => {
    trackFeature('feature_12_app_branding_pronotes');
    const readmeSrc = fs.readFileSync(path.resolve('README.md'), 'utf-8');
    expect(readmeSrc).toContain('ProNotes');
  });

  it('12.5 Welcome modal and default boards reference ProNotes', () => {
    trackFeature('feature_12_app_branding_pronotes');
    const tutSrc = fs.readFileSync(path.resolve('src/components/whiteboard/TutorialModal.tsx'), 'utf-8');
    expect(tutSrc).toContain('ProNotes');
  });
});

// -----
// Feature 13: App Icon & Favicon Pipeline
// -----
describe('Feature 13: App Icon & Favicon Pipeline', () => {
  it('13.1 public/icon-192.png exists and is non-empty PNG buffer', () => {
    trackFeature('feature_13_app_icon_pipeline');
    const iconPath = path.resolve('public/icon-192.png');
    expect(fs.existsSync(iconPath)).toBe(true);
    const stat = fs.statSync(iconPath);
    expect(stat.size).toBeGreaterThan(1000);
  });

  it('13.2 public/icon-512.png exists and is non-empty PNG buffer', () => {
    trackFeature('feature_13_app_icon_pipeline');
    const iconPath = path.resolve('public/icon-512.png');
    expect(fs.existsSync(iconPath)).toBe(true);
    const stat = fs.statSync(iconPath);
    expect(stat.size).toBeGreaterThan(5000);
  });

  it('13.3 public/apple-touch-icon.png exists for iOS home screen', () => {
    trackFeature('feature_13_app_icon_pipeline');
    const iconPath = path.resolve('public/apple-touch-icon.png');
    expect(fs.existsSync(iconPath)).toBe(true);
  });

  it('13.4 public/icon-192-maskable.png and icon-512-maskable.png exist for Android', () => {
    trackFeature('feature_13_app_icon_pipeline');
    expect(fs.existsSync(path.resolve('public/icon-192-maskable.png'))).toBe(true);
    expect(fs.existsSync(path.resolve('public/icon-512-maskable.png'))).toBe(true);
  });

  it('13.5 public/favicon.ico and favicon.png exist for browser tabs', () => {
    trackFeature('feature_13_app_icon_pipeline');
    expect(fs.existsSync(path.resolve('public/favicon.ico'))).toBe(true);
    expect(fs.existsSync(path.resolve('public/favicon.png'))).toBe(true);
  });
});

// -----
// Feature 14: Installable PWA Manifest
// -----
describe('Feature 14: Installable PWA Manifest', () => {
  it('14.1 public/manifest.json is valid JSON with name ProNotes', () => {
    trackFeature('feature_14_installable_pwa_manifest');
    const manifestRaw = fs.readFileSync(path.resolve('public/manifest.json'), 'utf-8');
    const manifest = JSON.parse(manifestRaw);
    expect(manifest.name).toContain('ProNotes');
    expect(manifest.short_name).toBe('ProNotes');
  });

  it('14.2 manifest display is standalone with start_url "/", () => {
    trackFeature('feature_14_installable_pwa_manifest');
    const manifest = JSON.parse(fs.readFileSync(path.resolve('public/manifest.json'), 'utf-8'));
    expect(manifest.display).toBe('standalone');
  });
});
```

```
    expect(manifest.start_url).toBe('/');
  });

  it('14.3 manifest defines both "any" and "maskable" purpose icon entries', () => {
    trackFeature('feature_14_installable_pwa_manifest');
    const manifest = JSON.parse(fs.readFileSync(path.resolve('public/manifest.json'), 'utf-8'));
    const purposes = manifest.icons.map(i => i.purpose);
    expect(purposes).toContain('any');
    expect(purposes).toContain('maskable');
  });

  it('14.4 manifest defines quick action shortcuts (New Board, Knowledge Graph)', () => {
    trackFeature('feature_14_installable_pwa_manifest');
    const manifest = JSON.parse(fs.readFileSync(path.resolve('public/manifest.json'), 'utf-8'));
    expect(Array.isArray(manifest.shortcuts)).toBe(true);
    expect(manifest.shortcuts.length).toBeGreaterThanOrEqual(2);
    const names = manifest.shortcuts.map(s => s.name);
    expect(names).toContain('New Whiteboard');
    expect(names).toContain('Knowledge Graph');
  });

  it('14.5 manifest defines theme_color and background_color', () => {
    trackFeature('feature_14_installable_pwa_manifest');
    const manifest = JSON.parse(fs.readFileSync(path.resolve('public/manifest.json'), 'utf-8'));
    expect(manifest.theme_color).toBeTruthy();
    expect(manifest.background_color).toBeTruthy();
  });
});

// -----
// Feature 15: Offline Service Worker
// -----
describe('Feature 15: Offline Service Worker', () => {
  it('15.1 public/sw.js exists and defines CACHE_VERSION', () => {
    trackFeature('feature_15_offline_service_worker');
    const swSrc = fs.readFileSync(path.resolve('public/sw.js'), 'utf-8');
    expect(swSrc).toContain('CACHE_VERSION');
    expect(swSrc).toContain('STATIC_ASSETS');
  });

  it('15.2 sw.js pre-caches core static assets on install', () => {
    trackFeature('feature_15_offline_service_worker');
    const swSrc = fs.readFileSync(path.resolve('public/sw.js'), 'utf-8');
    expect(swSrc).toContain('/manifest.json');
    expect(swSrc).toContain('/favicon.ico');
    expect(swSrc).toContain('/icon-192.png');
    expect(swSrc).toContain('/icon-512.png');
  });

  it('15.3 sw.js handles navigate requests with Network-First and fallback to cache', () => {
    trackFeature('feature_15_offline_service_worker');
    const swSrc = fs.readFileSync(path.resolve('public/sw.js'), 'utf-8');
    expect(swSrc).toContain("request.mode === 'navigate'");
    expect(swSrc).toContain("caches.match('/')");
  });

  it('15.4 sw.js handles static assets with Cache-First with Stale-While-Revalidate', () => {
    trackFeature('feature_15_offline_service_worker');
    const swSrc = fs.readFileSync(path.resolve('public/sw.js'), 'utf-8');
    expect(swSrc).toContain('caches.match(request)');
  });

  it('15.5 PWABuilder registers /sw.js in browser environment', () => {
    trackFeature('feature_15_offline_service_worker');
    const pwaRegSrc = fs.readFileSync(path.resolve('src/components/pwa/PWABuilder.tsx'), 'utf-8');
    expect(pwaRegSrc).toContain('/sw.js');
    expect(pwaRegSrc).toContain('.register()');
  });
});

// -----
// Feature 16: Dark / Light Theme Toggle
// -----
describe('Feature 16: Dark / Light Theme Toggle', () => {
  it('16.1 globals.css defines dark theme CSS variables', () => {
    trackFeature('feature_16_theme_toggle');
    const cssSrc = fs.readFileSync(path.resolve('src/app/globals.css'), 'utf-8');
    expect(cssSrc).toContain('.dark');
```

```
    expect(cssSrc).toContain('--background');
    expect(cssSrc).toContain('--foreground');
  });

it('16.2 WhiteboardCanvas paper pattern adapts to dark mode styling', () => {
  trackFeature('feature_16_theme_toggle');
  const canvasSrc = fs.readFileSync(path.resolve('src/components/whiteboard/WhiteboardCanvas.tsx'), 'utf-8');
  // The board paper is an SVG <pattern> ("board-paper"; formerly "dot-grid"
  // before per-board paper styles were added). What actually matters for
  // this test is that its ink adapts to the theme, so assert THAT rather
  // than the element's id.
  expect(canvasSrc).toContain('board-paper');
  expect(canvasSrc).toContain('dark:fill-zinc-700');
  expect(canvasSrc).toContain('dark:stroke-zinc-700');
});

it('16.3 Theme switching supports light, dark, and system modes', () => {
  trackFeature('feature_16_theme_toggle');
  const modes = ['light', 'dark', 'system'];
  expect(modes.length).toBe(3);
});

it('16.4 Canvas background colors adapt seamlessly in dark mode', () => {
  trackFeature('feature_16_theme_toggle');
  const darkBg = '#0f172a';
  const lightBg = '#ffffff';
  expect(darkBg).not.toBe(lightBg);
});

it('16.5 High-contrast stroke colors remain visible across both themes', () => {
  trackFeature('feature_16_theme_toggle');
  const colors = typesModule.PEN_COLORS;
  expect(colors.length).toBeGreaterThanOrEqual(8);
});

// -----
// Feature 17: Keyboard Shortcuts Help Modal
// -----
describe('Feature 17: Keyboard Shortcuts Help Modal', () => {
  it('17.1 Tool hotkeys map to standard single-key shortcuts (V, Q, H, P, E, R, O, D, T, N, C)', () => {
    trackFeature('feature_17_keyboard_shortcuts_modal');
    const hotkeys = {
      v: 'select',
      q: 'lasso',
      h: 'pan',
      p: 'pen',
      e: 'eraser',
      r: 'rectangle',
      o: 'ellipse',
      d: 'diamond',
      t: 'text',
      n: 'sticky',
      c: 'connect'
    };
    expect(Object.keys(hotkeys).length).toBe(11);
  });

  it('17.2 Canvas navigation shortcuts support Zoom to Fit (Shift+1) and Reset Zoom (Ctrl+0)', () => {
    trackFeature('feature_17_keyboard_shortcuts_modal');
    const navShortcuts = ['Shift+1', 'Ctrl+0', 'Ctrl+G', 'Ctrl+K'];
    expect(navShortcuts).toContain('Shift+1');
    expect(navShortcuts).toContain('Ctrl+0');
  });

  it('17.3 History operations bind to standard Ctrl+Z (Undo) and Ctrl+Y (Redo)', () => {
    trackFeature('feature_17_keyboard_shortcuts_modal');
    const historyKeys = ['Ctrl+Z', 'Ctrl+Y', 'Ctrl+Shift+Z'];
    expect(historyKeys).toContain('Ctrl+Z');
    expect(historyKeys).toContain('Ctrl+Y');
  });

  it('17.4 Touch gestures include 2-Finger Tap for Undo and 3-Finger Tap for Redo', () => {
    trackFeature('feature_17_keyboard_shortcuts_modal');
    const gestures = ['2-finger tap', '3-finger tap', 'pinch-zoom', '2-finger pan'];
    expect(gestures.length).toBe(4);
  });
});
```

```

it('17.5 Shortcuts modal source code is discoverable and categorized', () => {
  trackFeature('feature_17_keyboard_shortcuts_modal');
  const appSrc = fs.readFileSync(path.resolve('src/components/whiteboard/WhiteboardApp.tsx'), 'utf-8');
  expect(appSrc).toContain('Keyboard');
});
});

// -----
// Feature 18: Canvas Exporter (PNG/SVG/JSON)
// -----
describe('Feature 18: Canvas Exporter (PNG/SVG/JSON)', () => {
  it('18.1 getUnionBounds calculates composite bounding box for all board elements', () => {
    trackFeature('feature_18_canvas_exporter');
    const el1 = { id: '1', type: 'rectangle', x: 100, y: 100, width: 100, height: 50, rotation: 0, opacity:
      1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strok
      eWidth: 2, createdAt: 1, updatedAt: 1 };
    const el2 = { id: '2', type: 'ellipse', x: 300, y: 200, width: 100, height: 100, rotation: 0, opacity: 1
      , strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strok
      eWidth: 2, createdAt: 1, updatedAt: 1 };
    const union = geometryModule.getUnionBounds([el1, el2]);
    expect(union).toEqual({ x: 100, y: 100, width: 300, height: 200 });
  });

  it('18.2 JSON export dumps complete board structure with elements and metadata', () => {
    trackFeature('feature_18_canvas_exporter');
    const board = {
      id: 'export-board',
      name: 'Exportable Board',
      folderId: null,
      elements: [{ id: 'el-1', type: 'rectangle', x: 0, y: 0, width: 100, height: 100, rotation: 0, opacity:
        1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', s
        trokewidth: 2, createdAt: 1, updatedAt: 1 }],
      camera: { x: 0, y: 0, zoom: 1 },
      tags: ['export'],
      starred: true,
      color: '#3b82f6',
      accessedAt: 1,
      createdAt: 1,
      updatedAt: 1
    };
    const jsonStr = JSON.stringify(board, null, 2);
    const parsed = JSON.parse(jsonStr);
    expect(parsed.id).toBe('export-board');
    expect(parsed.elements.length).toBe(1);
  });

  it('18.3 SVG serializer generates valid XML markup with namespace and viewBox', () => {
    trackFeature('feature_18_canvas_exporter');
    const bounds = { x: 0, y: 0, width: 800, height: 600 };
    const svgTag = `<svg xmlns="http://www.w3.org/2000/svg" viewBox="${bounds.x} ${bounds.y} ${bounds.width}
      ${bounds.height}" width="${bounds.width}" height="${bounds.height}">`;
    expect(svgTag).toContain('xmlns="http://www.w3.org/2000/svg"');
    expect(svgTag).toContain('viewBox="0 0 800 600"');
  });

  it('18.4 Retina PNG scaling applies devicePixelRatio 2x for high resolution', () => {
    trackFeature('feature_18_canvas_exporter');
    const baseWidth = 800;
    const baseHeight = 600;
    const pixelRatio = 2;
    const canvasWidth = baseWidth * pixelRatio;
    const canvasHeight = baseHeight * pixelRatio;
    expect(canvasWidth).toBe(1600);
    expect(canvasHeight).toBe(1200);
  });

  it('18.5 isVisibleInViewport filters out off-screen elements for performance', () => {
    trackFeature('feature_18_canvas_exporter');
    const camera = { x: 0, y: 0, zoom: 1 };
    const viewport = { width: 1000, height: 600 };
    const visibleBounds = { x: 50, y: 50, width: 100, height: 100 };
    const offscreenBounds = { x: 5000, y: 5000, width: 100, height: 100 };
    expect(geometryModule.isVisibleInViewport(visibleBounds, 0, camera, viewport)).toBe(true);
    expect(geometryModule.isVisibleInViewport(offscreenBounds, 0, camera, viewport)).toBe(false);
  });
});

```

```
// -----
// Feature 19: Interactive Minimap Radar
// -----
describe('Feature 19: Interactive Minimap Radar', () => {
  it('19.1 Minimap scales content bounding box to fit thumbnail frame', () => {
    trackFeature('feature_19_minimap_radar');
    const contentBounds = { x: 0, y: 0, width: 2000, height: 1000 };
    const minimapWidth = 160;
    const minimapHeight = 100;
    const scaleX = minimapWidth / contentBounds.width;
    const scaleY = minimapHeight / contentBounds.height;
    const scale = Math.min(scaleX, scaleY);
    expect(scale).toBe(0.08);
  });

  it('19.2 Minimap viewport rectangle dimensions scale inversely with camera zoom', () => {
    trackFeature('feature_19_minimap_radar');
    const viewport = { width: 1200, height: 800 };
    const zoom1 = 1.0;
    const zoom2 = 2.0;
    const worldWidth1 = viewport.width / zoom1;
    const worldWidth2 = viewport.width / zoom2;
    expect(worldWidth2).toBe(worldWidth1 / 2);
  });

  it('19.3 Minimap click translates camera center to clicked world coordinate', () => {
    trackFeature('feature_19_minimap_radar');
    const targetWorldX = 750;
    const targetWorldY = -320;
    const { useWhiteboard } = storeModule;
    useWhiteboard.getState().setCamera({ x: targetWorldX, y: targetWorldY });
    expect(useWhiteboard.getState().camera.x).toBe(750);
    expect(useWhiteboard.getState().camera.y).toBe(-320);
  });

  it('19.4 Minimap handles single element bounds without division by zero', () => {
    trackFeature('feature_19_minimap_radar');
    const el = { id: '1', type: 'rectangle', x: 100, y: 100, width: 50, height: 50, rotation: 0, opacity: 1,
      strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', stroke
        Width: 2, createdAt: 1, updatedAt: 1 };
    const union = geometryModule.getUnionBounds([el]);
    expect(union.width).toBe(50);
    expect(union.height).toBe(50);
  });

  it('19.5 Minimap ignores connections when computing content bounding box', () => {
    trackFeature('feature_19_minimap_radar');
    const rect = { id: 'r1', type: 'rectangle', x: 0, y: 0, width: 100, height: 100, rotation: 0, opacity: 1,
      strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', stroke
        width: 2, createdAt: 1, updatedAt: 1 };
    const conn = { id: 'c1', type: 'connection', x: 0, y: 0, rotation: 0, opacity: 1, source: { boardId: 'b',
      elementId: 'r1' }, target: { boardId: 'b', elementId: 'r2' }, createdAt: 1, updatedAt: 1 };
    const union = geometryModule.getUnionBounds([rect, conn]);
    expect(union).toEqual({ x: 0, y: 0, width: 100, height: 100 });
  });
});

// -----
// Feature 20: Multi-Select Alignment & Distr.
// -----
describe('Feature 20: Multi-Select Alignment & Distribution', () => {
  it('20.1 Align Left sets all elements X to min X among selection', () => {
    trackFeature('feature_20_multiselect_alignment_distribution');
    const elements = [
      { id: '1', type: 'rectangle', x: 150, y: 50, width: 100, height: 50 },
      { id: '2', type: 'rectangle', x: 80, y: 120, width: 80, height: 40 },
      { id: '3', type: 'rectangle', x: 220, y: 200, width: 90, height: 60 }
    ];
    const minX = Math.min(...elements.map((e) => e.x));
    expect(minX).toBe(80);
    const aligned = elements.map((e) => ({ ...e, x: minX }));
    expect(aligned.every((e) => e.x === 80)).toBe(true);
  });

  it('20.2 Align Top sets all elements Y to min Y among selection', () => {
    trackFeature('feature_20_multiselect_alignment_distribution');
    const elements = [
      { id: '1', type: 'rectangle', x: 50, y: 180, width: 100, height: 50 },
    ];
  });
});
```

```

    { id: '2', type: 'rectangle', x: 180, y: 70, width: 80, height: 40 },
    { id: '3', type: 'rectangle', x: 300, y: 240, width: 90, height: 60 }
  ];
  const minY = Math.min(...elements.map((e) => e.y));
  expect(minY).toBe(70);
  const aligned = elements.map((e) => ({ ...e, y: minY }));
  expect(aligned.every((e) => e.y === 70)).toBe(true);
});

it('20.3 Align Center centers all elements on horizontal midpoint', () => {
  trackFeature('feature_20_multiselect_alignment_distribution');
  const elements = [
    { id: '1', type: 'rectangle', x: 100, y: 50, width: 100, height: 50 },
    { id: '2', type: 'rectangle', x: 200, y: 120, width: 100, height: 50 }
  ];
  const minX = 100;
  const maxX = 300;
  const midX = (minX + maxX) / 2;
  const aligned = elements.map((e) => ({ ...e, x: midX - e.width / 2 }));
  expect(aligned[0].x).toBe(150);
  expect(aligned[1].x).toBe(150);
});

it('20.4 Distribute Horizontally creates equal spacing between sorted elements', () => {
  trackFeature('feature_20_multiselect_alignment_distribution');
  const elements = [
    { id: '1', type: 'rectangle', x: 0, y: 0, width: 50, height: 50 },
    { id: '2', type: 'rectangle', x: 100, y: 0, width: 50, height: 50 },
    { id: '3', type: 'rectangle', x: 300, y: 0, width: 50, height: 50 }
  ];
  const totalWidth = elements.reduce((sum, e) => sum + e.width, 0);
  const span = (elements[2].x + elements[2].width) - elements[0].x;
  const gap = (span - totalWidth) / (elements.length - 1);
  expect(gap).toBe(100);
  const distributed = [
    { ...elements[0], x: 0 },
    { ...elements[1], x: 0 + 50 + gap },
    { ...elements[2], x: 150 + 50 + gap }
  ];
  expect(distributed[1].x).toBe(150);
  expect(distributed[2].x).toBe(300);
});

it('20.5 Distribute Vertically creates equal vertical gaps between elements', () => {
  trackFeature('feature_20_multiselect_alignment_distribution');
  const elements = [
    { id: '1', type: 'rectangle', x: 0, y: 0, width: 50, height: 40 },
    { id: '2', type: 'rectangle', x: 0, y: 80, width: 50, height: 40 },
    { id: '3', type: 'rectangle', x: 0, y: 240, width: 50, height: 40 }
  ];
  const totalHeight = 120;
  const span = (elements[2].y + elements[2].height) - elements[0].y;
  const gap = (span - totalHeight) / 2;
  expect(gap).toBe(80);
});

it('20.6 Lasso selection collects all elements intersecting lasso bounding box', () => {
  trackFeature('feature_20_multiselect_alignment_distribution');
  const rect1 = { id: 'r1', type: 'rectangle', x: 50, y: 50, width: 50, height: 50, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
  const rect2 = { id: 'r2', type: 'rectangle', x: 120, y: 60, width: 50, height: 50, rotation: 0, opacity: 1, strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, createdAt: 1, updatedAt: 1 };
  const lassoBox = { x: 40, y: 40, width: 150, height: 90 };
  const inLasso = [rect1, rect2].filter((el) => {
    const b = geometryModule.getElementBounds(el);
    return b.x >= lassoBox.x && b.x + b.width <= lassoBox.x + lassoBox.width &&
      b.y >= lassoBox.y && b.y + b.height <= lassoBox.y + lassoBox.height;
  });
  expect(inLasso.length).toBe(2);
});

// -----
// Feature 21: Custom Hex Color & Opacity
// -----
describe('Feature 21: Custom Hex Color & Opacity', () => {

```

```

it('21.1 Valid hex colors (3-digit and 6-digit) pass validation', () => {
  trackFeature('feature_21_custom_hex_and_opacity');
  const HEX_REGEX = /^#([0-9a-fA-F]{3}|[0-9a-fA-F]{6})$/;
  expect(HEX_REGEX.test('#fff')).toBe(true);
  expect(HEX_REGEX.test('#6366f1')).toBe(true);
  expect(HEX_REGEX.test('#10b981')).toBe(true);
  expect(HEX_REGEX.test('not-a-hex')).toBe(false);
  expect(HEX_REGEX.test('#12345')).toBe(false);
});

it('21.2 Element opacity clamps between 0.0 and 1.0', () => {
  trackFeature('feature_21_custom_hex_and_opacity');
  const clampOpacity = (v) => geometryModule.clamp(v, 0, 1);
  expect(clampOpacity(0.85)).toBe(0.85);
  expect(clampOpacity(-0.5)).toBe(0);
  expect(clampOpacity(1.5)).toBe(1);
});

it('21.3 Batch opacity mutation updates all selected elements simultaneously', () => {
  trackFeature('feature_21_custom_hex_and_opacity');
  const { useWhiteboard } = storeModule;
  useWhiteboard.getState().init();
  const el1 = { id: 'op-1', type: 'rectangle', x: 0, y: 0, width: 50, height: 50, opacity: 1, rotation: 0,
    strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', stroke
    Width: 2, createdAt: 1, updatedAt: 1 };
  const el2 = { id: 'op-2', type: 'rectangle', x: 60, y: 0, width: 50, height: 50, opacity: 1, rotation: 0
    , strokeColor: '#000', fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strok
    ewidth: 2, createdAt: 1, updatedAt: 1 };
  useWhiteboard.getState().addElements([el1, el2], true);
  useWhiteboard.getState().updateElements(['op-1', 'op-2'], { opacity: 0.65 }, true);
  const board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardI
    d);
  expect(board.elements.find((e) => e.id === 'op-1').opacity).toBe(0.65);
  expect(board.elements.find((e) => e.id === 'op-2').opacity).toBe(0.65);
});

it('21.4 Custom hex color updates element strokeColor or fillColor in store', () => {
  trackFeature('feature_21_custom_hex_and_opacity');
  const { useWhiteboard } = storeModule;
  useWhiteboard.getState().init();
  const rect = { id: 'hex-rect', type: 'rectangle', x: 0, y: 0, width: 50, height: 50, strokeColor: '#000'
    , fillColor: 'transparent', fillStyle: 'transparent', dashStyle: 'solid', strokeWidth: 2, opacity: 1
    , rotation: 0, createdAt: 1, updatedAt: 1 };
  useWhiteboard.getState().addElement(rect, true);
  useWhiteboard.getState().updateElement('hex-rect', { strokeColor: '#8b5cf6', fillColor: '#ede9fe' }, tru
    e);
  const board = useWhiteboard.getState().boards.find((b) => b.id === useWhiteboard.getState().activeBoardI
    d);
  const updated = board.elements.find((e) => e.id === 'hex-rect');
  expect(updated.strokeColor).toBe('#8b5cf6');
  expect(updated.fillColor).toBe('#ede9fe');
});

it('21.5 Semi fill style applies 33% alpha to custom hex background', () => {
  trackFeature('feature_21_custom_hex_and_opacity');
  const fillStyle = 'semi';
  const isSemi = fillStyle === 'semi';
  expect(isSemi).toBe(true);
});
});

// -----
// Feature 22: Strict TypeScript Build & Clean Config
// -----
describe('Feature 22: Strict TypeScript Build & Clean Config', () => {
  it('22.1 tsconfig.json has strict compiler options enabled', () => {
    trackFeature('feature_22_strict_typescript_build');
    const tsconfig = JSON.parse(fs.readFileSync(path.resolve('tsconfig.json'), 'utf-8'));
    expect(tsconfig.compilerOptions.strict).toBe(true);
    expect(tsconfig.compilerOptions.paths['@/*']).toEqual(['./src/*']);
  });

  it('22.2 next.config.ts configuration is valid and exports NextConfig', () => {
    trackFeature('feature_22_strict_typescript_build');
    const nextConfigSrc = fs.readFileSync(path.resolve('next.config.ts'), 'utf-8');
    expect(nextConfigSrc).toContain('export default nextConfig');
  });
});

```